



TRAILHEAD

Guided Selling

Exercise Guide

Version 1.0

TABLE OF CONTENTS

Preface	6
Overview	6
What You Will Learn	6
Prerequisites	7
Building Guided Selling User Experiences	8
What is Guided Selling?	8
Guided Selling: OmniScript and Industries CPQ	8
Two Approaches to Building Guided Selling	9
Getting Started with Digital Commerce Guided Selling	9
Designing a Guided Selling Experience	11
Building a Guided Selling Experience	12
Exercise 8-1: Setting up a Sales Catalog	13
What are sales catalogs?	14
Task 1: Set up the Catalog	15
Task 2: Check the Products' Status	17
Task 3: Create Catalog Product Relationships	18
Exercise 8-2: Initializing and Populating the Cache	20
The API Cache	21
Task 1: Initialize and run cache jobs	23
Exercise 8-3: Configure a Guided Selling Experience in OmniScript	26
The Industries Digital Commerce User Experience	27
What is a LWC OmniScript?	27
What is the Digital Commerce sample app?	28
Task 1: Add your domain to the CORS setting page	31
Task 2: Add Domain URL to CSP Trusted Sites	35
Task 3: Open the DC LWC OmniScript sample app and deactivate unneeded components	37
Task 4: Explore the DC LWC OmniScript sample app	43

Task 5: Deploy the DC LWC OmniScript sample app	47
Exercise 8-4: Configuring the DC LWC Sample App	48
The Set Values OmniScript Component	49
How does dcOffersList work?	49
Task 1: Set a Context Id in the initialiseValues component	51
Task 2: Override the catalogCode for dcOffersList	54
Task 3: Activate the OmniScript	55
Task 4: Restore the original functionality of dcOffersList	57
Exercise 8-5: Add Context Rules to a Guided Selling Experience	58
What are Digital Commerce Cacheable API parameters?	59
How are context rules used in Digital Commerce?	60
How do you extend context rules for Digital Commerce?	62
Task 1: Add a context rule to one of your sales catalog products	63
Task 2: Regenerate the cache	64
Task 3: Add context rules to your guided selling experience	66
Exercise 8-6: Configure the Selling End and Launch Points of the Script	72
The Sample App Checkout Phase OmniScript	73
Creating and Submitting Orders	74
The Navigate Action	77
Task 1: Explore the Checkout OmniScript	78
Task 2: Remove the ContextId from initialiseValues	81
Task 3: Configure the end of script navigation	82
OmniStudio Actions	84
Task 4: Create an action to launch the script	85
Task 5: Test the guided selling OmniScript	89
Cart-Based Guided Selling Overview	92
Cart-Based Guided Selling Examples	92
Creating the Business Process Flow in OmniScript	92
Building a Cart-Based Guided Selling Experience	93
Exercise 8-7: Create the Order Header	95

Using CPQ in Guided Selling	96
CpqAppHandler and its methods	97
createCart Method	97
Task 1: Prepare the sample OmniScript	99
Task 2: Create the order header	103
Task 3: Set the Order Id value	108
Task 4: Set the Context Id value	109
Task 5: Test the CreateOrder remote action	111
Exercise 8-8: Get Data from the Shared Catalog	113
Getting Data from the Shared Catalog	114
Getting Products using getCartsProducts Method	114
Getting Specific Groups of Products	115
Task 1: Build portability into the script	121
Task 2: Get product data using getCartsProducts remote actions	123
Task 3: Test the getCartsProducts remote action	127
Task 4: Refine the getCartsProducts response using a query	128
Task 5: Test the query parameter	128
Exercise 8-9: Create Product Selection Steps	129
What is a selectable items component?	130
Adding Products with postCartsItems	130
Task 1: Configure product selection steps	132
Task 2: Test the guided selling experience	135
Exercise 8-10: Configure the Selling End and Launch Points of the Script	139
Submitting Orders using the checkout Method	140
Task 1: Configure order checkout	142
Task 2: Configure the end of script navigation	143
Task 3: Test the guided selling OmniScript	144
Exercise 8-11: Configure the Persistent Cart	146
What is the persistent cart?	147
Persistent Cart Features	147

Configuring the Persistent Cart	148
Task 1: Configure the persistent cart	149
Task 2: Test the persistent cart	154
What are vlcCart and vlcCart_Top and why are they important?	154
Task 3: Prepare the persistent cart's data JSON	156
Exercise 8-12: Selling Accessories Challenge	158
Task 1: Get accessory products from the shared catalog	159
Task 2: Create a step to select accessories	159
Task 3: Test the new accessories step	159
Task 4: Limit accessories to only iPhone accessories	159
Appendix A: Digital Commerce LWC OmniScript Sample App	160
Appendix B: Digital Commerce LWC Sample Checkout Lightning web components	166
Appendix C: Digital Commerce LWC Sample Checkout OmniStudio Integration Procedures	170
Yay! All done!	177

Preface

These training exercises are based on the Spring '22 release of Salesforce Industries Communications, Media, and Energy & Utilities Clouds. For additional information about the topics covered in this module, see the documentation available in Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

Overview

This module provides practical knowledge for creating guided selling experiences using OmniScript and Industries Digital Commerce Lightning web components. Throughout this guide, we will shorten the names of these tools to “Digital Commerce”, “Digital Commerce Lightning web components”, and “DC LWC OmniScript.” You will learn about the four phases of the user experience model, as well as how to test and configure Guided Selling experiences built with Digital Commerce Lightning web components in OmniScript Designer. As you progress through this training, you complete practical, hands-on lab exercises designed for use with a training playground provided by Salesforce Industries.

What You Will Learn

When you complete this training, you will be able to:

- Distinguish between the Browse, Configure, Cart, and Checkout phases of the Digital Commerce user experience and how they relate to the structure of guided selling
- Create guided selling buy flows using Digital Commerce Lightning web components
- Make basic catalog and functional modifications to a sample DC LWC OmniScript
- Qualify customers for certain products in your sales catalog by adding a component to the DC LWC OmniScript that allows you to use context rules with dcOffersList

Prerequisites

The prerequisites for this training include a solid understanding of basic Salesforce concepts and a working knowledge of OmniScript. You should also have a working knowledge of telecommunications, media, or energy and utilities industry business objectives. You should have a basic understanding of customer relationship management and configuration, pricing, and quoting. The prerequisite also includes completion of the following learning modules:

- Enterprise Product Catalog (EPC)
- Pricing
- Promotions
- Rules

Building Guided Selling User Experiences

What is Guided Selling?

Guided selling is a shopping experience with a set of steps that guides the user through the process of recommending, selecting, and ordering products. A guided selling interaction can be launched from almost anywhere by anyone, such as from a service console by a customer service agent, from an account page by a sales representative, or from a website by a customer.

Guided Selling: OmniScript and Industries CPQ

With the Digital Interaction Platform, you can build guided selling experiences using [OmniScript](#) and [Industries CPQ](#). OmniScript is a declarative scripting tool, meaning you create it with clicks, not code, and it allows you to build all sorts of guided interactions, including guided selling.

OmniScripts can be launched from almost anywhere. You can launch them using an OmniStudio Action button within the Salesforce Industries application, or from outside the Industries application, such as a customer website. When used in a guided selling context, OmniScripts can be configured to include the following:

- A series of steps to be taken during the interaction and instructional text to provide a better customer experience
- A list of products from your shared catalog that use the same pricing and rules used elsewhere in CPQ
- The ability to configure items in the cart
- An order summary and running total bar at the bottom of the cart

Two Approaches to Building Guided Selling

There are two different approaches to building a guided selling experience.

Cart-based guided selling experiences use cart-based APIs and Angular JS templates to feature a visually persistent cart as the user navigates through the order process. You can build cart-based guided selling experiences using Industries CPQ and OmniScript very quickly while leveraging your existing catalog products, rules, and pricing. This technology works well for internal processes, with known traffic patterns and logged in users.

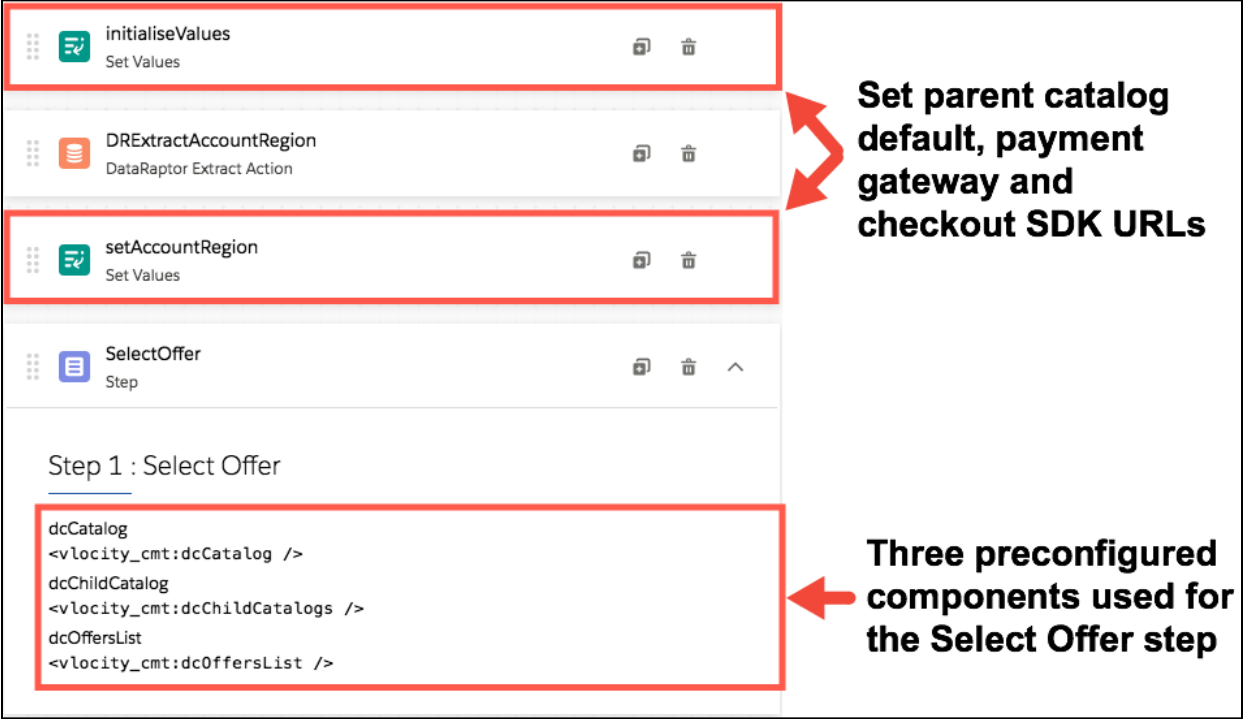
But most commerce companies also need to serve customers directly through their websites, which means unpredictable usage patterns. And unlike internal employees, 1 in 4 customers will abandon a process if it takes longer than 4 seconds to load, which impacts bounce rates, conversion rates, and return visitors. **Digital Commerce** guided selling experiences use cacheable APIs and Lightning web components to build a fast loading step-by-step order process.

Getting Started with Digital Commerce Guided Selling

Digital Commerce includes preconfigured web components and Lightning web components to build guided selling experiences. For this exercise, we will specifically examine the Digital Commerce Lightning web components app designed in OmniScript Designer.

To create a business process flow in the guided selling interaction, you can use the OmniScript components and custom Lightning web components included with the Digital Commerce LWC OmniScript Sample App, such as the following:

- A set values component to set the sales catalog, context Id, and context (i.e., in this exercise you will designate the customer's account region as the context)
- Custom Lightning web components that utilize preconfigured and designed Digital Commerce Lightning Web Components to build this buy flow
- A step component to group items into a segment of the interaction
- A navigate action component to redirect the user to their order record page after they have finished the buy flow



The screenshot displays a list of four preconfigured components in a Salesforce Guided Selling interface. The first three components are highlighted with red boxes and annotated with a red double-headed arrow pointing to a text box. The fourth component, 'SelectOffer', is also highlighted with a red box and annotated with a red arrow pointing to a text box. Below the components, the code for 'Step 1 : Select Offer' is shown, with the XML tags highlighted by a red box and annotated with a red arrow pointing to a text box.

Set parent catalog default, payment gateway and checkout SDK URLs

Three preconfigured components used for the Select Offer step

Step 1 : Select Offer

```
dcCatalog
<vlocity_cmt:dcCatalog />
dcChildCatalog
<vlocity_cmt:dcChildCatalogs />
dcOffersList
<vlocity_cmt:dcOffersList />
```

Designing a Guided Selling Experience

Before you begin designing your guided selling experience, it's important to consider the following questions.

1. **Who are you guiding?** With user-focused design, you always begin with understanding your user. Is it an internal user, such as a customer service representative? Or is it a customer?
2. **What products do you want to sell during the experience?** How should they be organized?
3. **What should the selling experience feel like?** Should it feel like part of the Industries application using standard templates? Or does it require your company's branding? Where will the guided selling experience begin? How will it end?



Building a Guided Selling Experience

With a design plan in hand, you can then begin configuring the guided selling experience.

1. **Build sales catalogs.** Thinking about the products you want to sell during the guided selling experience, how will you retrieve them from the shared catalog?
2. **Initialize the sales catalog cache.** Before you can configure your guided selling experience, are all of your product rules established and ready to be cached?
3. **Get data from the sales catalog.** Have you decided which sales catalog will be featured first in your selling experience?
4. **Modify sample app parameters.** How many products should load per page upon launch of the selling experience? Do customers need to be qualified? If so, by what context dimension?
5. **Configure the script end and launch points.** You need to configure where the user goes after completing the selling experience, and also a way to launch the guided selling script.



Exercise 8-1: Setting up a Sales Catalog

Scenario

Eliza is ready to create a brand-new Digital Commerce guided selling experience to prepare for the upcoming release of new mobile devices. Before she can start working on a guided selling experience, she needs to get the sales catalog ready. By creating a sales catalog with the new devices already associated with it, it will allow her to test the guided selling OmniScript later.

Goal

- Understand how sales catalogs are linked to the shared product catalog
- Determine how to create sales catalogs
- Create sales catalogs and product relationships

Tasks

1. Set up the Catalog
2. Check the Products' Status
3. Create Catalog Product Relationships

Time: 10 mins

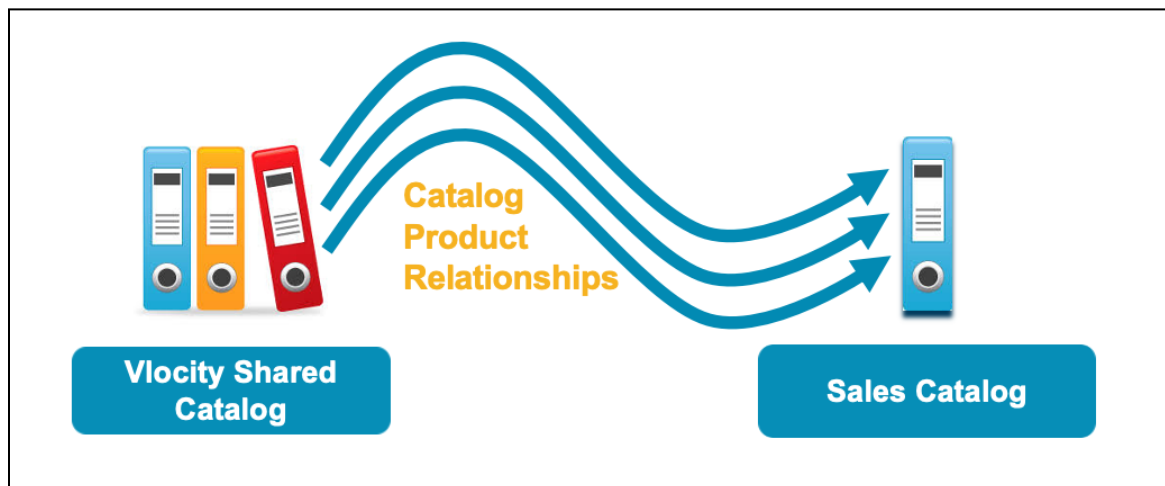


ALERT:

If you've just received your training playground, add your email address to the system administrator profile to ensure you receive all system notifications. In the upper-right, click on the **Avatar** and select **Settings**. Enter your email address in the **Email** field on the Personal Information page and click **Save**.

What are sales catalogs?

[Sales catalogs](#) are a subset of products/promotions from the Shared Catalog. Sales catalogs are ideal for Digital Commerce guided selling apps because they can be curated for specific product marketing strategies. In order to start building a guided selling experience, you will need to first create an ecommerce sales catalog.



Sales catalogs are made up of **Catalog Product Relationships**, which define products and promotions included in the shared catalog.

For this exercise, you will [create a catalog](#) and associate some products with it.

Task 1: Set up the Catalog

1. Using the Lightning App Launcher  , search for **Digital Commerce**, and then select it.
2. In the Lightning navigation bar, click **Catalogs**.
3. Click **New** in the top-right corner.
4. In the **New Catalog** dialog, enter the following information.

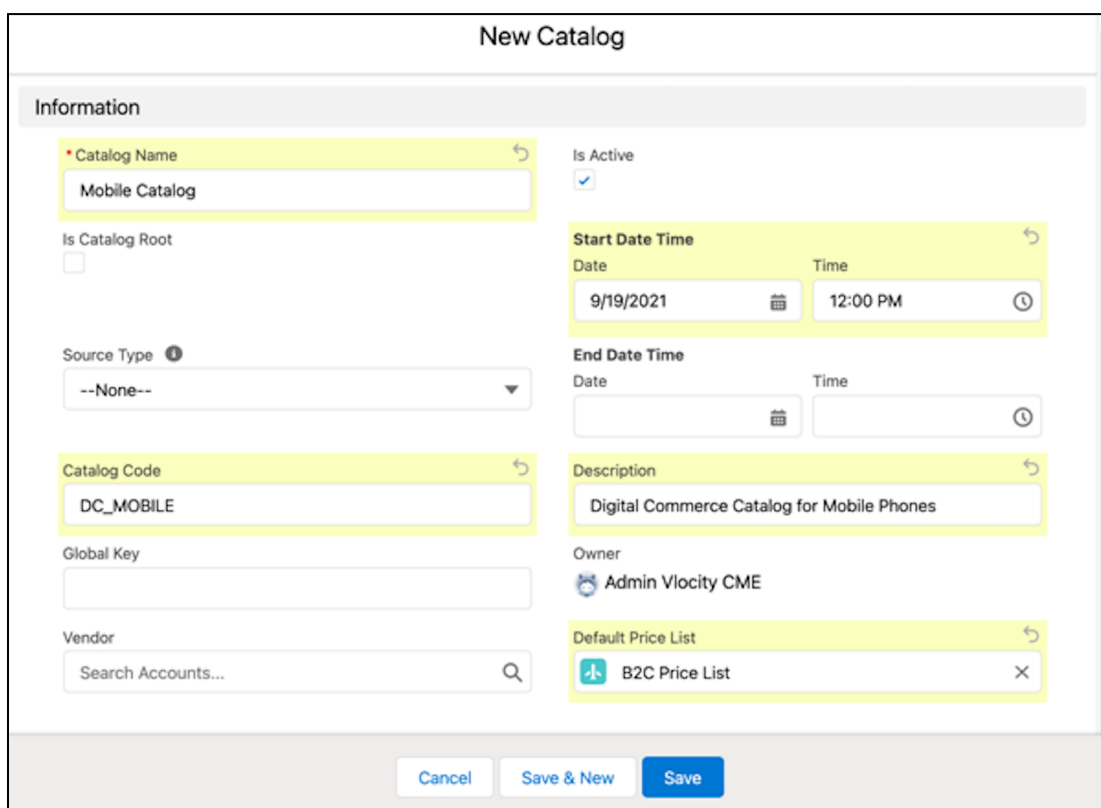
Field	Value	Notes
Catalog Name	Mobile Catalog	
Catalog Code	DC_MOBILE	Catalog codes are used by DC Web Components, SDK, and APIs in order to retrieve and display products to the UI. Salesforce Industries recommends using a DC_prefix to easily identify DC catalogs. This value is not required to be unique.
Is Active	✓	This field must be set to active as it is enforced.
Start Date Time	[yesterday's date]	The Start Date Time is enforced. If you select today's date, it will default to 12 PM which is midday. For that reason, you will select yesterday's date to ensure the catalog will be active right now.
Description	Digital Commerce Catalog for Mobile Phones	
Default Price List	B2C Price List	All of the products in a catalog should be on the same price list. This price list will be used

to look up the price list entry of the products that are included in the catalog.



NOTE:

The **Vendor**, **Is Catalog Root**, and **Source Type** fields are not currently used by the cacheable APIs and will not be reflected in your guided selling experience. The **Vendor** field can lookup an Account record for the sales catalog for reference purposes and has no impact on catalog products.



New Catalog

Information

* Catalog Name
Mobile Catalog

Is Active
☒

Start Date Time
Date: 9/19/2021 Time: 12:00 PM

End Date Time
Date: Time:

Source Type
--None--

Catalog Code
DC_MOBILE

Description
Digital Commerce Catalog for Mobile Phones

Owner
Admin Vlocity CME

Default Price List
B2C Price List

Cancel Save & New Save

5. Click **Save**.

Task 2: Check the Products' Status

Eliza's first priority is getting the new phones up on her new digital commerce guided selling experience. Before she adds them to her new digital commerce catalog, she needs to make sure that they are marked as active and have price entries in the shared catalog.

In this exercise, the products are already entered into the shared catalog for you.

1. In the Lightning navigation bar, click **Vlocity Product Console**.
2. In the Dashboard under Product Management, click the **search icon**  next to **Product**.
3. Click the **search icon** .
4. Ensure the following products are present and marked as **Active** in your training playground:
 - a. **Google Pixel2**
 - b. **Samsung Galaxy S10**



ALERT:

Products must be marked **active**, **orderable**, and have a valid **selling start/end date** otherwise they will not be returned in the API response and will not be cached.

5. Click on each product and check the **Pricing** facet to ensure it has at least one price entry defined for the **B2C Price List**.

Task 3: Create Catalog Product Relationships

1. In the Lightning navigation bar, click **Catalogs**.
2. Click **Mobile Catalog**.
3. Click the **Related** tab header.
4. Next to Catalog Product Relationships (0), click the **New** button.
5. In the **New Catalog Product Relationship** dialog, enter the following information.

Field	Value	Notes
Catalog Product Relationship Name	Google Pixel 2	
Catalog	Mobile Catalog	
Product	Google Pixel2	
Effective Date	[yesterday's date]	The Effective Date is enforced. You will select yesterday's date to ensure the catalog relationship will be active right now.
Is Active	✓	



NOTE:

The **Sequence Number** field can be used to manipulate which offers show first in the offer list. We will not be using it for this guided selling experience exercise.

6. Click **Save**.
7. Repeat the previous two steps for **Samsung Galaxy S10**. Eliza's first sales catalog is now created and has products associated with it.

 **Catalog**
Mobile Catalog

Related Details

 **Catalog Relationships (0)** [New](#)

 **Catalog Product Relationships (2)** [New](#)

Catalog Product Relationship Name	Product	Sequence Number
Google Pixel 2	Google Pixel2	
Samsung Galaxy S10	Samsung Galaxy S10	

[View All](#)



ALERT:

This catalog's products have not yet been cached and are therefore not yet available to the Digital Commerce cacheable APIs. You will learn how to populate the Digital Commerce cache in the next exercise.

Exercise 8-2: Initializing and Populating the Cache

Scenario

Eliza has a sales catalog with products and is almost ready to build her guided selling experience. Before she can do that, she needs to make sure her sales catalog's products are cached. She knows that performance is important to users and that the cacheable APIs will retrieve cached results quickly.

Goal

- Understand the benefits of using the API cache
- Initialize, populate, and refresh the cache

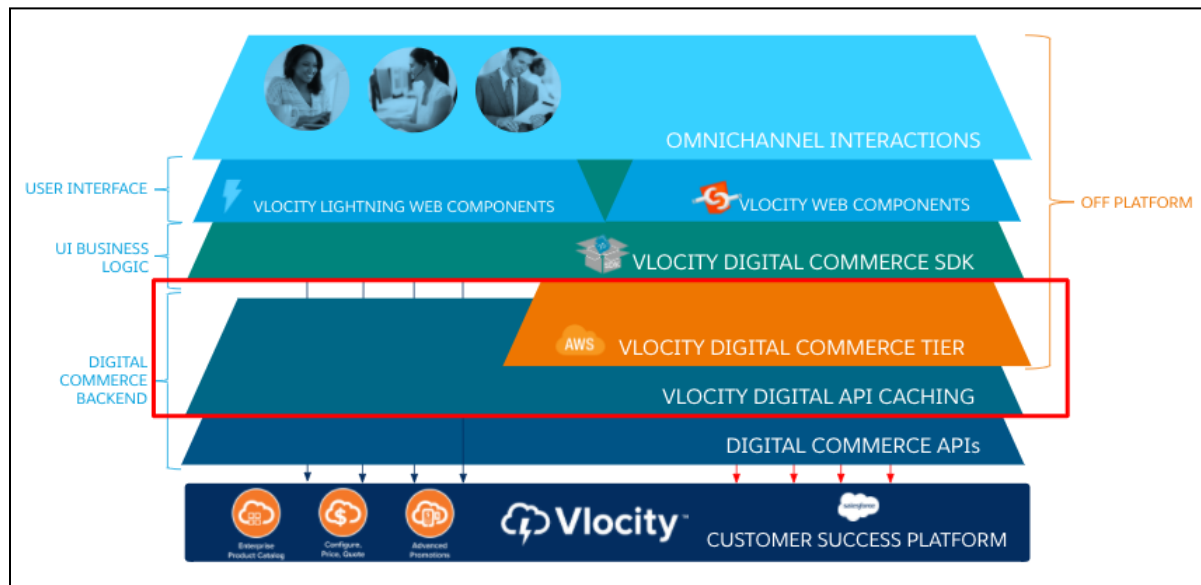
Tasks

1. Initialize and run cache jobs

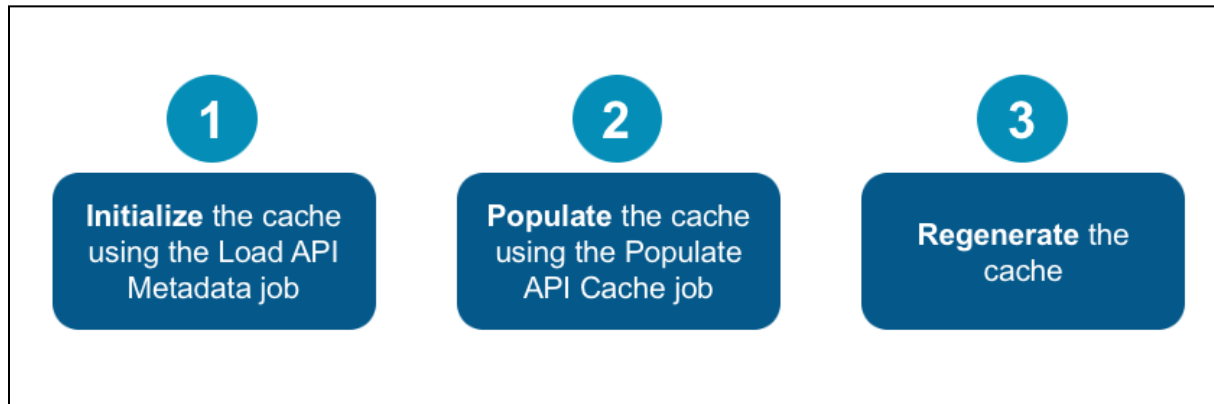
Time: 5 mins

The API Cache

In the digital commerce world, a large number of interactions are anonymous or nearly anonymous. The early stage of browsing catalogs and products lends itself to caching since many users receive the same responses. Therefore, intelligent caching strategies and cacheable APIs are critical in any enterprise-scale digital commerce solution.



Consequently, the Industries Digital Commerce solution includes a new [caching mechanism](#) called the Digital Commerce API Cache layer. When requests are made by users, Digital Commerce first refers to the Digital Commerce API Cache before making a new request. This cache stores pre-populated API responses inside Salesforce for on-platform solutions. In addition, the optional Digital Commerce Tier extends this [caching layer to AWS](#) for off-platform solutions. This caching design results in extremely high-performance response times and supports high-volume anonymous and nearly anonymous browsing traffic that is common on consumer-facing websites.

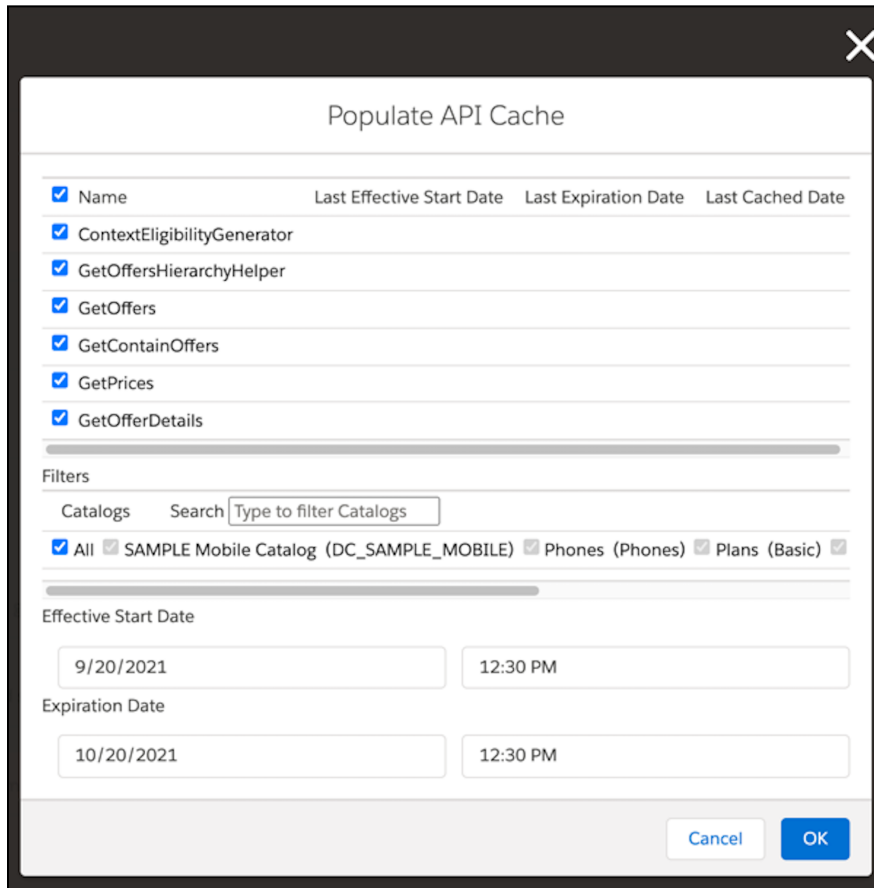


Catalog administrators have several cache management tasks that they must do to keep the cache up to date for digital commerce.

1. **Initialize the cache using the Load API MetaData job.** This job is run when first setting up Digital Commerce. It initializes the cacheable API Metadata sObject, which is a custom object that stores the metadata required by Digital Commerce APIs.
2. **Populate the cache using the Populate API Cache job.** This job populates the cache with products, product bundles, and promotions defined in the sales catalogs. In addition, context rules combinations, prices, offer details, and other data is also pre-populated and stored to ensure fast response times.
3. **Regenerate the cache.** When changes are made to any of the catalogs, products or promotions used in digital commerce, the cache must be regenerated.

Task 1: Initialize and run cache jobs

1. In the Lightning navigation bar, click **Vlocity CMT Administration**.
2. Below Admin Console, select **Cacheable API Jobs**.
3. Click the **Start** button next to **LOAD API METADATA**. This is considered initializing the cache. You only need to complete this step once for your org. Conversely, you will run the POPULATE API CACHE job every time you have made changes to your sales catalog products or if your cache has expired.
4. Click the **Start** button next to **POPULATE API CACHE**. This is considered populating the cache.
This process will populate the cache for all of the products in all of your active sales catalogs. It will create a “customer shape” or cached record for every valid result derived from any cached context rules that are applicable to your sales catalog products.
5. Select the **checkbox** at the top, which will then automatically select all of the items in the list.
6. Select the **checkbox** next to All so that the cache is populated for all of the sales catalogs in your training playground.



The dialog box is titled "Populate API Cache" and has a close button (X) in the top right corner. It contains a table with columns: Name, Last Effective Start Date, Last Expiration Date, and Last Cached Date. The table lists several API endpoints, all of which are checked. Below the table is a "Filters" section with a "Catalogs" label and a "Search" input field. Below the search field are four checkboxes: "All", "SAMPLE Mobile Catalog (DC_SAMPLE_MOBILE)", "Phones (Phones)", and "Plans (Basic)". Below the filters is an "Effective Start Date" section with two input fields: "9/20/2021" and "12:30 PM". Below that is an "Expiration Date" section with two input fields: "10/20/2021" and "12:30 PM". At the bottom right are "Cancel" and "OK" buttons.

Name	Last Effective Start Date	Last Expiration Date	Last Cached Date
☒ ContextEligibilityGenerator			
☒ GetOffersHierarchyHelper			
☒ GetOffers			
☒ GetContainOffers			
☒ GetPrices			
☒ GetOfferDetails			

Filters

Catalogs Search

☒ All ☒ SAMPLE Mobile Catalog (DC_SAMPLE_MOBILE) ☒ Phones (Phones) ☒ Plans (Basic)

Effective Start Date

Expiration Date

7. Select the **OK** button. It will take a few minutes for all of the cache jobs to finish running.

POPULATE API CACHE

This will process existing data and populate the API cache.

Job Name	Status	Total Batches	Batches Processed	Created Date	Completion Date
PromotionWrapper	Completed	0	0	9/20/21 6:55 PM	9/20/21 6:55 PM
GetOfferDetails	Completed	21	21	9/20/21 6:53 PM	9/20/21 6:55 PM
GetPrices	Completed	2	2	9/20/21 6:53 PM	9/20/21 6:53 PM
GetContainOffer	Completed	5	5	9/20/21 6:53 PM	9/20/21 6:53 PM
GetOffers	Completed	5	5	9/20/21 6:52 PM	9/20/21 6:53 PM
GetOffersItem	Completed	5	5	9/20/21 6:52 PM	9/20/21 6:52 PM
GetOffersHierarchy Helper	Completed	22	22	9/20/21 6:51 PM	9/20/21 6:52 PM
ContextEligibilityG enerator	Completed	5	5	9/20/21 6:51 PM	9/20/21 6:51 PM
RuleSetCombinatio nGenerator	Completed	7	7	9/20/21 6:51 PM	9/20/21 6:51 PM
EcomDataProfileG enerator	Completed	5	5	9/20/21 6:50 PM	9/20/21 6:51 PM



ALERT:

Any time you make changes to your sales catalog products (e.g., modifying a price entry, adding a context rule to a product), you must regenerate the cache. You will regenerate the cache in Exercise 8-5.

Exercise 8-3: Configure a Guided Selling Experience in OmniScript

Scenario

Eliza is ready to explore the sample app Salesforce Industries provided her. She just needs to launch it in OmniScript Designer and understand how it works.

Goal

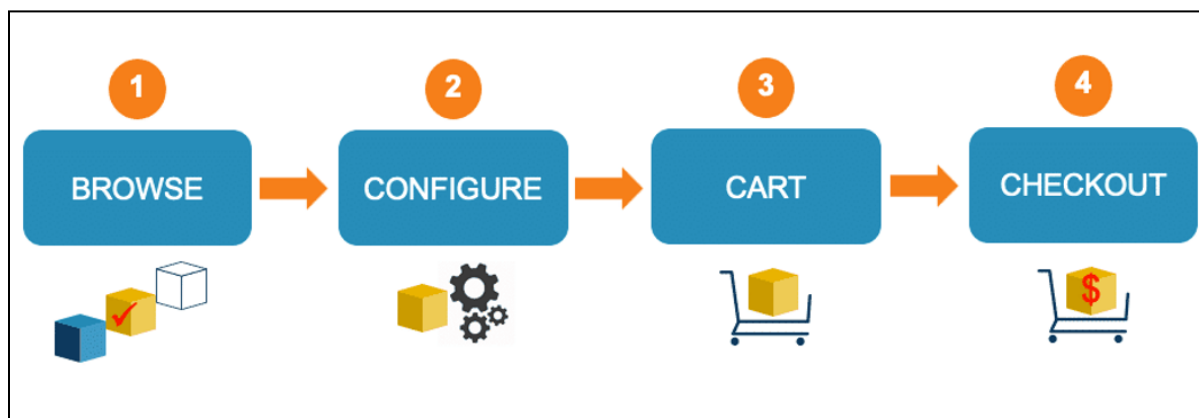
- Allow your domain in your org's CORS and CSP settings
- Launch the DC LWC OmniScript
- Make a minor modification to the Offers List Digital Commerce Lightning web component

Tasks

1. Add your domain to the CORS setting page
2. Add Domain URL to CSP Trusted Sites
3. Open the DC LWC OmniScript sample app and deactivate unneeded components
4. Explore the DC LWC OmniScript sample app
5. Deploy the DC LWC OmniScript sample app

Time: 25 mins

The Industries Digital Commerce User Experience



There are four distinct phases of a digital commerce experience. Digital Commerce Lightning web components support the same four phases, which are **Browse**, **Configure**, **Cart**, and **Checkout**. This model also helps you understand the general buy flow a user will go through when using a Digital Commerce guided selling experience.

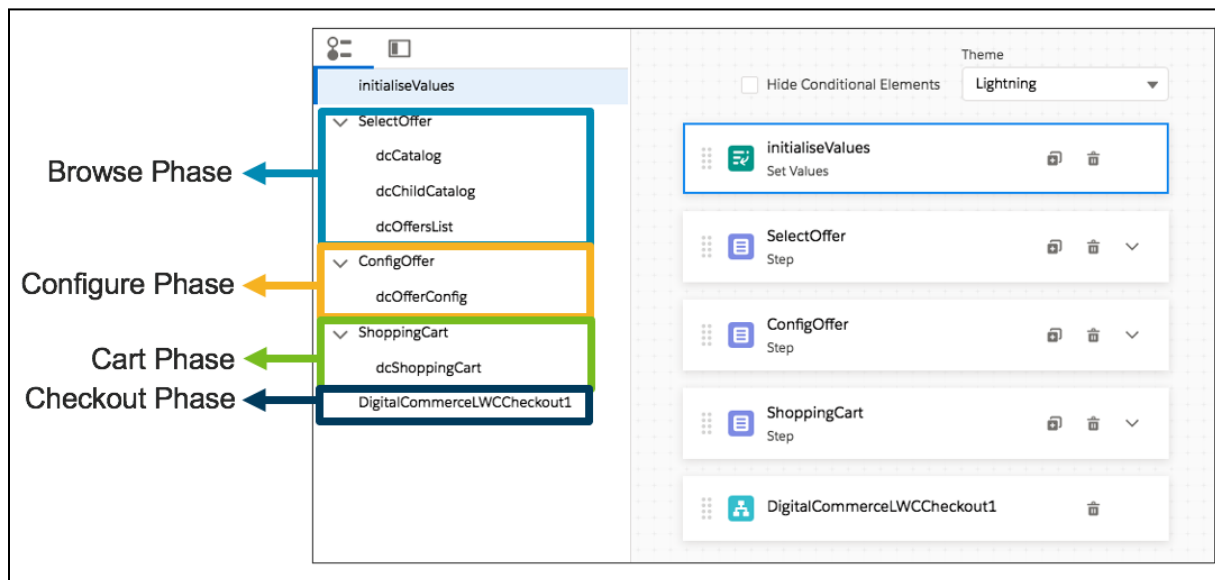
What is a LWC OmniScript?

LWC OmniScripts enable you to enhance an existing OmniScript element or add custom functionality to the OmniScript through the use of custom Lightning web components.

Use LWC OmniScripts when your digital commerce app is on-platform and your audience is known or guest users.

While it is possible to switch an OmniScript to either an LWC OmniScript or an Angular OmniScript at any time, Angular OmniScripts and LWC OmniScripts [do not support the same components and functionalities](#). For the purposes of this exercise, you will be using a LWC OmniScript designed exclusively for digital commerce.

What is the Digital Commerce sample app?

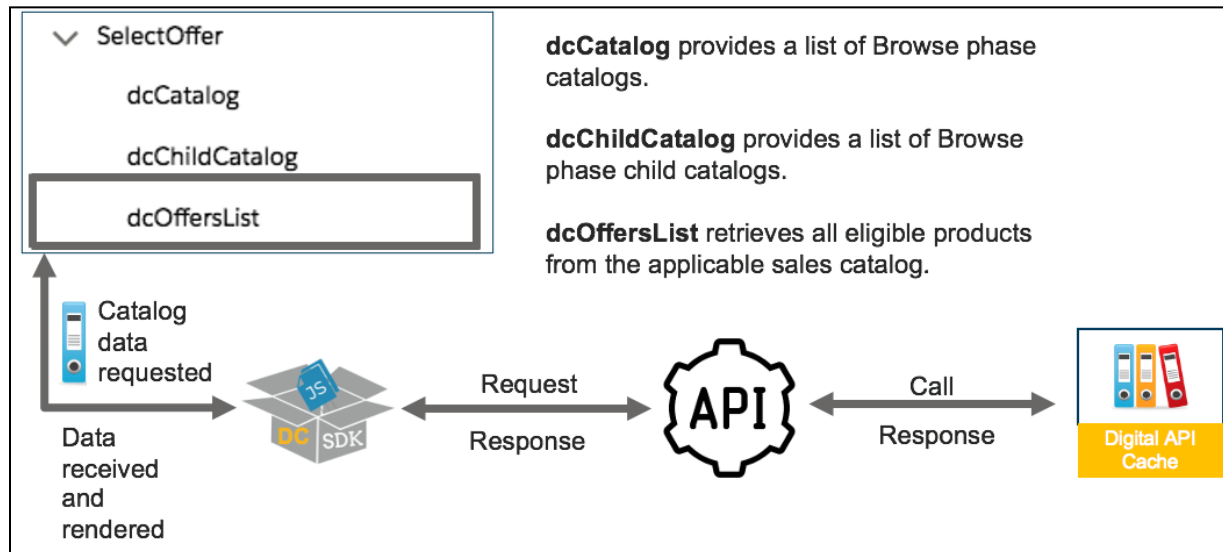


This is a high-level look at the DC LWC OmniScript Sample App. This app is running on the Salesforce platform and is designed for known (i.e., not anonymous) users with accounts in your training playground. At this time, we recommend an off-platform guided selling solution if you need to support anonymous users. This sample app is intended to be run on-platform.

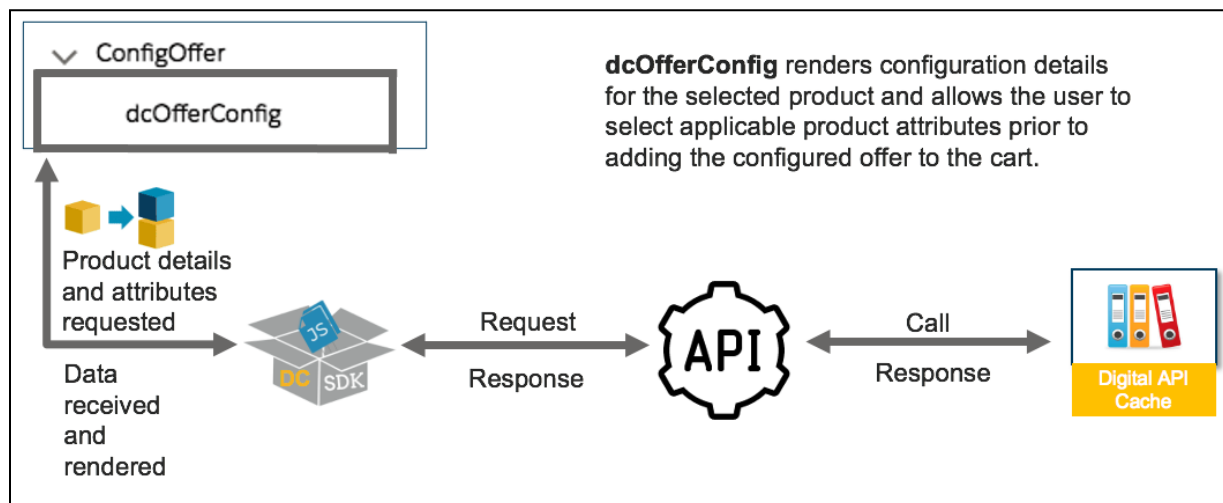


RESOURCE:

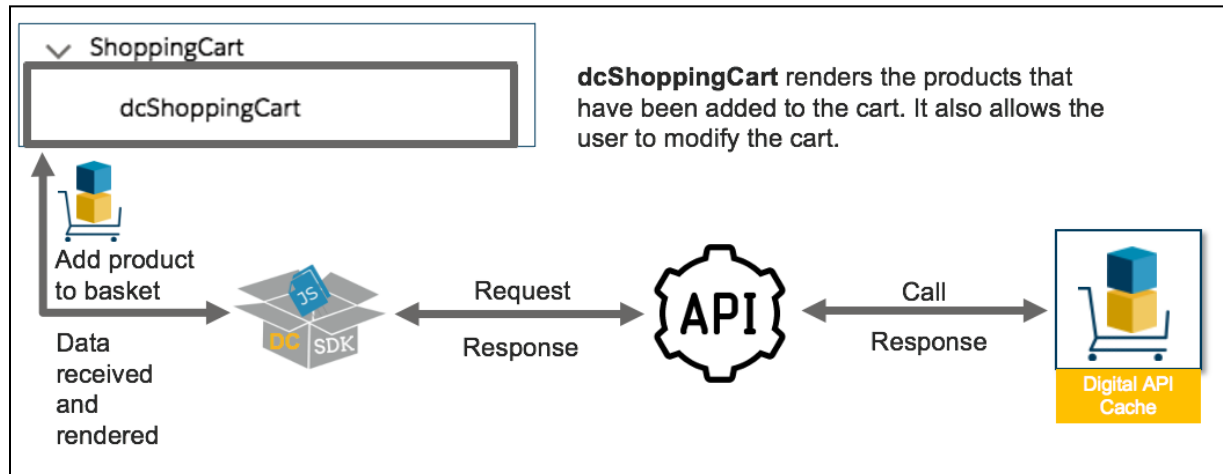
Please see the [Building User Experiences with DC Web Components](#) course for using Digital Commerce off-platform with web components.



First, users browse the available offers in the catalog. This is the **Browse** phase. The Digital Commerce solution supports this phase by giving you home page and catalog Lightning web components to retrieve and display catalog offers and individual offer details.



Second, users may select an offer and configure it, which may change the pricing and availability, this is the **Configure** phase. Digital Commerce Lightning web components support this phase by offering ways to render offer pricing, colors, other attributes, and media. In this phase, the virtual "basket" is not shown to the user.



Third, users may add the configured offer to their cart (a container of saved line items), and we'll call this the Cart phase. The Digital Commerce Lightning web components here allows the user these functions as well as render the cart (virtual "basket") itself. The cart will show the total price and allow the user to proceed to checkout (i.e., initialize the checkout process by creating an order from the cart's contents).

We will review the Checkout phase in the final exercise.

Task 1: Add your domain to the CORS setting page

CORS (Cross-Origin Resource Sharing) is a security policy that allows data requests made by your browser (i.e., the client) to other domains and sub-domains. It will also block potentially malicious requests. It is triggered whenever you are making an HTTP (i.e., GET, POST) request (i.e., cacheable API calls).

The VPL **Digital Commerce Explore & Checkout using Digital Commerce LWC OmniScript** sample app, and likely any app that you build, will utilize GET and POST cacheable API calls to retrieve and write to Salesforce. Salesforce apps inherently use several sub-domains based on your org's domain name, such as visualforce.com and force.com, that necessitate the use of CORS and CSP allowlists.

For legacy reasons, the CORS setting page has maintained the name “whitelist”, though all other instances in our training no longer use the term “whitelist.” Throughout this training that follows, “allowlist” is substituted for “whitelist”. A longer-term solution is being worked on.

Because you will be testing components that load cached resources (e.g., products) from origins other than their own, you will need to add your domain to the CORS allowlist. If you do not set up the CORS allowlist, the DC LWC OmniScript sample app will not load in OmniScript Designer Preview. If you do not set up the CORS allowlist, you will receive similar error messages to what is shown below in your [Chrome DevTools Console](#).

CSP (Content Security Policy) is another security policy to block the loading of potentially malicious scripts from unexpected sites. The Lightning component framework's main objective with CSP, is to help prevent cross-site scripting ([XSS](#)) and other code injection attacks. CSP header directives allow Lightning components to load resources, such as images, styles, and fonts, for the app.

```
Access to XMLHttpRequest at 'https://sushil010-tso.my.salesforce.com/services/apexrest/vlocity_cmt/v1/u.Toast_OmniscryptlwcPreview_fdcIFrameHost=web
SuccessTitle,DCChildProductFailure,DCAvailablePromotions,DCApplPromo' from origin 'https://sushil010-tso-vlocity-cmt.visualforce.com' has been
blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

1. Click the **gear icon**  on the top-right corner of the page, and then select **Setup**.

- In the **Quick Find** box, type `CORS`. Click **CORS**. Over the following steps in this task, you will add three URLs to your CORS allowlist.
- Select the **New** button within the **Allowed Origins List**. Enter the following information.

Field	Value	Notes
Origin URL Pattern	<code>https://[DOMAIN_URL].lightning.force.com</code>	Copy and paste your domain URL from the browser address bar into the Origin URL Pattern box. For example, your URL may look like this: <code>https://student9999-cme.lightning.force.com</code>

- Click **Save**.
- In the **Quick Find** box, type `CORS`. Click **CORS**.
- Select the **New** button. Enter the following information.

Field	Value	Notes
Origin URL Pattern	<code>https://[DOMAIN_URL]--c.visualforce.com</code>	Copy and paste your domain URL from the browser address bar into the Origin URL Pattern box. Delete <code>.lightning.force.com</code> from the URL you just copied and pasted. Copy and paste the following to the end of the URL: <code>--c.visualforce.com</code> For example, your URL may look like this: <code>https://student9999-cme--c.visualforce.com</code>

7. Click **Save**.
8. In the **Quick Find** box, type `CORS`. Click **CORS**.
9. Select the **New** button. Enter the following information.

Field	Value	Notes
Origin URL Pattern	<code>https://[DOMAIN_URL]--vlocity-cmt.visualforce.com</code>	Copy and paste your domain URL from the browser address bar into the Origin URL Pattern box. Delete <code>.lightning.force.com</code> from the URL you just copied and pasted. Copy and paste the following to the end of the URL: <code>--vlocity-cmt.visualforce.com</code> For example, your URL may look like this: <code>https://student9999-cme--vlocity-cmt.visualforce.com</code>

10. Click **Save**.



SETUP

CORS

CORS

Help for this Page

This page lists origins that are allowed for cross-origin resource sharing (CORS).

To allow code (such as JavaScript) running in a Web browser to communicate with Salesforce from a specific origin, add the origin to the allowed list.

View: All [Create New View](#)

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other **All**

Allowed Origins List

New

Action	Origin URL Pattern ↑	Created By	Created Date	Last Modified By	Last Modified Date
Edit Del	https://knowledge-digital-31735--c.visualforce.com	SShar	9/21/2021, 12:32 AM	SShar	9/21/2021, 12:32 AM
Edit Del	https://knowledge-digital-31735--vlocity-cmt.visualforce.com	SShar	9/21/2021, 12:34 AM	SShar	9/21/2021, 12:34 AM
Edit Del	https://knowledge-digital-31735.lightning.force.com	SShar	9/21/2021, 12:31 AM	SShar	9/21/2021, 12:31 AM
Edit Del	https://vu-cme--vlocity-cmt.visualforce.com	SShar	9/1/2021, 7:40 PM	SShar	9/1/2021, 7:40 PM
Edit Del	https://vu-cme--vlocity.cmt.visualforce.com	SShar	9/1/2021, 7:40 PM	SShar	9/1/2021, 7:40 PM

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other **All**

Cross-Origin Resource Sharing (CORS) Policy Settings

Edit

☐ Enable CORS for OAuth endpoints

Task 2: Add Domain URL to CSP Trusted Sites

Similarly, to CORS, you need to allow your site domain under your org's Content Security Policy (CSP) Trusted Sites page.

CSP (Content Security Policy) is a tool to prevent the loading of potentially malicious scripts from unexpected sites. The Lightning Component framework's main objective with CSP, is to help prevent cross-site scripting (XSS) and other code injection attacks. CSP header directives allow Lightning components to load resources, such as images, styles, and fonts, for the app.

1. In the **Quick Find** box, type **CSP**. Click **CSP Trusted Sites**.
2. Click the **New Trusted Site** button.

Field	Value	Notes
Trusted Site Name	My_domain_custom_namespace	
Origin URL Pattern	<code>https://[DOMAIN_URL]--c.my.salesforce.com</code>	Copy and paste your domain URL from the browser address bar into the Origin URL Pattern box. Delete <code>.lightning.force.com</code> from the URL you just copied and pasted. Copy and paste the following to the end of the URL: <code>--c.my.salesforce.com</code> For example, your URL may look like this: <code>https://student9999-cme--c.my.salesforce.com</code>

3. Select every **unchecked checkbox**  below the **CSP Directives** heading. It should look like the image below when you are done.

CSP Directives

Select at least one directive.

- Allow site for connect-src 
- Allow site for font-src 
- Allow site for frame-src 
- Allow site for img-src 
- Allow site for media-src 
- Allow site for style-src 

4. Click **Save & New**.

Field	Value	Notes
Trusted Site Name	My_domain	
Origin URL Pattern	https://[DOMAIN_URL].my.salesforce.com	Copy and paste your domain URL from the browser address bar into the Trusted Site URL box. Delete .lightning.force.com from the URL you just copied and pasted. Copy and paste the following to the end of the URL: .my.salesforce.com For example, your URL may look like this: https://student9999-cme.my.salesforce.com

5. Select every **unchecked checkbox**  below the **CSP Directives** heading.
6. Click **Save**.

Task 3: Open the DC LWC OmniScript sample app and deactivate unneeded components

1. Using the **Lightning App Launcher** , search for **Digital Commerce**, and then select it.
2. Select the **OmniScripts** tab from the Lightning navigation bar. You may have to click the **More** tab in the bar to reveal the **OmniScripts** tab.
3. Select **dc/lwc** from the list of OmniScripts.
4. Select **DCLWCSampleOS (Version 2)**.

It may take a couple of minutes for this to load the first time. If nothing seems to be happening when you click on it, please wait and be patient for it to load.



NOTE:

For this exercise, the Explore & Checkout using Digital Commerce LWC OmniScript has been included in your training playground. This is the OmniScript you will configure to create a guided selling interaction. You will start out by removing a component of the OmniScript not relevant to Eliza's guided selling design needs.

5. Create a new version.

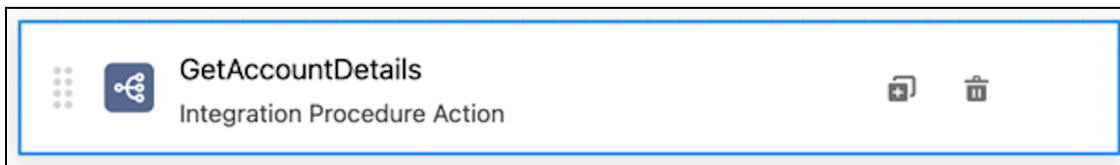


BEST PRACTICE:

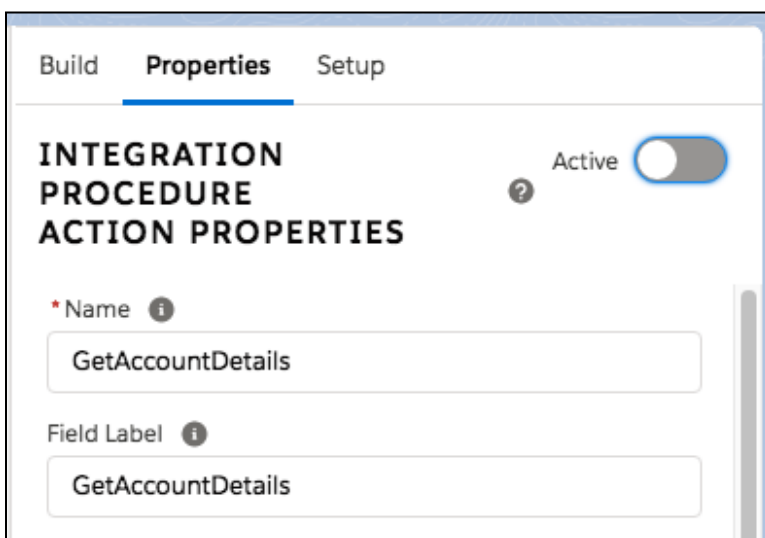
It is best practice to create a new version when you need to make changes to the original. This allows you to revert back to the original should you need to do so.

- a. Click the **New Version** button near the top-right corner of the page.

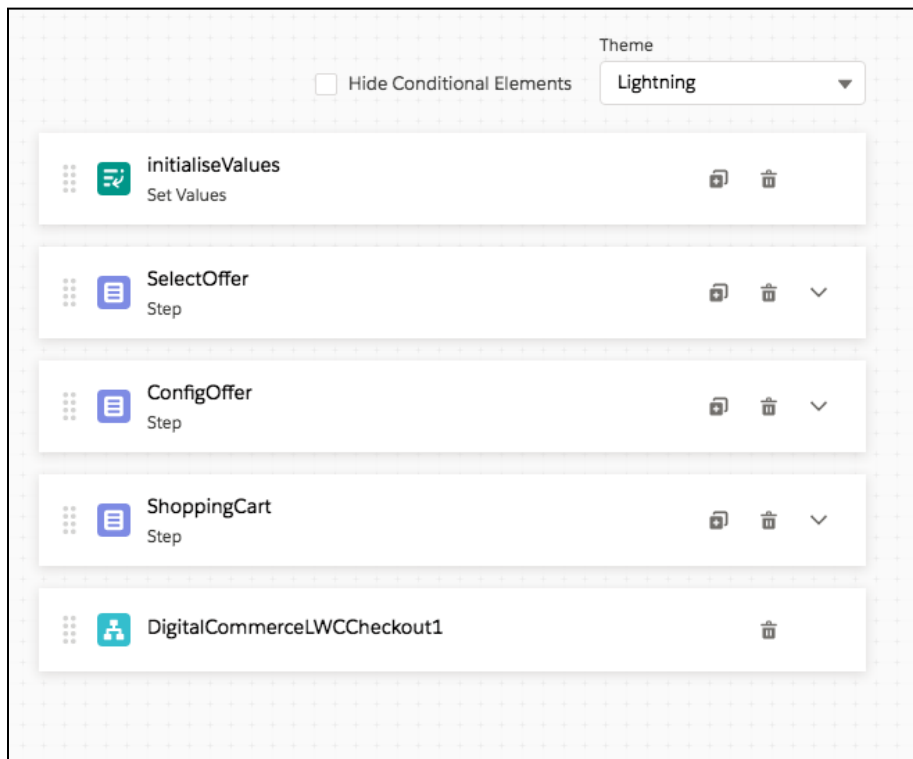
6. You're going to cover the app in detail in the next task, for now you'll deactivate unneeded components for your scenario. Click the **GetAccountDetails** rectangle.



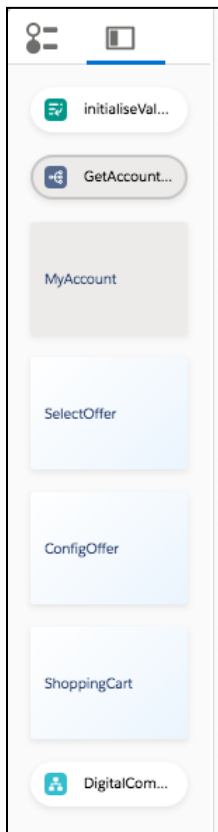
7. Click the toggle switch next to **Active** in the right-side properties panel to deactivate this component, which in this case is an OmniStudio Integration Procedure.



Notice that it disappears from the middle canvas panel.

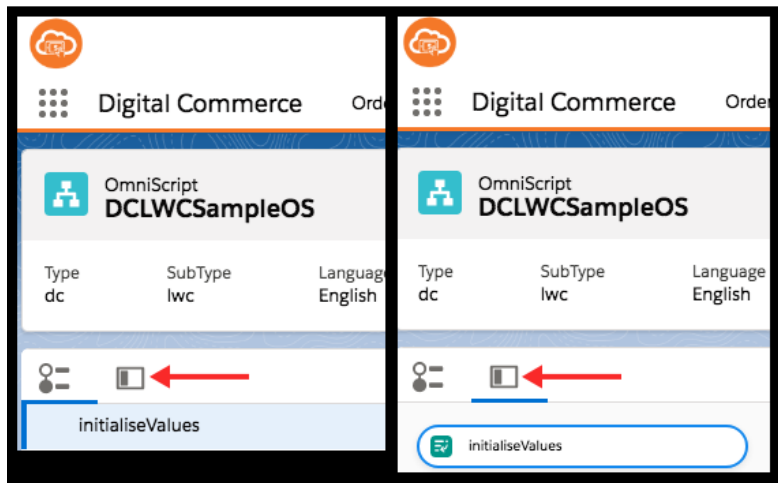


Take a look at the left-side navigation panel as well. It is greyed out to indicate it is deactivated.



8. We have already deactivated the rest of the components not needed for this exercise for you. However, let's take a moment to understand the OmniScript interface, as well as what we are deactivating.

Click the **detailed view icon**  in the left-side navigation panel in the top-left corner to toggle between views.



The tree view allows you to expand each step and see the nested components, including deactivated ones.

9. Expand the **SelectOffer**, **ConfigOffer**, and **ShoppingCart** steps by clicking the arrow

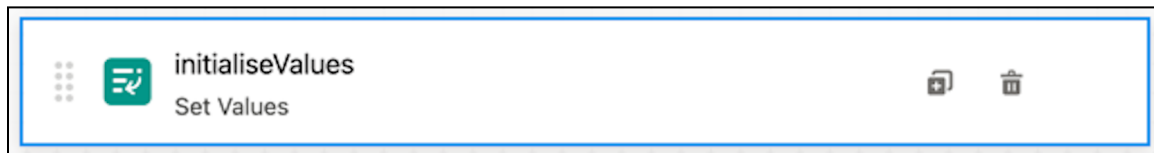
icons  next to each one.



Notice the greyed-out components in the OmniScript. For your scenario, you don't need the functionality that comes with the [global header](#) component which houses asset management features, so we have deactivated those components for you.

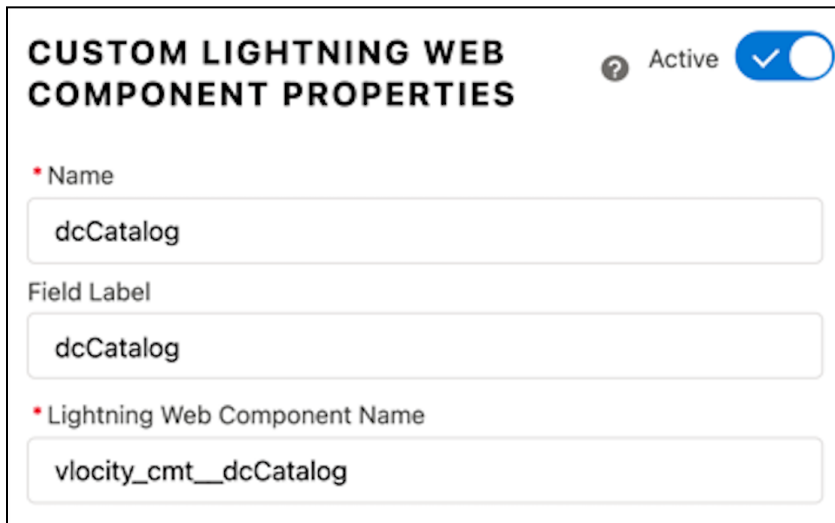
Task 4: Explore the DC LWC OmniScript sample app

1. Click on the **initialiseValues** rectangle in the middle navigation panel. Below **Element Value Map** in the right-side **SET VALUES PROPERTIES** panel, you are presented with two elements which pass property values to the components in the OmniScript.



- a. By establishing a **catalogCode** value at the beginning of your OmniScript, you can pass that value to your subsequent **dcCatalog** Lightning web component. The **dcCatalog** component will then render the UI. Because **dcCatalog** is expecting a **catalogCode**, you cannot leave this blank, otherwise it will not render the UI.
 - b. **brainTreePaymentUrl** is the URL for the off-platform payment gateway service that will be used during checkout by the **BraintreepaymentLWC** Lightning web component to handle the user's payment information.
2. Click the **SelectOffer** rectangle. This is a step component which represents the browse phase (i.e., user selecting a product from a list) and contains three Lightning web components.
 3. Click on the **dcCatalog** component. This component provides a list of Browse phase catalogs.

This OmniScript component is a **Custom LWC**. The Lightning web component this OmniScript component uses is specified by the **Lightning Web Component Name** field.



- a. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.

This is where the catalog code merge field **%catalogCode%** below **Property Source** is passed to the **catalog-code** property for this **dcCatalog** Lightning web component. For the user, this represents the selected parent catalog shown on the UI.

dcCatalog is one of three Lightning web components that make up the **SelectOffer** step. Each step represents a stage of the buying process and is shown in a step chart. Below is an example of what the step chart can look like. You can customize the step chart in **Script Configuration**.



4. Click on the **dcChildCatalog** component. This component provides a list of Browse phase child catalogs.

- a. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.

This component follows a similar pattern as **dcCatalog** to ensure that the child catalog displays the appropriate parent catalog. For the user, this represents the parent catalog shown on the UI just above the child catalogs.

5. Click on the **dcOffersList** component. This component retrieves all eligible products from the applicable sales catalog.

- a. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**. This component uses three values to render offers on the page for the user.

catalog-code will retrieve offers from the sales catalog associated with this code.

page-size determines how many offers show on the page on its initial load. The user can still retrieve all of the offers from the sales catalog by using the UI. You will experiment with this property later.

context is used to determine which offers are rendered for the user, as determined by context rules. You will use this value later when you apply context rules to this guided selling experience.

6. Click the **ConfigOffer** rectangle. This is a step component which represents the configure phase (i.e., user configuring a product) and contains one Lightning web component.
7. Click on the **dcOfferConfig** component. This component renders configuration details for the selected product and allows the user to select applicable product attributes prior to adding the configured offer to the cart.

- a. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.

This component is passed values from the **SelectOffer** step's **dcOffersList** component for the catalog code, as well as the offer code for the offer/product the user selected. The previous step is identified as **SelectOffer** such as in this value:

%SelectOffer:dcOffersList:catalogCode%. Similarly, the product **offerCode** is specified.

Finally, the **apiUrl** can be specified. The **apiUrl** refers to a custom domain URL for your potential digital commerce site and is only utilized for anonymous users, which is not needed by your scenario.

8. Click the **ShoppingCart** rectangle. This is a step component which represents the cart phase (i.e., user viewing the configured products in their cart) and contains one Lightning web component. This is a temporary cached cart and not an order in Salesforce yet, that comes in the next phase.
9. Click on the **dcShoppingCart** component. This component renders the products that have been added to the cart. It also allows the user to modify the cart.

- a. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.

This component is passed values from the **SelectOffer** and **ConfigOffer** steps, specifically **dcOffersList** and **dcOfferConfig** components. At this point you probably recognize the catalog code, but not the **cart-context-key** property. This property is generated when offers/products are added to the basket. It is a unique identifier for the basket. It is generated by the **dcOfferConfig** component and passed to this one.

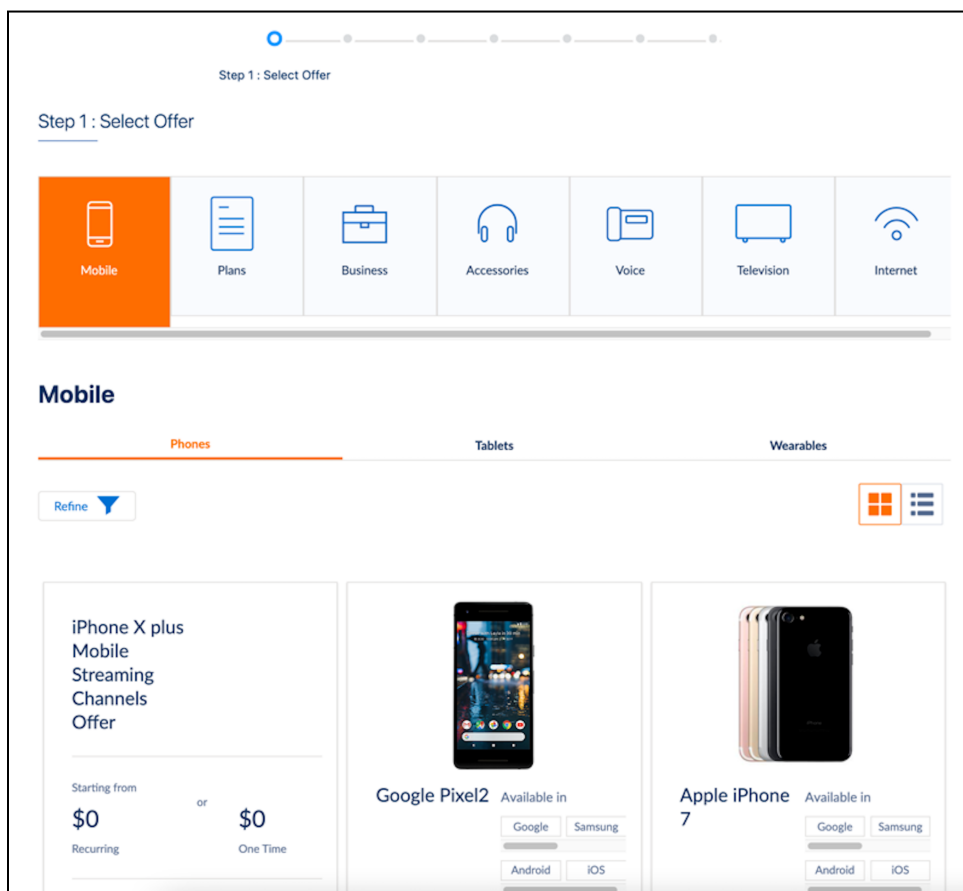
is-community is a property for using this app in a Salesforce community site that does not utilize the embedded Checkout OmniScript. For your scenario, it does not apply, so it is set to **false**.

Finally, the **apiUrl** can be specified. The **apiUrl** refers to a custom domain URL for your potential digital commerce site and is only utilized for anonymous users, which is not needed by your scenario.

The next component is an embedded reusable OmniScript that handles the checkout phase of the user experience. You will come back and explore it in the next exercise.

Task 5: Deploy the DC LWC OmniScript sample app

1. Click the **Activate Version** button at the top of the page.
2. Click the **Done** button.
3. Select the **Preview** button near the top of the page. It may take a minute for the OmniScript to deploy the Lightning web components. You will know the deployment is successful when the UI loads and displays the products for the **Phones** child catalog.



Exercise 8-4: Configuring the DC LWC Sample App

Scenario

Since Eliza ran cache jobs earlier, she can now see how the sample app functions when using her sales catalog. With a tested sample app, she is one step closer to having a functional guided selling experience.

Goal

- Modify the DC LWC OmniScript sample app to utilize your sales catalog

Tasks

1. Set a Context Id in the initialiseValues component
2. Override the catalogCode for dcOffersList
3. Activate the OmniScript
4. Restore the original functionality of dcOffersList

Time: 15 mins

The Set Values OmniScript Component

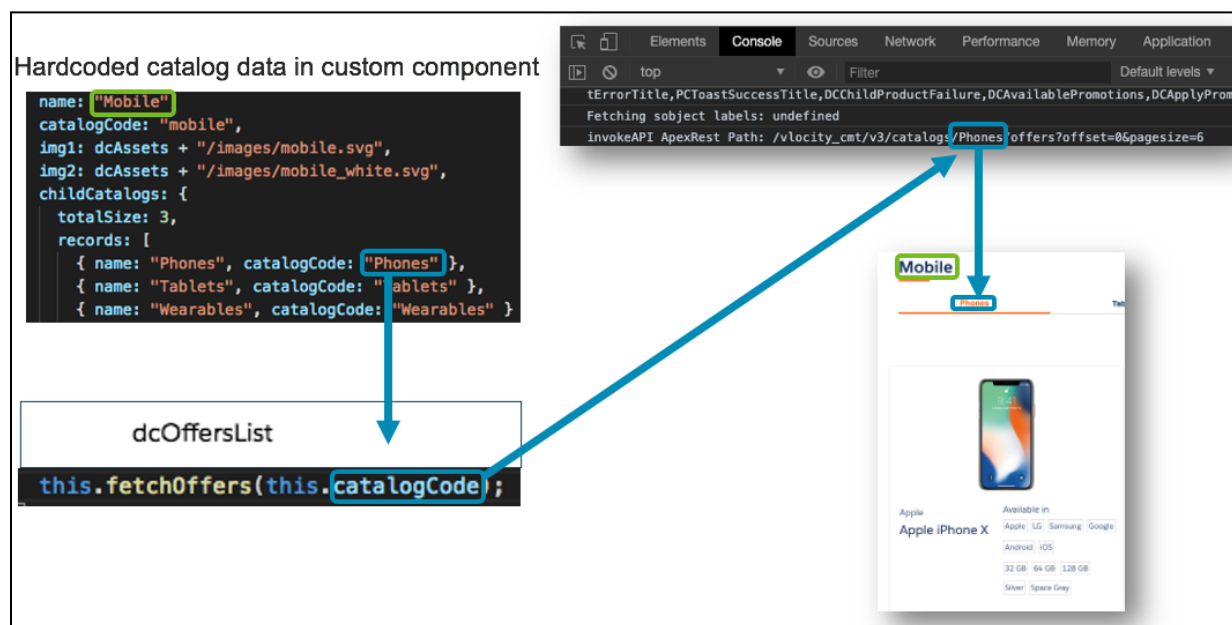
[Set Values](#) elements allow you to add key/value pairs to the OmniScript's JSON, from which you read and write values for the OmniScript components.

Here are the options for values:

- Merge fields from a previous step (%elementName%)
- Literal value
- Concatenated values
- Results of formulas and functions
- Expressions that combine the options: "Case Status: %caseStatus%"

At runtime or during Preview, once the OmniScript engine finds the matched JSON node, it will pull the entire section of data out of the data JSON and use it to populate the list. This can be confusing sometimes when your entire product list seems to disappear from the data JSON!

How does dcOffersList work?



Do not confuse the **parent catalogCode** with the **child catalogCode**. The **child catalogCodes** are used by **dcChildCatalog** and **dcOfferList** to render the sales catalog product. The entirety of this UI is hardcoded in the various Digital Commerce sample app Lightning web components' code and is effectively invisible to you.

During app development, a developer will create a custom Lightning web component that builds off of the out-of-the-box **dcCatalog** to hardcode the sales catalogs that you plan on using. In this scenario, you will not be modifying catalog codes in OmniScript Designer.

In this image you can see the hardcoded parent catalog **Mobile** outlined in green, and the child sales catalog code **Phones** outlined in blue. **dcOffersList** uses the child sales catalog code to call the APIs and get a response with the appropriate products. The parent catalog name is simply rendered on the page with no API calls necessary.

For the sake of seeing how **dcOffersList** works, you will temporarily override it to see how it can utilize a different sales catalog than the ones it is hardcoded for.

Task 1: Set a Context Id in the initialiseValues component

It's often helpful to create a context id set values component, so that you don't have to manually add the context id each time you preview your OmniScript. This special set values component is ignored at run-time, so it's safe to use it as a design-time convenience. This script will be launched in an account context, so it will use an Account Id as its context id.

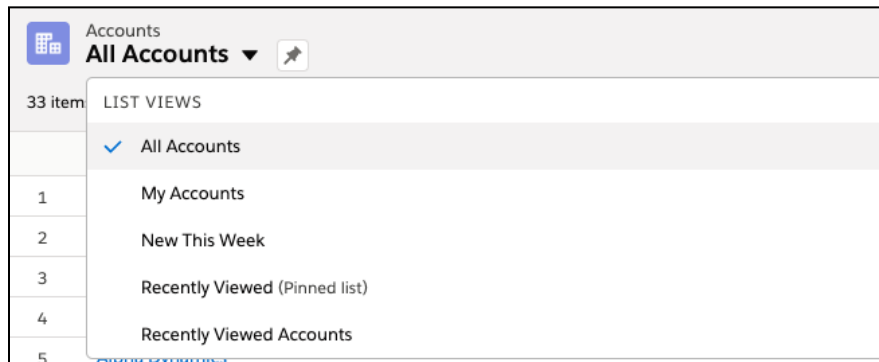
1. Select the **Design** button near the top of the page.
2. Select the **Deactivate Version** button at the top of the page.



3. Click the **Done** button.
4. Get an Account Id.
 - a. Select the **initialiseValues** Set Values component.



- b. Right-click on **Accounts** in the Lightning navigation bar, and select **Open link in new tab**.
- c. Change the list view to **All Accounts**.



- d. Select **John Smith**.
- e. Click the **Details** tab. Notice that the **Account Record Type** is “Consumer”. This means this is an eligible user for our B2C (Business to Consumer) sample sales catalog. Also take note that his **Region** is “West”. These details will be important in the next exercise when you implement context rules.



NOTE:

Each sales catalog can only be paired with one price list. To see which price list applies to a sales catalog, navigate to the **Catalogs** page, select a catalog, select the **Details** tab, and view the value for **Default Price List**.

- f. Click on the browser URL bar.
- g. Copy the value that follows **Account/** and precedes **/view**. This is the Account ID for this user. Copy this value.

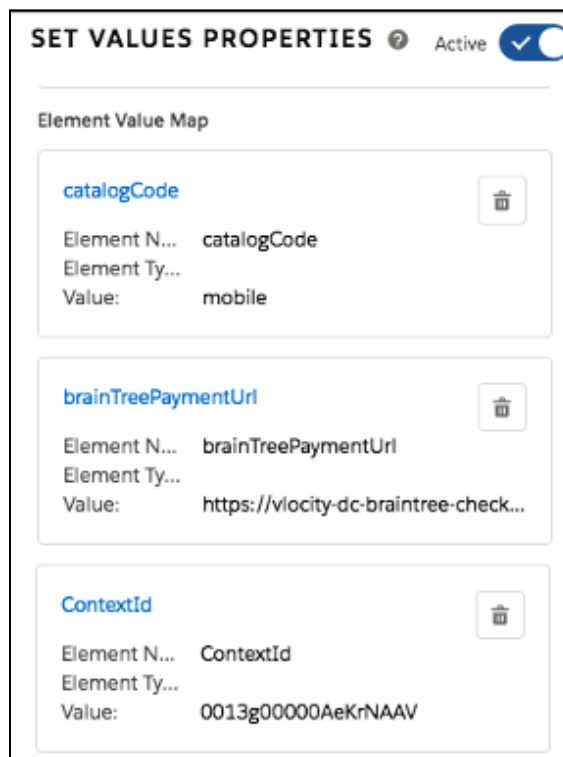
`lightning.force.com/lightning/r/Account/0016g00000CgxXYAAZ/view`

- 5. Navigate back to the tab with the DC LWC OmniScript.

6. Add the ContextId to the Set Values element.
 - a. Under **Element Value Map**, click **+ Add New Element Value** to add a new key/value pair:

Element Name	Value
ContextId	[Paste the Account Id for John Smith]

- b. Click the **Save** button.



SET VALUES PROPERTIES ? Active ☒

Element Value Map

catalogCode 

Element N... catalogCode

Element Ty... mobile

Value: mobile

brainTreePaymentUrl 

Element N... brainTreePaymentUrl

Element Ty... https://vlocity-dc-braintree-check...

Value: https://vlocity-dc-braintree-check...

ContextId 

Element N... ContextId

Element Ty... 0013g00000AeKrNAAV

Value: 0013g00000AeKrNAAV

Task 2: Override the catalogCode for dcOffersList

As you can see, the app is loading products but they don't match up with the catalog you created earlier. This is because there is already a catalog created in your training playground that matches the default catalog code configured in this OmniScript. Let's temporarily configure this OmniScript to use the catalog you created earlier. This will give you a chance to see your sales catalog in action.

1. Click **dcOffersList** Custom LWC component from the navigation panel on the left-side of the page.
2. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.
3. Click the box below **Property Source** next to **catalog-code**.
4. Enter in `DC_MOBILE`.

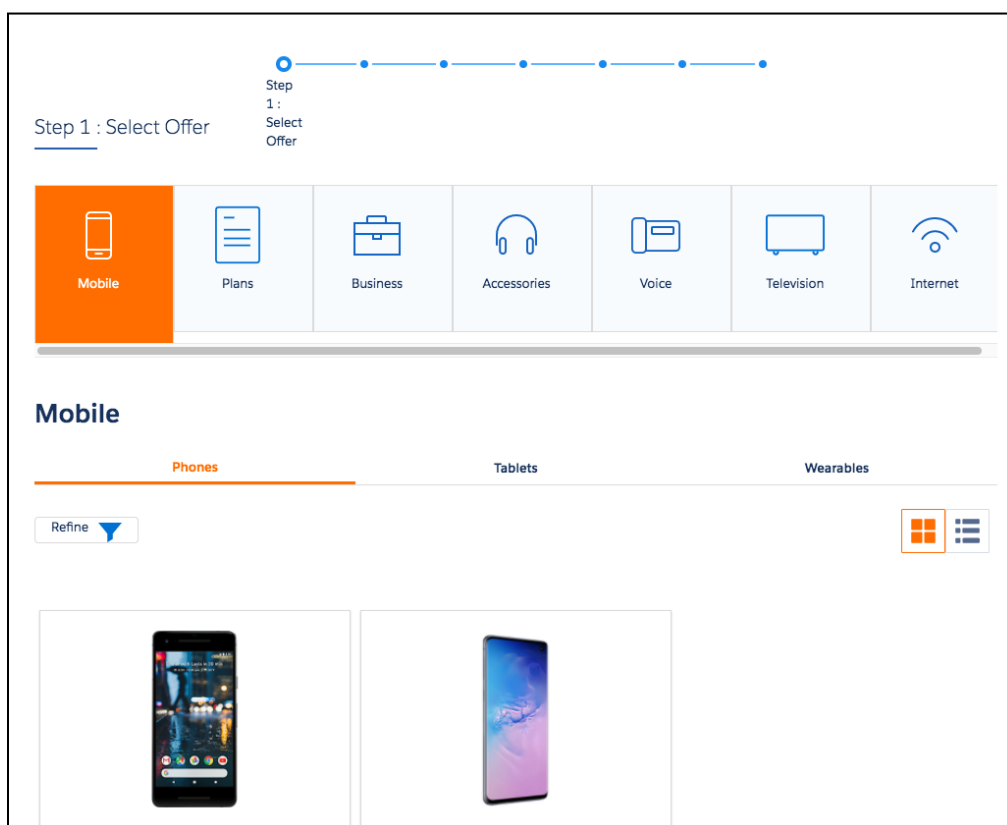
CUSTOM LIGHTNING WEB COMPONENT PROPERTIES 

Property Name	Property Source	
catalog-code	DC_MOBILE	
page-size	3	
context	%context%	

[+ Add New Property](#)

Task 3: Activate the OmniScript

1. Click **Activate Version**.
2. Click the **Done** button. It may take a minute for this process to complete.
3. Select the **Preview** button near the top of the page. It may take a minute for the OmniScript to deploy the Lightning web components. If for some reason the cache is not reflecting your catalog, you may need to rerun the **POPULATE API CACHE** job from Vlocity CMT Administration, Cacheable API Jobs.



You should now be able to see products loaded onto the page from your Mobile catalog. Because you are no longer using the catalogCodes hardcoded into the sample app, you will only be able to see your Mobile catalog in the app, even if you click on other child catalogs.

You will test the OmniScript guided selling experience later but you may optionally do so now:

You can now click through and test the user experience by selecting a product and continuing through the buy flow. On the Config Offer page, make sure that the product attributes are selected before you continue on. You will know they are selected if they have an orange box around each of them. Use the **Add To Cart** button to advance.

During checkout and in every subsequent time you test this OmniScript, use the following billing/shipping and payment information (it is a sample credit card and will not process any real charges):

Billing & Shipping Information:

Field	Value
Address Line	1012 Layton Rd
City	Redding
State	CA
Zip Code	96002

Payment Information:

Field	Value
Card Number	4111 1111 1111 1111
Expiration Date	09/24
CVV	444
Postal Code	4444

After you submit an order, go back to John Smith's account page and see if the order shows up. If you get stuck on a loading screen, refresh the page and start over.

Task 4: Restore the original functionality of dcOffersList

1. Select the **Design** button near the top of the page.
2. Select the **Deactivate Version** button at the top of the page.
3. Click the **Done** button.
4. Click **dcOffersList** Custom LWC component.
5. Click **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**.
6. Click the box below **Property Source** next to **catalog-code**.
7. Enter in `%SelectOffer:dcChildCatalog:catalogCode%`

This ensures dcOffersList uses the active catalogCode value provided by dcChildCatalog.

Exercise 8-5: Add Context Rules to a Guided Selling Experience

Scenario

One of Eliza's business requirements is to only show products that customers are eligible for, based on their geographical region. Fortunately, Eliza knows that rules in the shared catalog can be cached alongside products, and that the OmniScript sample app can be modified to accommodate the new eligibility rule.

Goal

- Modify the sample app to use a Digital Commerce cacheable API parameter that will qualify customers based on their region

Tasks

1. Add a context rule to one of your sales catalog products
2. Regenerate the cache
3. Add context rules to your guided selling experience

Time: 20 mins

What are Digital Commerce Cacheable API parameters?

Cacheable API calls can be modified with many different parameters. This allows you to customize the functionality of your guided selling experience.

Return a list of offers from the “Phones” sales catalog
`vlocity_cmt/v3/catalogs/Phones/offers`

Return a list of offers and specify a context:
`vlocity_cmt/v3/catalogs/Phones/offers?context={"Status":"Active"}`

Return a list of offers and specify a page size of 20:
`vlocity_cmt/v3/catalogs/Phones/offers?pageSize=20`

Return a list of offers for a logged-in user:
`vlocity_cmt/v3/catalogs/Phones/offers?context={"accountId":"0014x00000Di4O1AAJ"}&loggedIn=true`

This image shows off just a few of the possible parameters that can be used with the Get Offers cacheable API. The first one simply returns a list of offers for the “Phones” sales catalog. This API call has no parameters applied to it.

The second API call shown above appends a context, in this case it is specifying the context dimension of “Status” and the value of “Active”. In other words, it will retrieve all offers with a status of active.

The third API call shown above returns a list of offers with page size of 20. This means that up to 20 offers will load on the user’s initial page load. If more than 20 offers exist in the “Phones” catalog, the user will need to click a button to load the rest.

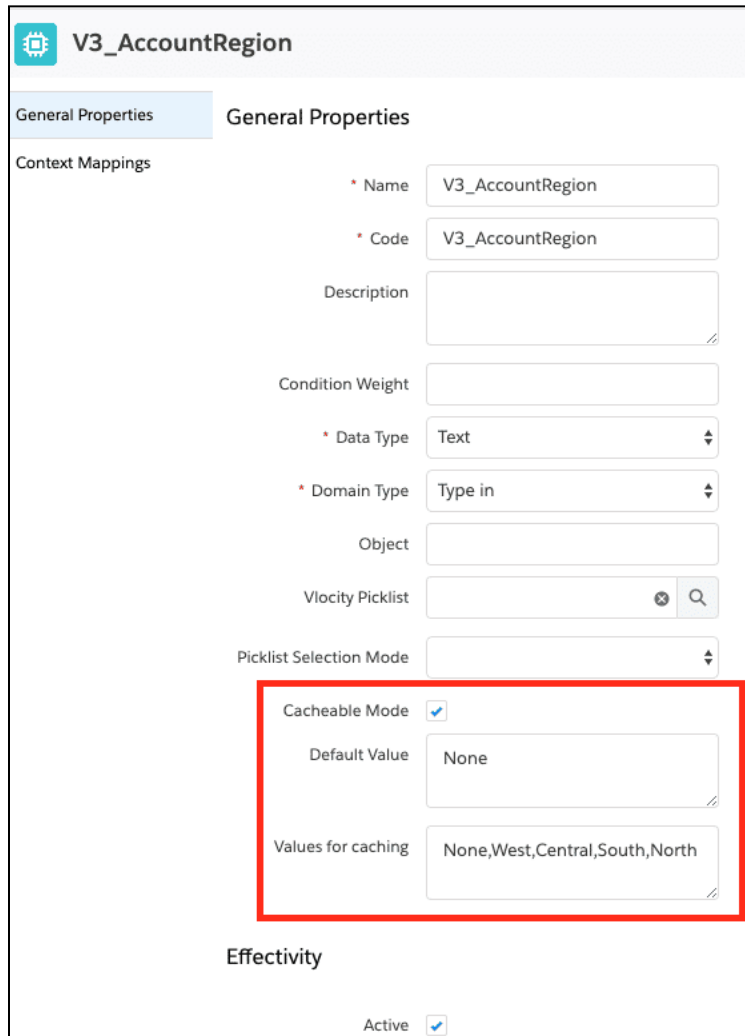
The fourth API call shown above retrieves a list of offers specific to a certain user. In this lesson, we will customize the Get Offers API call by specifying a context for the `dcOffersList` Lightning web component. This component will then convey the context parameter to the API.

In this lesson, we will customize the Get Offers API call by **specifying a context** for the **dcOffersList** Lightning web component. **This component will then convey the context parameter to the API.**

How are context rules used in Digital Commerce?

Context rules are applied to products and promotions in Digital Commerce very similarly to that of the Cart, just with a few extra [context dimension fields](#) to specify. The key addition is that context rules can now be cached in correspondence to the products and promotions available in active sales catalogs. For instance, if you wanted to create a catalog with region-specific 5G data plans and products, you could define context rules for that region so that only customers from a specific region are eligible to browse and purchase those digital commerce products.

The following fields must be filled out to use context dimensions with Digital Commerce sales catalogs:



V3_AccountRegion

General Properties

Context Mappings

* Name V3_AccountRegion

* Code V3_AccountRegion

Description

Condition Weight

* Data Type Text

* Domain Type Type in

Object

Vlocity Picklist

Picklist Selection Mode

Cacheable Mode ☒

Default Value None

Values for caching None,West,Central,South,North

Effectivity

Active ☒

Cacheable Mode: when this option is checked, cacheable jobs (i.e., Populate API Cache) will utilize this context dimension. If this option is not selected, the context dimension will not be cached, and therefore not available to Digital Commerce APIs.

Default Value: this is the value that will be used for the context dimension (and passed to the context rules) when the input context parameters do not have a given value. The value is used in cases of tightest match for price list entries. If this field is left empty, you will receive an error message when you run cacheable API jobs.

Values for caching: this is the list of possible values for this dimension. For each of these values, the cacheable jobs (e.g., Populate API Cache) will cache a response.

During runtime, if the input context parameter is passed a value not present in this list, the CPQ engine will evaluate the response on the fly and then cache the result.



RESOURCE:

Need a refresher on context rules? Take the [Context Rules course](#).

How do you extend context rules for Digital Commerce?

When API calls are made by users, the new APIs will first check the API response cache before processing the request. If a response to the user's request is in the cache, the response is returned to the user. Because the API response cache is created for all permutations of eligible (as defined by your context rules) products, you will need to make sure that either yourself, or a developer, are refreshing the cache for your catalog after you implement new rules. The checkout method requires the cartId (order, quote or opportunity id). Based on the header object, checkout moves the object to the next state of the ordering process.

Task 1: Add a context rule to one of your sales catalog products

For the sake of this exercise, we have already created the [context dimension](#), [rule](#), and [rule sets](#) necessary to create the region [qualification of products](#). You will add this rule set to one of your sales catalog products. You will test this context rule in your guided selling experience in a later exercise. Learn more about using context rules with Digital Commerce in the next exercise.

1. In the Lightning navigation bar, right-click on **Vlocity Product Console** and select **Open link in new tab**.
2. In the Dashboard under **Product Management**, click the **search icon**  next to **Product**.
3. Click the **search icon** .
4. Select the **Google Pixel2** row.
5. Click on **Context Rules** in the left sidebar.
6. Click the **Add Rule Set** button. If the pop-up dialog box initially shows up blank, close it and click the **Add Rule Set** button again.
7. Click the **search icon**  next to **Vlocity Rule**.
8. Click the **search icon** .
9. Click the **Select** link next to **West Region Accounts Rule Set**.
10. Click the **Save** button. The Google Pixel 2 will now only be available to accounts in the West Region.

Next, you'll need to run the cache jobs again to ensure that this context rule is cached and usable by the cacheable APIs.

Task 2: Regenerate the cache



NOTE:

Starting with the CME Winter '20 release, cache management is automatic, and you no longer need to manually repopulate the cache after 30 days. This means, any new rules, pricing, and product configurations will automatically be updated in the cache after 30 days. As a best practice, when you make any product-related changes, you should deliberately regenerate the cache by running the **REGENERATE CACHED API RECORDS** job from the Cacheable API Jobs page.

1. In the Lightning navigation bar, right-click on **Vlocity CMT Administration** and select **Open link in new tab**.
2. Below **Admin Console**, select **Cacheable API Jobs**.
3. Click the **Start** button next to **REGENERATE CACHED API RECORDS**. This is considered regenerating the cache. It may take a few minutes for this job to complete.

REGENERATE CACHED API RECORDS						
This will regenerate the cached API response objects based on modified sales catalog and product models.						
Job Name	Status	Total Batches	Batches Processed	Created Date	Completion Date	Status Detail
ReGenerateCachedAPIRespBatch	Completed	333	333	12/11/20 2:31 PM	12/11/20 2:33 PM	
ReGenerateCachedAPIBatchProcessor	Completed	28	28	12/11/20 2:31 PM	12/11/20 2:31 PM	
InvalidateCachedResponsesBatch	Completed	19	19	12/11/20 2:30 PM	12/11/20 2:31 PM	
DeriveCMAAggregateEntityBatch	Completed	140	140	12/11/20 2:29 PM	12/11/20 2:30 PM	

If you receive a **Status Detail** message stating the job was aborted, please complete the following steps:

- Click the **Setup icon** .
- Click **Developer Console**.
- Click **Debug** from the menu bar at the top of the window.
- Click **Open Execute Anonymous Window**.

- Paste the following into the **Enter Apex Code** box:

```
delete[SELECT Id FROM vlocity_cmt__CachedAPIResponse__c];
```

- Click the **Execute** button.
- Close the **Developer Console** window.
- Click the **Start** button next to **POPULATE API CACHE**.
- Select the **checkbox** at the top, which will then automatically select all of the items in the list.
- Select the **checkbox** next to All so that the cache is populated for all of the sales catalogs in your training playground.
- Click the **OK** button.

NOTE:

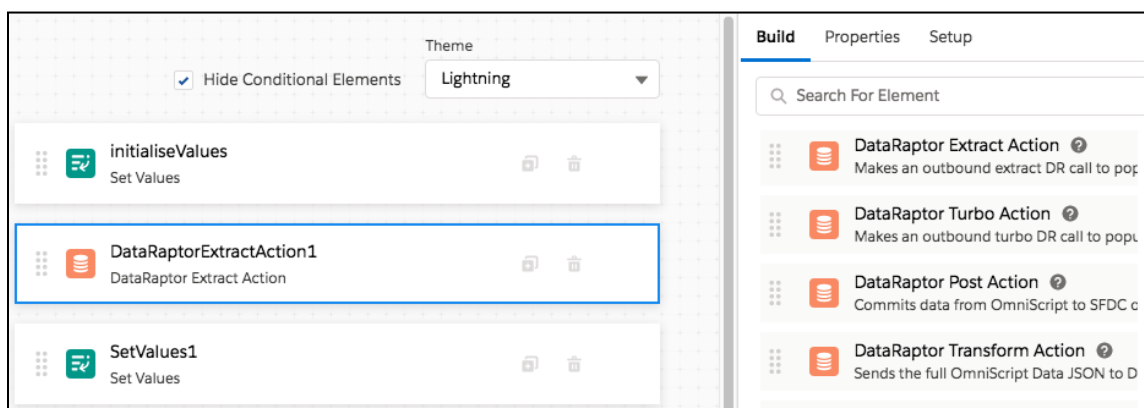
Any add, modify, or delete action performed on the items below will update the cache when the **REGENERATE CACHED API RECORDS** job is run:



- Catalog
- Catalog Product Relationships
- Offers
- Pricing: Pricelist entries, Pricing overrides, and adjustments
- Product structure, PCI, and group cardinality
- Context dimensions
- Product/ Promotion/ Pricing based context qualification rules
- Product Relationships
- Product Configuration Procedures
- Attribute-based Pricing
- Effectivity dates and Active flag on all the above

Task 3: Add context rules to your guided selling experience

1. Return to your browser tab that has the OmniScript open.
2. Click the **Build** tab from the properties panel on the right-side of the page.
3. Drag and drop the following components below the **initialiseValues** component located in the **ACTIONS** dropdown menu:
 - a. DataRaptor Extract Action
 - b. Set Values

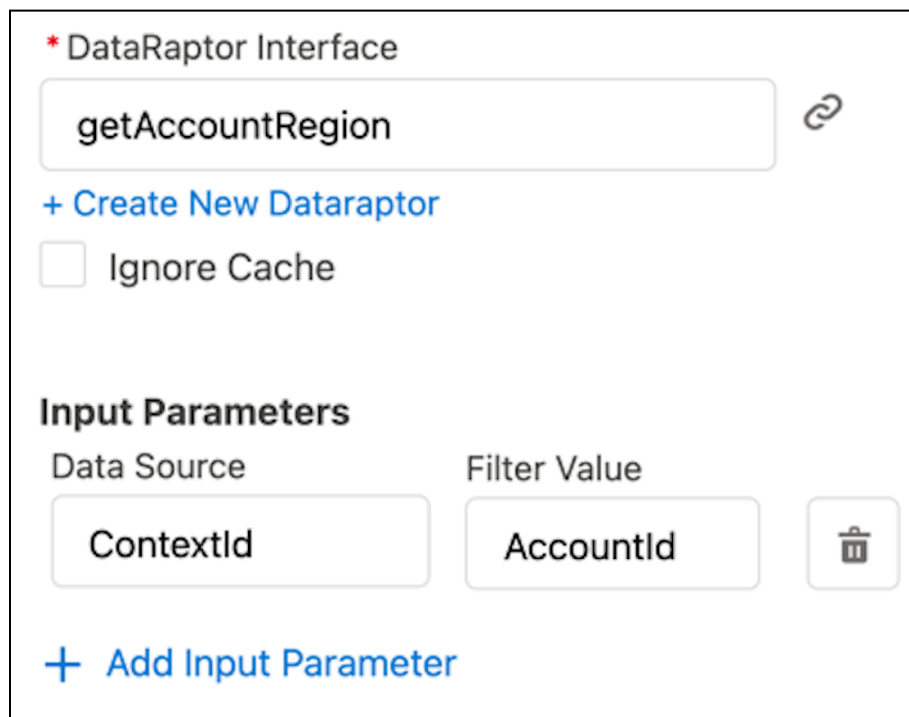


4. Configure the DataRaptor component.
 - a. Click on the **DataRaptorExtractAction1** component.
 - b. Change the **Element Name** and **Field Label** to `DRExtractAccountRegion`. This is a DataRaptor that will get the value of the Region field on the Account object identified by an Account ID.
 - c. Click on the **DataRaptor Interface** field.
 - d. Type `getAccount`.
 - e. Select **getAccountRegion** from the dropdown menu.
 - f. Scroll down to the **Input Parameters** section.
 - g. In the **Data Source** field, type `ContextId`.
 - h. In the **Filter Value** field, type `AccountId`.



RESOURCE:

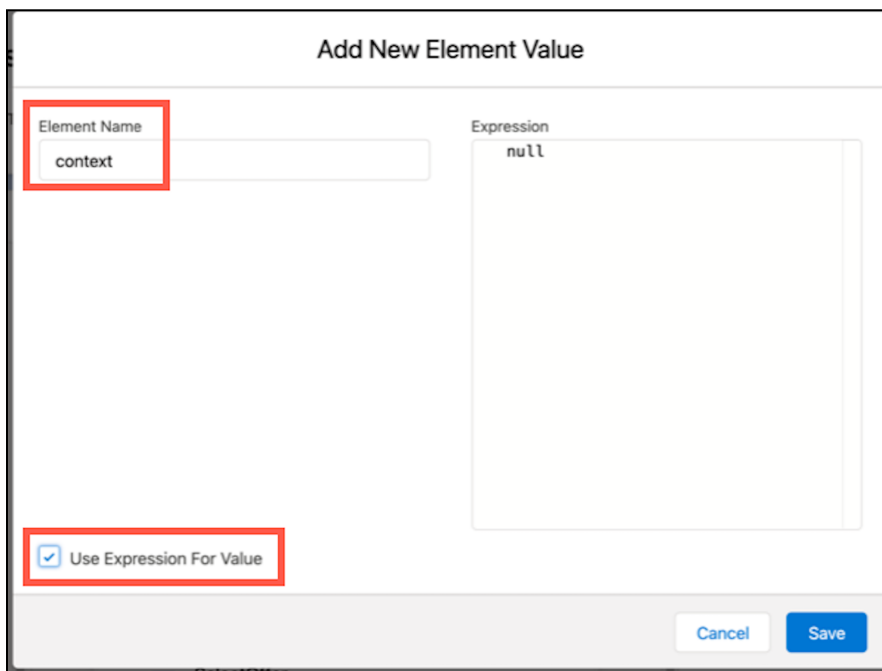
Need a refresher on DataRaptor? Take the [Building a 360° View of the Customer course](#).



The screenshot shows the 'DataRaptor Interface' configuration window. At the top, there is a red asterisk followed by the title '* DataRaptor Interface'. Below the title is a text input field containing 'getAccountRegion' with a link icon to its right. Underneath is a blue link '+ Create New Dataraptor'. Below that is a checkbox labeled 'Ignore Cache'. The next section is titled 'Input Parameters'. It contains two input fields: 'Data Source' with the value 'ContextId' and 'Filter Value' with the value 'AccountId'. To the right of the 'Filter Value' field is a trash can icon. At the bottom of the section is a blue link '+ Add Input Parameter'.

5. Configure the Set Values component.
 - a. Click on the **SetValues1** component.
 - b. Change the **Element Name** and **Field Label** to `setAccountRegion`.
 - c. Click the **+ Add New Element Value** button.
 - d. In the **Element Name** field, type `context`.

6. Click the checkbox next to **Use Expression For Value**.



- a. In the **Expression** field, replace **null** with the following value:
`'{"Region": "'" + %accountRegion% + "'"}`



ALERT:

Do **not** copy and paste the value in the previous step as OmniScript will parse the single and double quotes incorrectly. Be sure to use single and double quotes exactly as written above. Alternatively, you may copy and paste the expression value if you **paste without formatting** (Ctrl+Shift+V on Windows or Cmd+Shift+V on Mac).

This value will allow you to pass the account's region value for the "Region" context dimension, using the merge code for accountRegion. It must be specified in this format as this is how context is expected by the Digital Commerce cacheable APIs. In case you are wondering where this context property is referenced, you can see it in the **dcOffersList** component under its **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES** section.

- b. Click the **Save** button.

7. Click the **Activate Version** button. This will take a few minutes.
8. Click the **Done** button.
9. Click the **Preview** button near the top of the page.
10. Wait a few seconds for the app to load. Look at the **Data JSON** button on the right-side of the page. Notice that the context is passing the region context dimension as “West”. The \ characters that precede double quotes are [escape characters](#) ensuring that they are not parsed by the OmniScript, but rather the Digital Commerce Cacheable APIs.



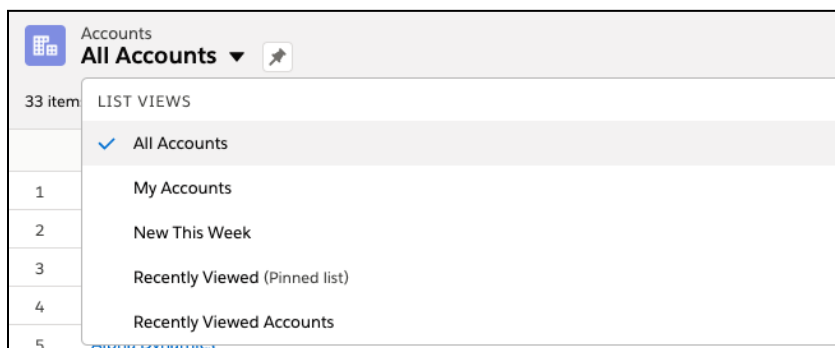
11. Now take a look at the app. You should see the **Google Pixel 2** product loaded. This is because we made it only available for customers in the “West” region, as qualified by your context rule, and the customer you selected earlier is in the “West” region.

If no products are loading, go back to step 3.5c and repeat the following sub-steps. You may copy and paste the expression value if you paste without formatting (Ctrl+Shift+V on Windows or Cmd+Shift+V on Mac). If those steps do not work, click on the **SelectOffer** rectangle and then on **dcOffersList** within the canvas panel. Click on **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES** within the properties panel. Delete the **context** property and then recreate it with `context` as the

Property Name and `%context%` as the **Property Source**. Activate the OmniScript and try previewing it again.

Next, you'll test if the Google Pixel 2 shows up for a customer in the "East" region.

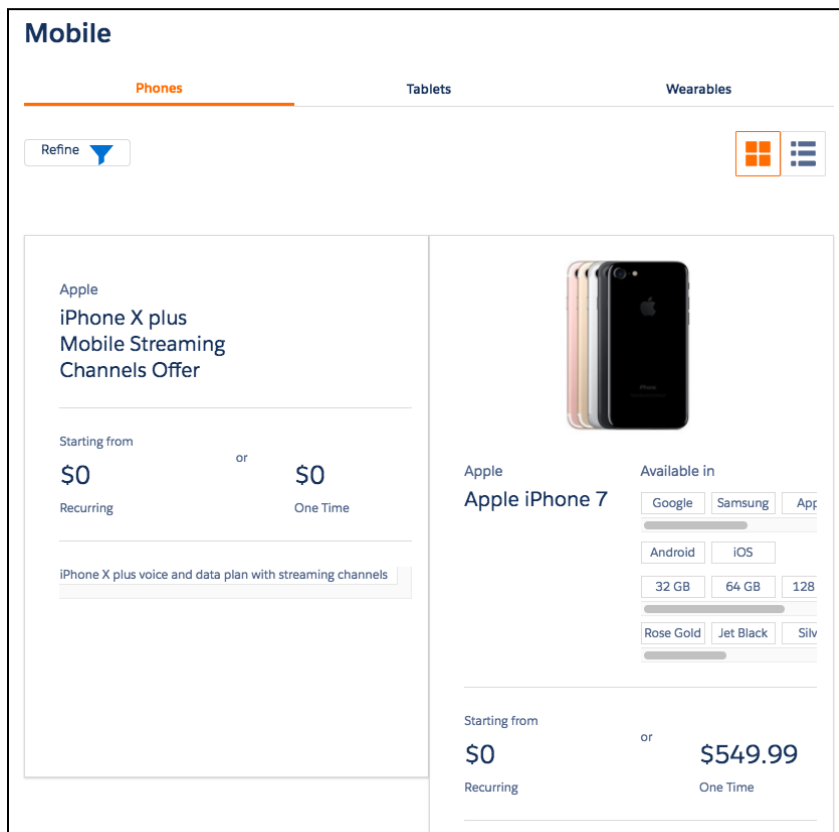
12. Click the **Design** button at the top of the page.
13. Click the **Deactivate Version** button.
14. Click the **Done** button.
15. Click the **initialiseValues** rectangle.
16. Right-click on **Accounts** in the Lightning navigation bar, and select **Open link in new tab**.
17. Change the list view to **All Accounts**.



18. Select **Acme**.
19. Click the **Details** tab. Take note that their **Region** is "East".
20. Click on the browser URL bar.
21. Copy the value that follows **Account/** and precedes **/view**. This is the Account ID for this user.

`lightning.force.com/lightning/r/Account/0016g00000CgxXYAAZ/view`

22. Navigate back to the tab with the DC LWC OmniScript.
23. Click on **ContextId** within the **SET VALUES PROPERTIES** right-side panel.
24. Paste the copied value into the **Value** box.
25. Click the **Save** button.
26. Click the **Activate Version** button. It may take a minute.
27. Click the **Done** button.
28. Click the **Preview** button near the top of the page. Wait a few seconds for the app to load. Look at the **Data JSON** panel on the right-side of the page. Notice that the context is passing the region context dimension as “East”. Therefore, the Google Pixel 2 no longer shows up in the guided selling experience for this customer.



Exercise 8-6: Configure the Selling End and Launch Points of the Script

Scenario

Eliza is in the home stretch! The remaining items are to review the Checkout OmniScript in the script, and then set the end and launch points for the script. The call center manager wants guided selling to launch from the account page. So, Eliza must create an OmniStudio Action button that will appear on the Account record page to launch the OmniScript and then she can create a Navigate action to take the user to the appropriate order record page after the order is submitted.

Goal

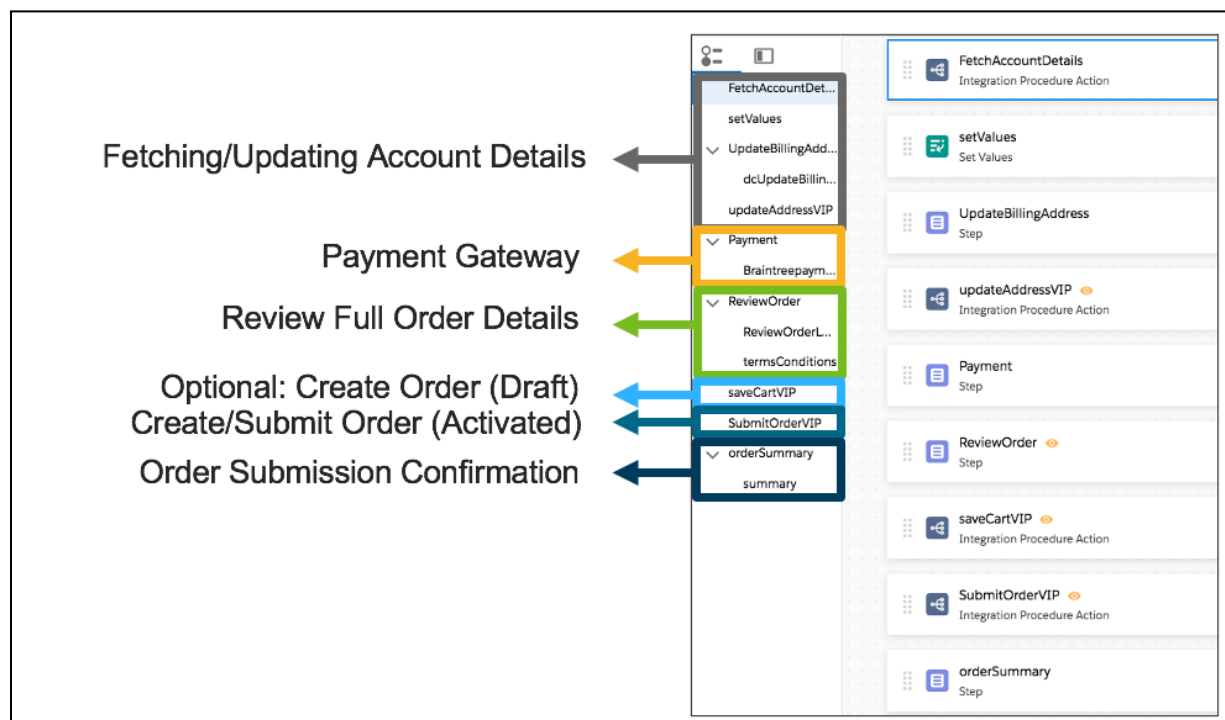
- Describe what happens during and after order creation and submission
- Explain the components and process of launching guided selling from an account record and then going to the appropriate order record page

Tasks

1. Explore the Checkout OmniScript
2. Remove the ContextId from initialiseValues
3. Configure the end of script navigation
4. Create an action to launch the script
5. Test the guided selling OmniScript

Time: 40 mins

The Sample App Checkout Phase OmniScript



Remember the very last component of the OmniScript, **DigitalCommerceLWCCheckout1?** It is a separate OmniScript being invoked to complete the end of the guided selling experience. Let's take a look.

Checkout is made up of several OmniStudio Integration Procedures and Lightning web components. In regards to the guided selling experience, you are now at the part of the app where the user will need to create an account or log in. The creation and fetching of account details is handled through integrated procedures and Lightning web components.

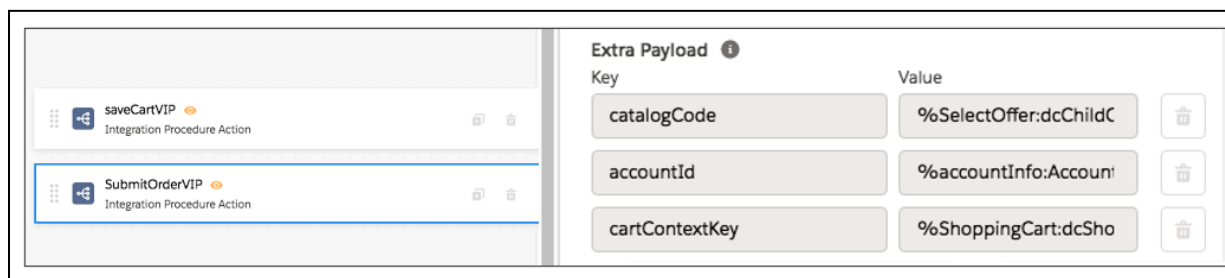
Following this, the user will need to enter their payment info. For the sample app, we are using Brain Tree Payment which is featured in the VPL library (<https://success.vlocity.com/s/process-library-detail?cardid=aBPo00000008OZGGA2&library=aBQo0000000PAsRGAW&processid=aBS1N000000GqchWAC>). We will provide you fake credit card info later so that you can test the checkout process.

Next in the guided selling experience, the user will get to review the order details, as it is displayed by the **ReviewOrderLWC** Lightning web component.

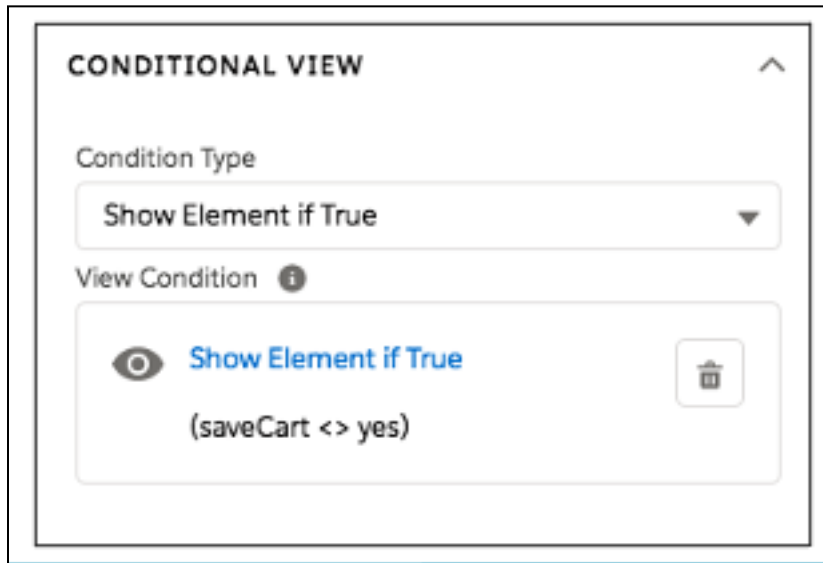
If the user chooses to save the order from the cart page, **saveCartVIP** will be used, otherwise **SubmitOrderVIP** will be used to create the order and submit it. Finally, the user will receive a confirmation of the order submission from the **summary** component. Let's examine these components in more depth.

Creating and Submitting Orders

The user can save their cart, as handled by **saveCartVIP**, which creates a draft order. Or, they can go through the full checkout process where **SubmitOrderVIP** creates and submits an order. In either case, the OmniScript will pass the **catalogCode**, **accountId**, and **cartContextKey** to each integration procedure to complete their remote actions.



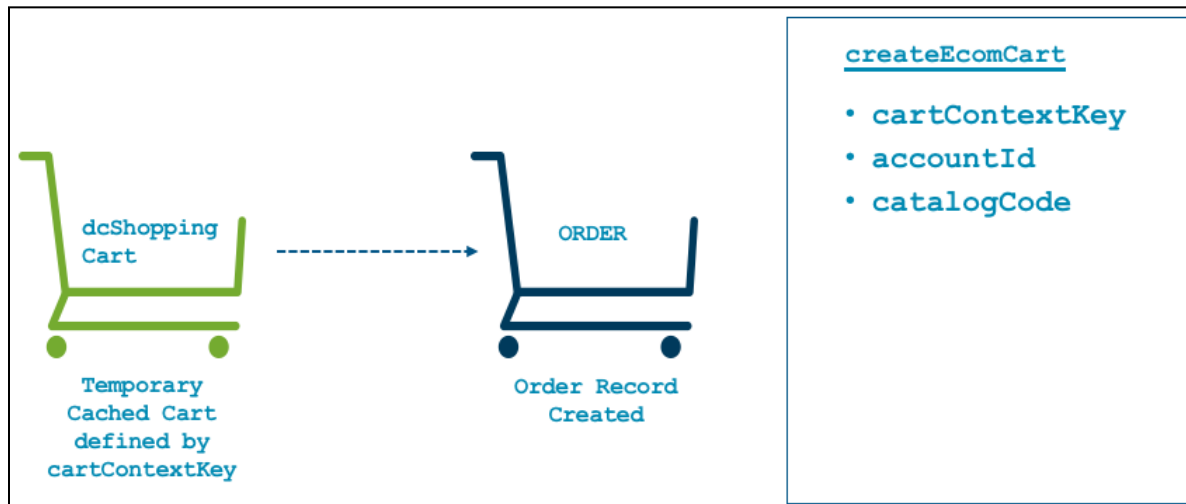
So how does the Checkout OmniScript decide whether to use the **saveCartVIP** or **SubmitOrderVIP**? The **saveCart** property is used both by the OmniScript and Lightning web components to determine whether an order is created, or whether an order is created AND submitted. You can see the **saveCart** property at work in the **CONDITIONAL VIEW** section of each OmniScript component.



A remote action in the **SubmitOrderVIP** integrated procedure called **CheckoutwithCPQ** is used to initiate the **checkout** method. This method submits the order, moving it to the next stage.

	Remote Class	Remote Method
CheckoutWithCPQ	<u>Vlocity_cmt.CpqAppHandler</u>	checkout

createEcomCart is one of CpqAppHandler's methods. It is responsible for creating an order and it validates cart product pricing with the sales catalog pricelist. You can invoke it using a Remote Action element in the OmniScript. **createEcomCart** requires a **catalogCode**, **accountId**, and **cartContextKey**. The **cartContextKey** is a value created by **dcShoppingCart** that points to the cache representation of the user's shopping cart. **createEcomCart** brings everything together for an accurate draft order record.

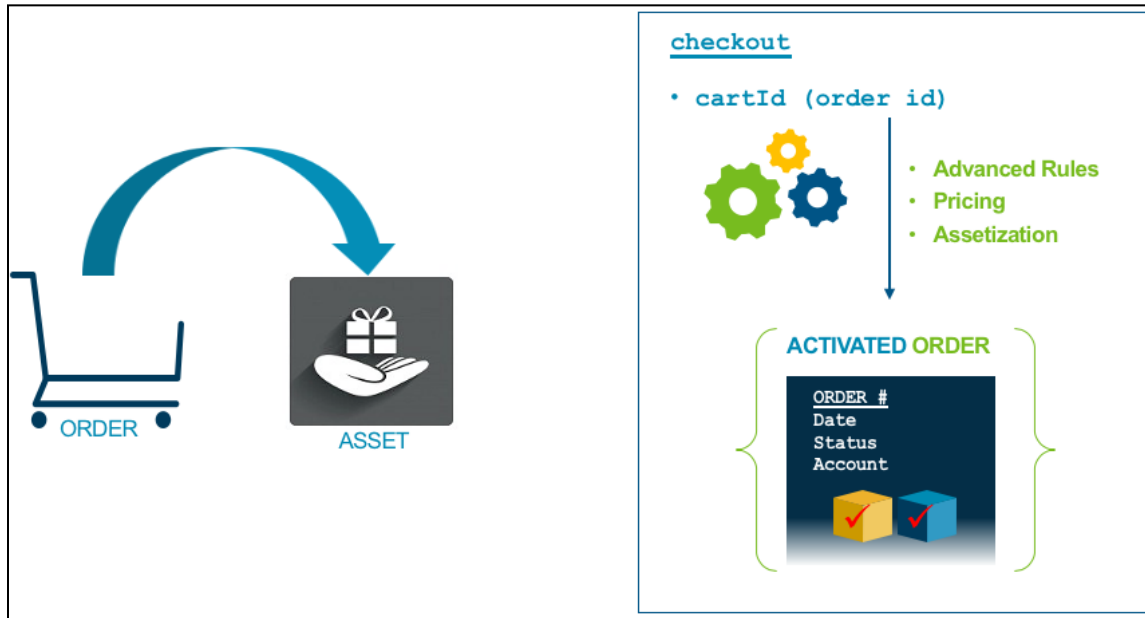


Once you've retrieved products, displayed them to the user, added them to the cart, there's one final step you don't want to forget—submitting the order! The CPQ interface, once again, comes into play and provides the checkout method to ensure every order is valid and can be fulfilled.

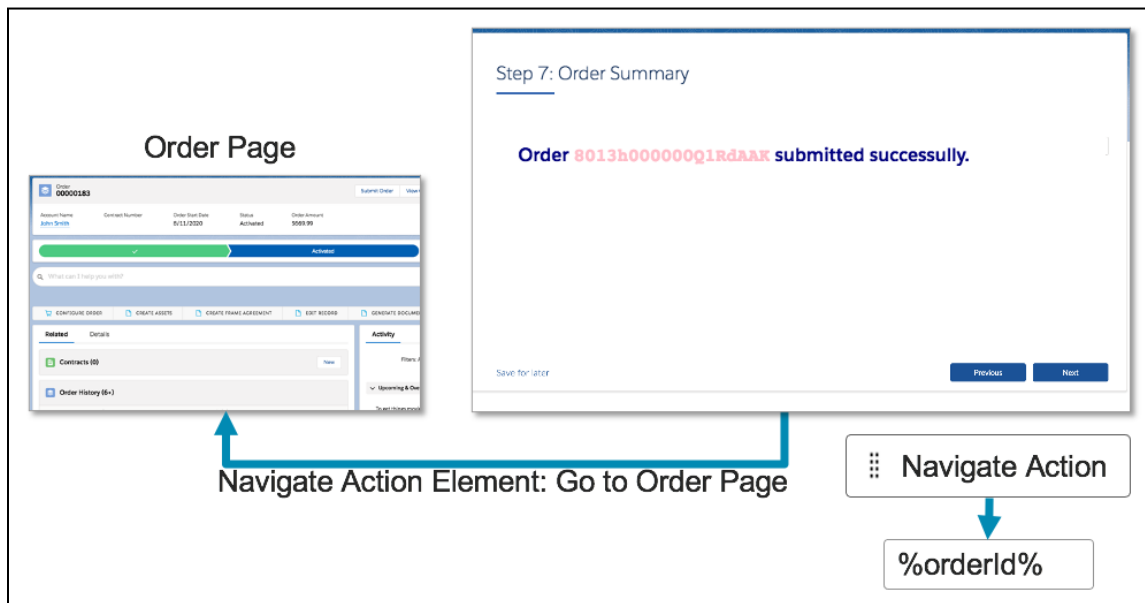
The **checkout** method requires the cartId (order id). For more information on the checkout method, see the documentation on APIs in the Industries Configure, Price, Quote Web APIs on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

Based on the header object, checkout moves the object to the next state of the ordering process. Since the header object is an **Order**, it submits the order to create assets.

When you checkout an order, rules and pricing will run one final time, and then the line items will be assetized, creating assets. The Salesforce order status will change from Draft to **Activated**.



The Navigate Action



When you're building a guided selling experience, you need to think about where the user should logically go after completing it. For the experience you've been building and configuring so far, the user is going through the steps to create and submit an order. Logically, you may want to see the order record for the newly submitted order, this is where the **Navigate** action comes in.

The **Navigate** action can take the user to a specified app, lightning component, object page, record page, and [more](#).

The **Navigate** action will use the **%orderId%** value retrieved by the **SubmitOrderVIP** Integrated Procedure to take the user to the order page.

Task 1: Explore the Checkout OmniScript

The Digital Commerce Lightning web components Checkout OmniScript create a user experience for submitting an order. This OmniScript allows a user to authenticate and then allows the user to update their billing/shipping address, specify payment details, review their order, and submit their order.

Don't worry, while there is a lot to cover and understand in this section, it will all make sense in the end. Revisit the Appendix section of this document after you finish this exercise to review the components and OmniStudio Integration Procedures in greater depth and to better solidify your overall understanding.

1. Open the Checkout OmniScript.
 - a. Right-click on the **OmniScripts** tab from the Lightning navigation bar.
 - b. Select **Open link in new tab**.
 - c. Click on **dc/lwccheckout**.
 - d. Click **Digital Commerce LWC Checkout (Version 2)**. It may take a minute to open.
2. Click the **FetchAccountDetails** rectangle. This integration allows the user to create a new account or log into their existing account and then passes that account information back into this OmniScript. For your convenience, you will not be creating new users, but using existing users.

3. Click the **setValues** rectangle.
 - a. By default, **dcReviewOrder** is set up to use the **userName** property. Because you are configuring this app for Eliza to create orders on the behalf of customers, you must reassign this property to the customer's information. For simplicity, you'll reassigned it to the customer's phone number. When you get to the order review page, instead of showing your admin email, it will show the customer's phone number.
4. Click the **UpdateBillingAddress** rectangle.
5. Click on the **dcUpdateBillingAddressLWC** component. This component renders the form that allows the user to submit a billing and shipping address. This data is handled by the next component.
6. Click on the **updateAddressVIP** component. This component allows the user to update their billing and shipping addresses. In your case, these details will update the customer's billing and shipping fields on their record.
7. Click the **Payment** rectangle.
8. Click on the **BraintreepaymentLWC** component.
 - a. Click on **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**. This component utilizes the **checkout-payment-url** specified in **InitialiseValues** in the DCLWCSampleOS OmniScript. For the sample app, it is pointing to the [Braintree](#) Payment Gateway hosted on Heroku. When it comes to your checkout solution, it will need to point to the URL of your payment gateway, as decided by the implementation team.

This component renders an interface for the user to enter their credit card details and submit for payment. The OmniScript will perform a remote call to the API to validate the credit card details and confirm payment with the supplied **total** cost provided from **dcShoppingCart**.

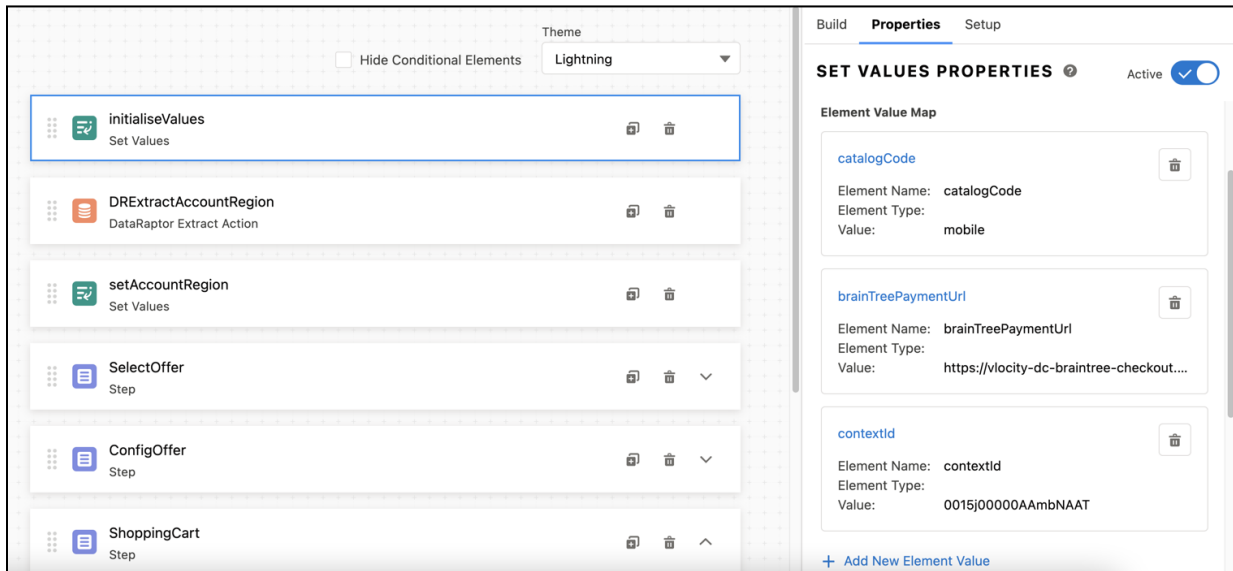
9. Click the **ReviewOrder** rectangle.

10. Click on the **ReviewOrder** step.
 - a. Notice under **CONDITIONAL VIEW**, it is only using this component if **saveCart** does not equal **yes**. If the user opted to save their cart from the Cart page rather than checking out, this OmniScript would not use this step, **updateAddressVIP**, or **SubmitOrderVIP**.
11. Click on the **ReviewOrderLWC** component.
 - a. Click on **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**. This component utilizes values passed from the **BraintreepaymentLWC** and **updateAddressVIP** components. This component renders billing, shipping, transaction, and user info details for the user to review before submitting their order.
12. Click on the **termsConditions** component. This component is a simple Checkbox that is fully editable within OmniScript Designer and [can be configured](#) to prevent the user from advancing past this **ReviewOrder** step. It has no consequence to our buy flow as it is currently configured but can be linked to a real set of terms and conditions in the future.
13. Click on the **saveCartVIP** component. If the user clicks the save cart button from the cart page, the Checkout OmniScript will use the saveCartVIP integration procedure. This procedure uses an Apex remote action to create a new order for the products in the user's cart.
14. Click on the **SubmitOrderVIP** component. This integration procedure creates an order (if saveCartVIP was not used, i.e., no orderId is present in the OmniScript JSON), validates it, and submits it.
15. Click the **orderSummary** rectangle.
16. Click on the **summary** component. This component presents a confirmation message to the user and utilizes the **orderId** value received from the previous **SubmitOrderVIP** integration procedure.

Task 2: Remove the ContextId from initialiseValues

Because you are going to pass the ContextId value into the OmniScript using an OmniStudio Action in this exercise, you will remove the hardcoded ContextId from the **initialiseValues** component.

1. Switch to your open browser tab for **DCLWCSampleOS**.
2. Click the **Design** button at the top of the page.
3. Click the **Deactivate Version** button.
4. Click the **Done** button.
5. Click **initialiseValues** rectangle.
6. In the **Element Value Map** section, click the  **trash can icon** on the right side of the **ContextId** row.



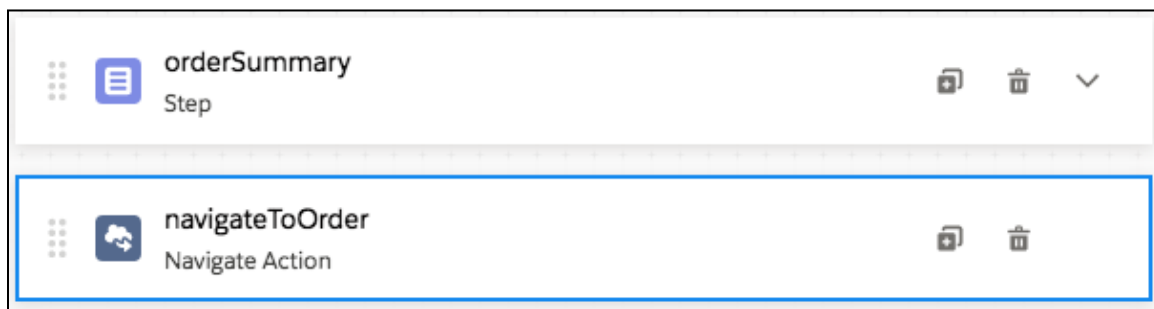
The screenshot displays the Salesforce OmniStudio interface. On the left, a list of components is shown, with 'initialiseValues' (Set Values) highlighted. On the right, the 'Properties' tab is active, showing the 'SET VALUES PROPERTIES' section. The 'Element Value Map' is visible, listing three elements: 'catalogCode', 'brainTreePaymentUrl', and 'contextId'. Each element has a trash can icon on the right. The 'contextId' element is currently set to the value '0015j00000AAmbNAAT'.

Element Name	Element Type	Value
catalogCode	mobile	
brainTreePaymentUrl	https://vlocity-dc-braintree-checkout....	
contextId	0015j00000AAmbNAAT	

Task 3: Configure the end of script navigation

Now you need to create a way for the user to return to the account page after going through the guided interaction.

1. Switch to your open browser tab for **Digital Commerce LWC Checkout**.
2. Click the **Deactivate Version** button.
3. Click the **Done** button.
4. In the **Build** panel, expand the **ACTIONS** components.
5. Drag a **Navigate Action** below the **orderSummary** step.
6. Click on the **NavigateAction1** component.
7. In the **NAVIGATE ACTION PROPERTIES** panel:
 - a. Change the **Name** to `navigateToOrder`.
 - b. Change the **Field Label** to `Go to Order`.



8. Modify the Navigate Action settings.
 - a. Click the checkbox next to **Replace**.
 - b. Enter `Order` for **Object API Name**.
 - c. Change **Record Id** to `%orderId%`.

NAVIGATE
ACTION PROPERTIES

Active 

Name

navigateToOrder

Field Label

Go to Order

Button Variant

brand

Icon Name

Page Reference Type

Record

☒ Replace

Object API Name

Order

Target Parameters

Record Action

View

Record ID

%orderId%

9. Activate the Checkout OmniScript.

- At the top of the page, click **Activate Version**. Wait for it to finish activating before moving on.
- Click the **Done** button.

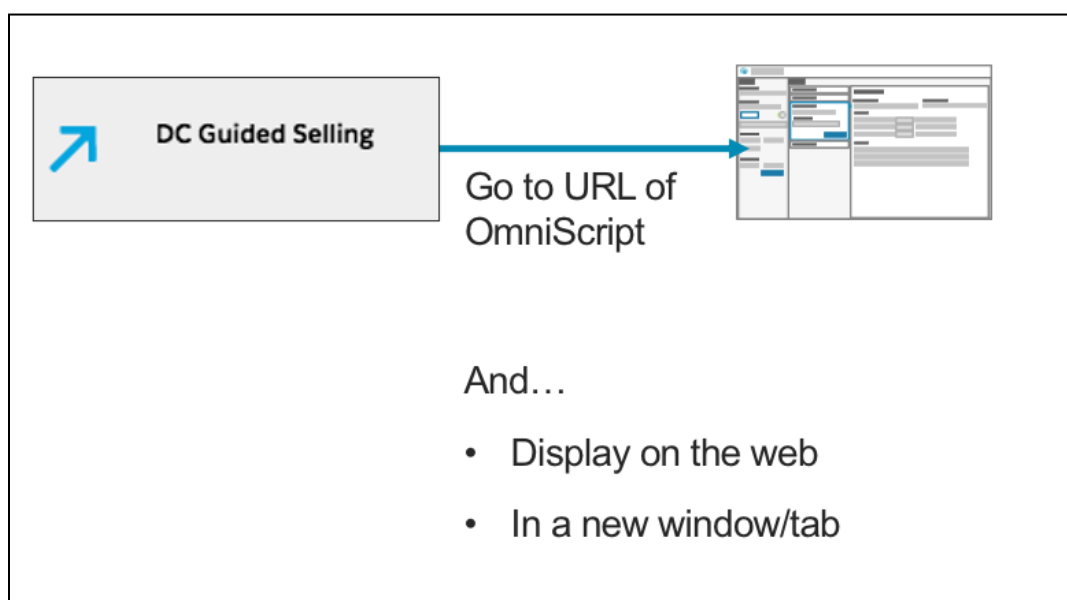
10. Switch to your open browser tab for **DCLWCSampleOS**.

11. Activate the DCLWCSampleOS OmniScript.

- At the top of the page, click **Activate Version**. Wait for it to finish activating before moving on.
- Click the **Done** button.

With the script completed, Eliza just needs to create a way to launch it. She'll use an OmniStudio Action, so that it can be launched using a button from the Account page.

OmniStudio Actions



OmniStudio Actions are automatically generated URLs that launch OmniScripts, OmniStudio FlexCards components, web pages, or external applications. Actions are typically specific to a given object type, such as Account, Contact, Policy, or Asset.

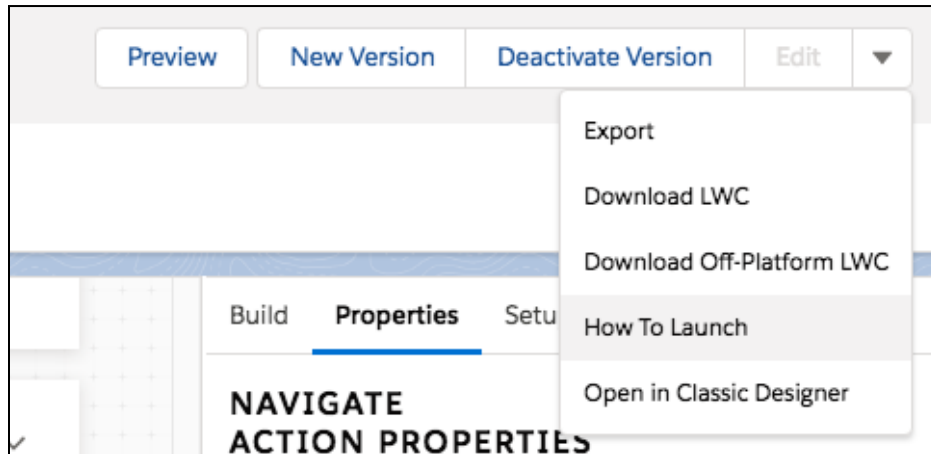


RESOURCE:

Need a refresher on OmniStudio Actions? Take the [Successful Interactions with Omniscript](#) course.

Task 4: Create an action to launch the script

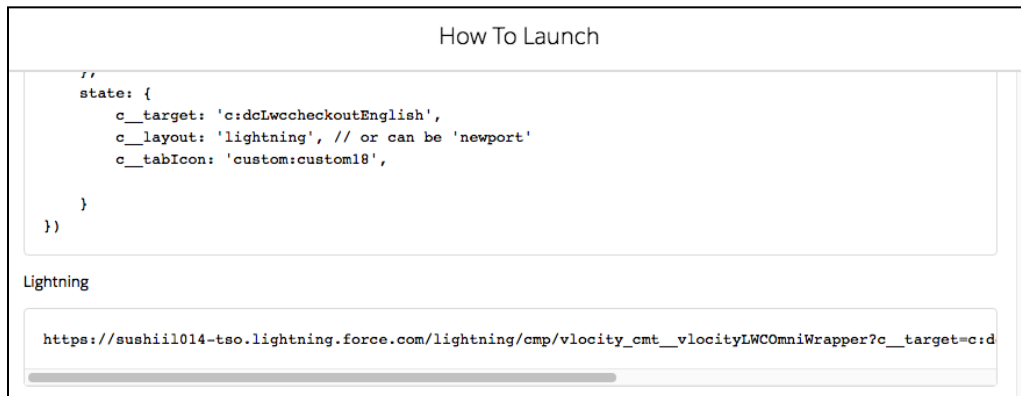
1. Copy the URL for the OmniScript by completing the following steps:
 - a. At the top of the page, click the **down arrow** .
 - b. Click the **How To Launch** link.



- c. Below the **Lightning** section in the URL box, copy the URL starting from “/lightning”.
 - d. Paste the URL into a text editor such as Notepad, TextEdit, or a document.
 - e. Append `&c__ContextId={0}` to the end of the URL.

This allows you to pass the Context Id of the page that you are launching this guided experience from into the Set Values component within the OmniScript. You should have a URL that looks like this:

```
/lightning/cmp/vlocity_cmt__vlocityLWC OmniWrapper?c__target=c:dcLwcEnglish&c__layout=lightning&c__tabIcon=custom:custom18&c__ContextId={0}
```



- f. Click **Done** to close the **HOW TO LAUNCH** dialog.
2. Click the Lightning App Launcher  , search for **Vlocity Actions**, right-click it, and then select **Open link in new tab**.
3. Click the **New** button.

4. In the **New Vlocity Action** dialog, enter the following information.

Field	Value	Notes
Vlocity Actions Name	DC Guided Selling	
Applicable Type	Account	Remove All . The action button will only appear on the account.
Active	✓	The action must be active to be available.
Display Label	DC Guided Selling	This label will appear on the action button.
Display Sequence	10	The action button will be the first to display if there is more than one action.
Display On	All	Display the action on the web and mobile.
Vlocity Icon	icon-v-avatar-group-circle-line	This icon will appear on the action button next to the button label.
Open URL in	New Tab / Window	Opens guided selling in a new tab.
Link Type	OmniScript	Indicates the type of link for the action.
Target URL	[Paste the URL you copied earlier in this task]	This is the URL of the guided selling OmniScript.
URL Parameter	Id	This is the current id (which will be the account Id) and it will be passed to the OmniScript as the context Id.

Vlocity Action

DC Guided Selling

Related

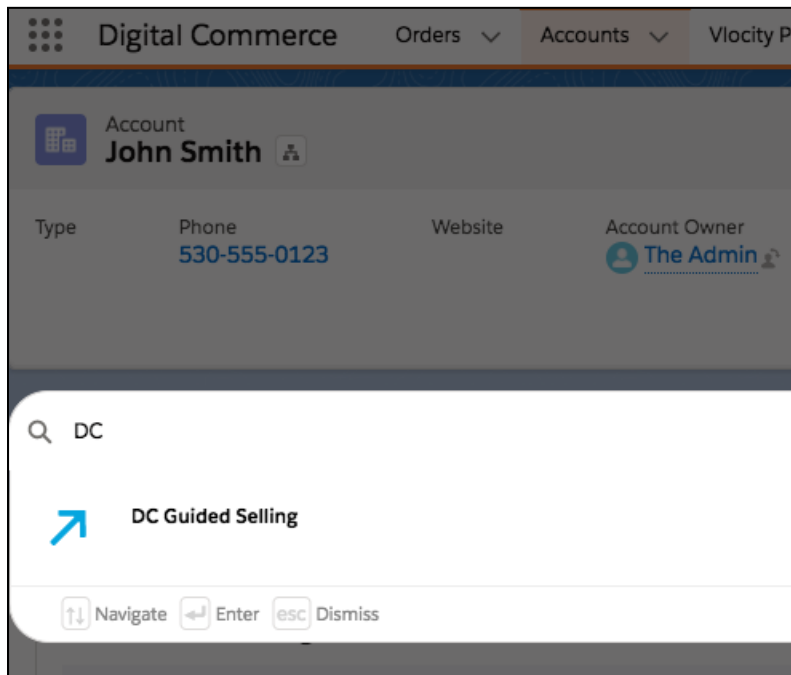
Details

Vlocity Actions Name	Target URL
DC Guided Selling	lightning/cmp/vlocity_cmt__vlocityLWComniWrapper? c__target=c:dcLwcEnglish&c__layout=lightning&c__tabIcon=custom:custom18&c__ContextId={0}
Applicable Type	URL Parameter
Account	Id
Applicable User Profile	Filter
All	
Active	Additional Filter
☒	
Display Label	State Model
DC Guided Selling	
Display Sequence	To State
10	
Display On	Invocation Class Name
All	
Vlocity Icon ⓘ	Invocation Method Name
icon-v-avatar-group-circle-line	
Open URL in	Validation Class Name
New Tab / Window	
Link Type	Validation Method Name
OmniScript	
Is Seed Action	
☐	

5. Click **Save**.

Task 5: Test the guided selling OmniScript

1. Select the **Accounts** tab from the Lightning navigation bar.
2. Open the account page for **John Smith**.
3. In the Power Launcher's **What can I help you with?** box, type **DC**.



4. Click **DC Guided Selling**.

The guided selling experience should load. If it takes longer than a minute to load, try refreshing the page.

5. Click the **Load More** button.
6. Select **Apple iPhone 7**.
7. Select a **color** and then click the **Add to Cart** button on the Config Offer page. If any errors pop-up, click the **Add to Cart** button again.

8. Click the **Check Out** button on the Cart page.

9. Enter the **Billing Address** information:

Field	Value
Address Line	1012 Layton Rd
City	Redding
State	CA
Zip Code	96002

10. Click the checkbox next to **Same as billing address**.

11. Click **Next**.

12. Enter the following **Payment** info:

Field	Value
Card Number	4111 1111 1111 1111
Expiration Date	09/24
CVV	444
Postal Code	4444

13. Click **Submit Payment**.

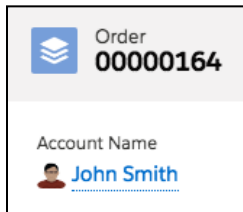
14. Notice the **Contact Information** lists **John Smith** and the phone number from his account record.

15. Click the **Next** button to submit the order.

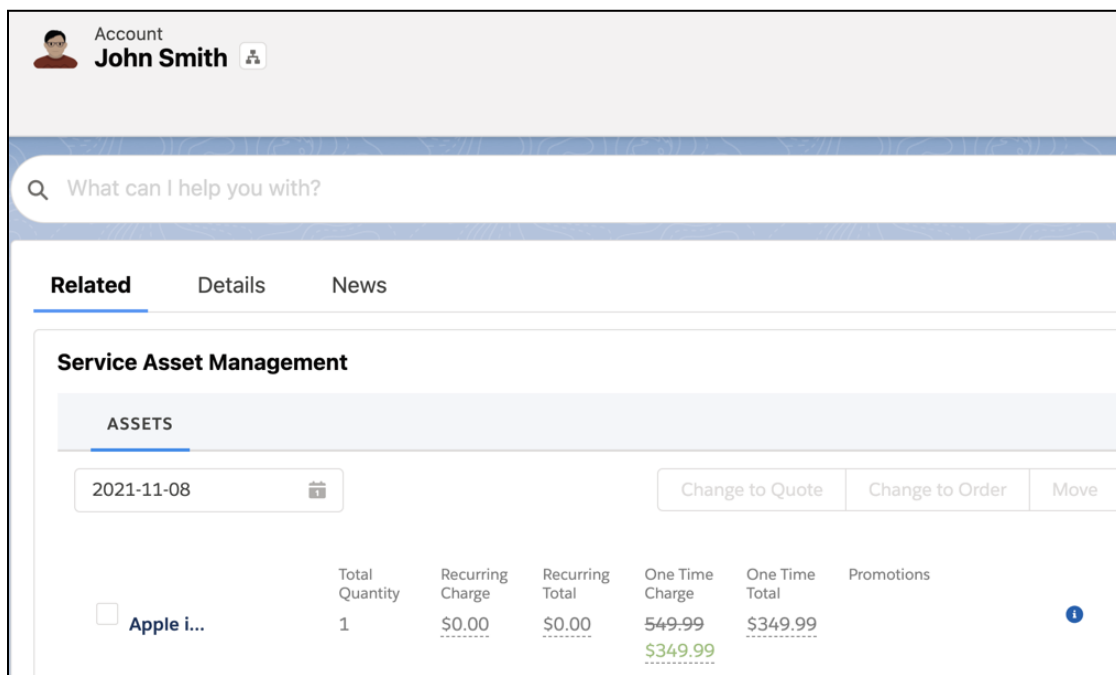
16. Click the **Next** button to continue.

Notice how the OmniScript Navigate action takes you to the appropriate order page.
Notice the order Status is **Activated**.

17. Click the **John Smith** link, located below **Account Name**.



Notice the **Apple iPhone 7** is listed under John's **Service Asset Management**.



Account
John Smith

What can I help you with?

Related Details News

Service Asset Management

ASSETS

2021-11-08 

[Change to Quote](#) [Change to Order](#) [Move](#)

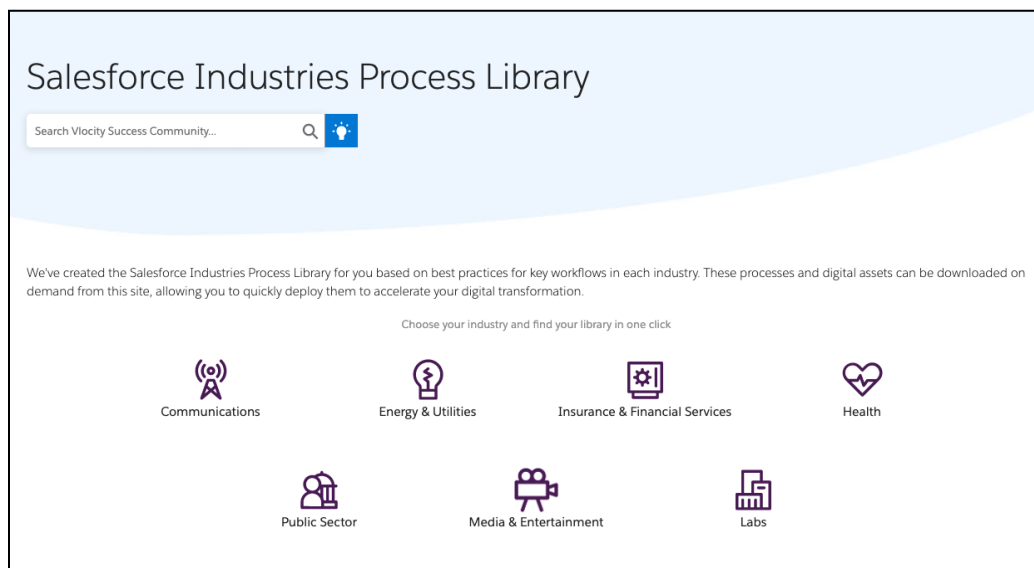
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	Promotions
☐ Apple i...	1	\$0.00	\$0.00	549.99 \$349.99	\$349.99	

Cart-Based Guided Selling Overview

Cart-Based Guided Selling Examples

The Salesforce Industries Process Library has several prebuilt examples of guided selling that you can download and customize for your processes. In this module, you will customize a guided selling OmniScript from the Salesforce Industries Process Library that has been downloaded and included in your training playground.

To review the other example guided selling processes, go to the Process Library on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>. Click **Communications**, and then review the processes under **B2B Sales** and **B2C Sales**.

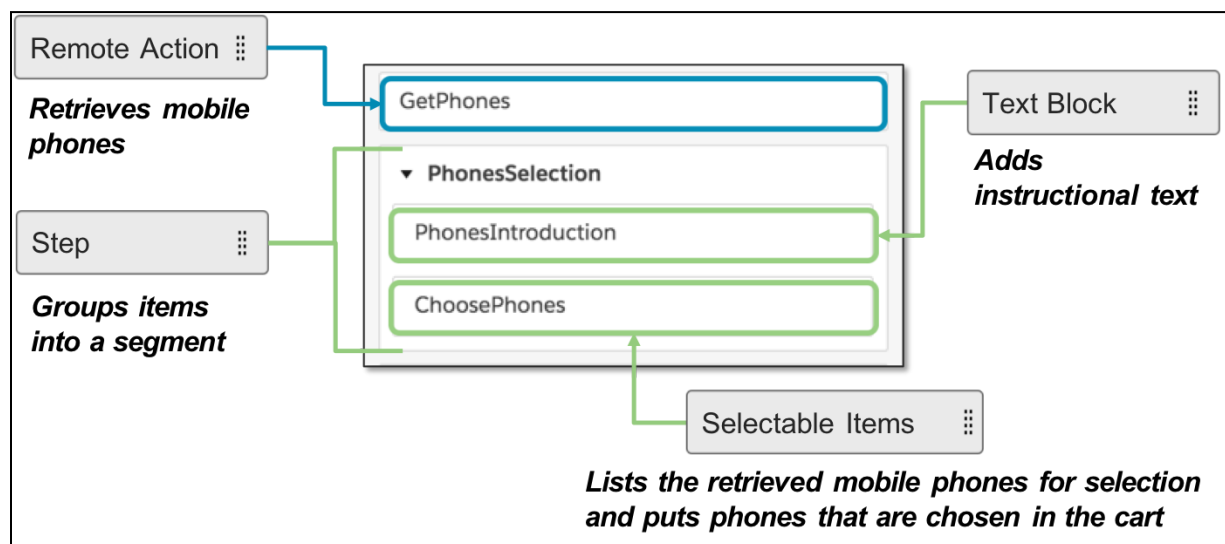


Creating the Business Process Flow in OmniScript

To create a business process flow in the cart-based guided selling interaction, you use these OmniScript components:

- A **remote action** component to retrieve specific products
- A **step** component to group items into a segment of the interaction

- A **text block** component to provide instructional text
- A **selectables items** component to list the product retrieved by the remote action and then place any items chosen for purchase in the cart



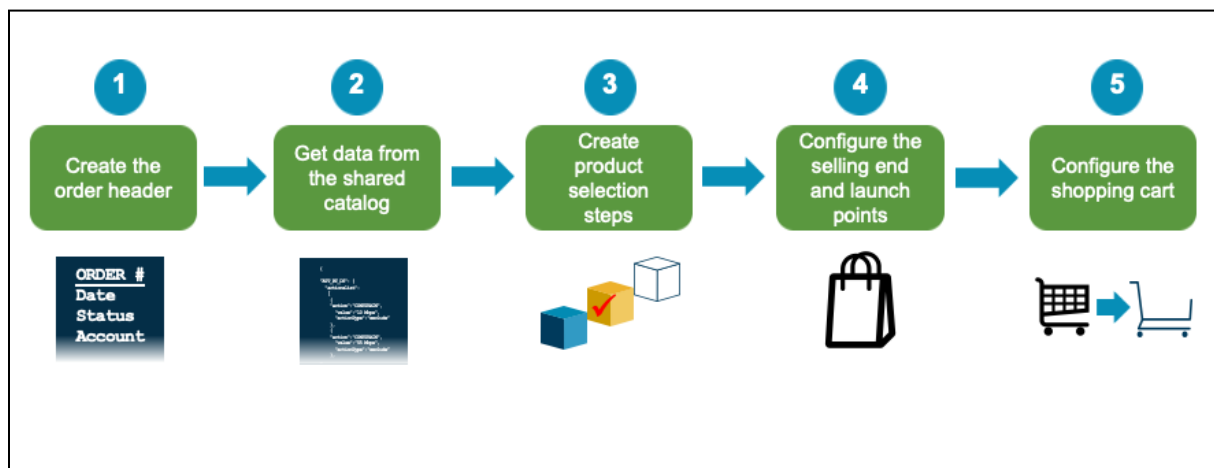
For more information on OmniScript component types, find OmniScript in the Documentation on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

Building a Cart-Based Guided Selling Experience

With a design plan in hand, you can then begin configuring the guided selling experience.

1. **Create the order header.** Before you can add products to the cart, you must create an order to hold the line items. (Or you could create a quote or an opportunity, if you're designing a B2B guided selling experience.)
2. **Get data from the shared catalog.** Thinking about the products you want to sell during the guided selling experience, how will you retrieve them from the shared catalog?
3. **Create product selection steps.** Once you've retrieved the product data from the shared catalog, how should it be organized in each step of the selling experience?

4. **Configure the script end and launch points.** You need to configure where the user goes after completing the selling experience, and also a way to launch the guided selling script.
5. **Configure the shopping cart** in the script. Should it be horizontal across the bottom or vertical, along the side? What should it look like?



Exercise 8-7: Create the Order Header

Scenario

The call center manager wants talk-time statistics reduced. She's thinking a guided interaction for agents to sell products to customers would help. She's asked Eliza for a demo in less than two hours to see how guided selling works. Eliza knows there is an example OmniScript for B2C guided selling in the Salesforce Industries Process Library. She'll use that to get a prototype running ASAP.

The next step is to create an order that will hold the line items when products are added to the cart. She can create orders from within an OmniScript using the CPQ interface, which will ensure that the orders created through guided selling are as perfect as the ones created during order capture using the Cart.

Goal

- Explain how and why the CPQ interface, CpqAppHandler, is used in guided selling
- Describe the purpose of the createCart method
- Understand what input is required by the createCart method and why

Tasks

1. Prepare the sample OmniScript
2. Create the order header
3. Set the Order Id value
4. Set the Context Id value
5. Test the CreateOrder remote action

Time: 15 mins

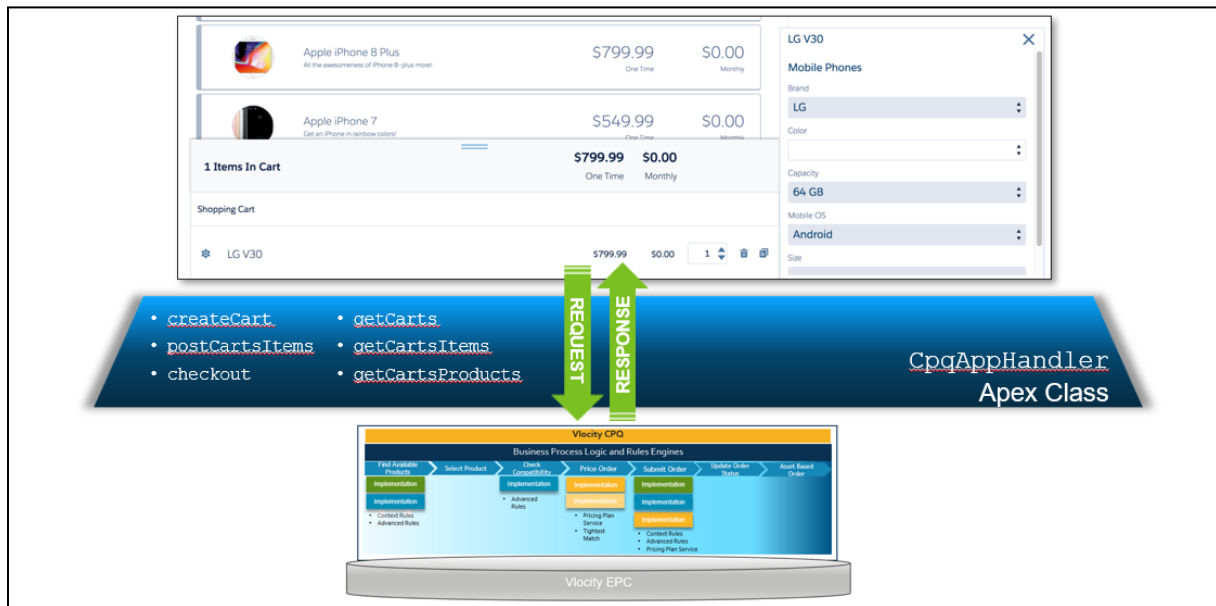
Using CPQ in Guided Selling

Just like any OmniScripts, a guided selling OmniScript needs data in order to perform its guiding magic. Since it's an OmniScript, you can use all of the standard data components (DataRaptor, REST and Remote Actions) to move data in and out of the script.

However, when you are using OmniScript in a guided selling context, you need more than just data. You need all of the business logic and rules that govern what products are available to which customers at what prices—in other words, CPQ. In order to access product and pricing data via CPQ, you must request that data using a CPQ interface. A CPQ interface guarantees that the correct logic is applied, making sure that every order is a perfect order, even when you create the order during a guided experience.

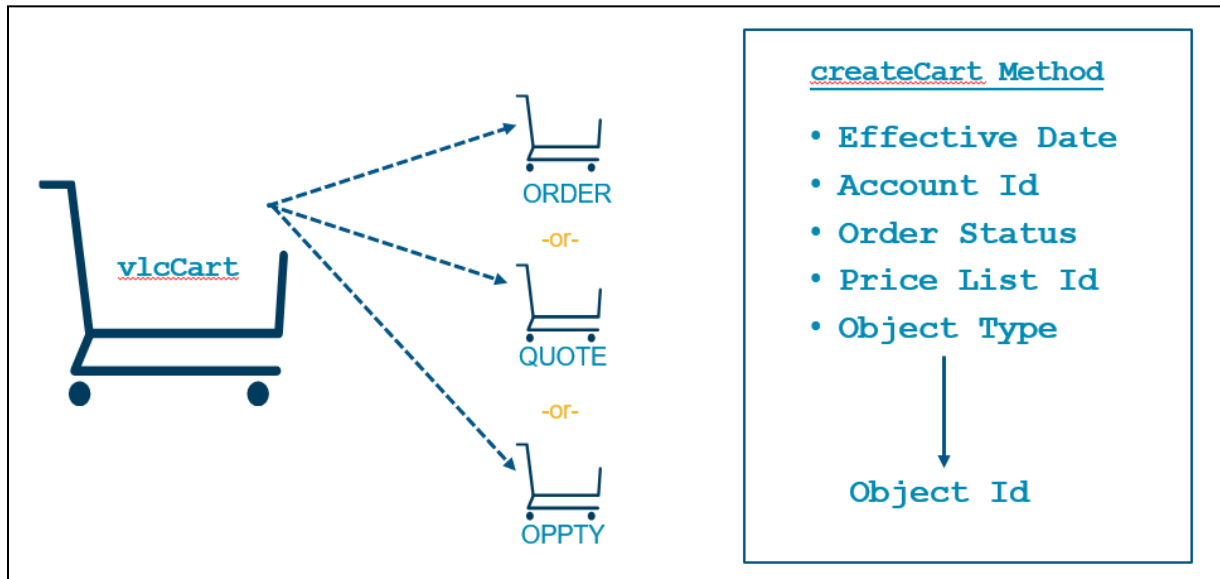
When you want to create orders using a CPQ interface from within Salesforce, you use `CpqAppHandler`. `CpqAppHandler` is a specialized global Apex class that includes a series of methods to perform all of the CPQ functions. These same methods are also exposed via REST interface (called the Cart-based APIs) and they are used when you need to invoke the CPQ interface from outside the Salesforce application.

CpqAppHandler and its methods



createCart Method

`createCart` is one of CpqAppHandlers methods, and it is used to create orders, quotes and opportunities. You can invoke it using a Remote Action component in the OmniScript. Similar to when you create an order manually, `createCart` requires a date, an account, a status, a price list, and the type of object (order, quote or oppty). When given all of these things in the correct format, `createCart` will return the newly created object id.



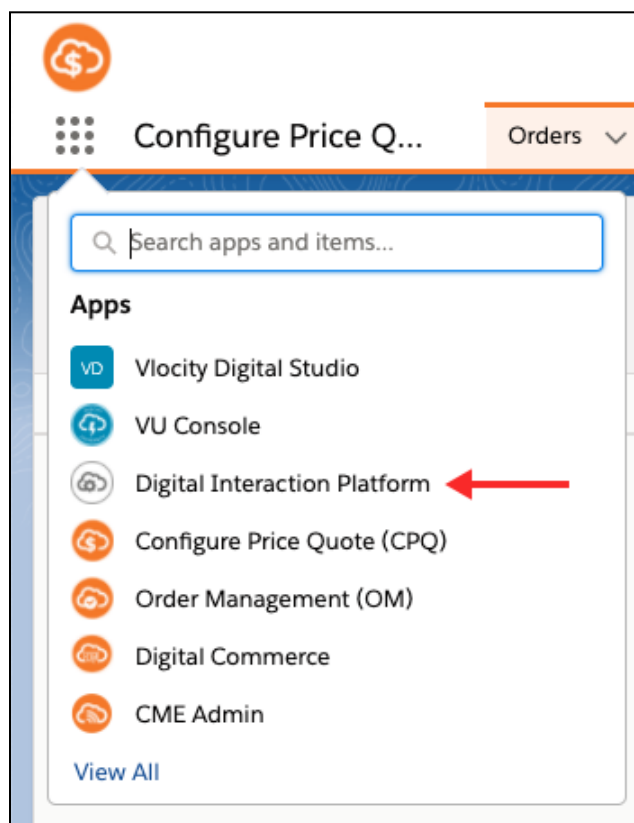
Task 1: Prepare the sample OmniScript



NOTE:

For this exercise, the Customer Mobile Order OmniScript has been included in your org. This is the OmniScript you will configure to create a guided selling interaction. You will start out by removing parts of the OmniScript not relevant to Eliza's guided selling design needs.

1. Open the OmniScript.
 - a. Click the **Lightning App Launcher**  in the upper left-hand corner of the screen.
 - b. Click **Digital Interaction Platform**.



- c. Click the **OmniScripts** tab.

- d. Click the expand icon next to **Guided Selling/B2C Bundle Order**.
 - e. Click **Customer Mobile Order (Version 2)** to open it for editing.
2. Create a new version.



BEST PRACTICE:

It is best practice to create a new version when you need to make changes to the original. This allows you to revert back to the original should you need to do so.

- a. As the script opens, you will be on **Script Configuration** component. At the bottom of the **PROPERTIES** pane, click **Create Version**.

Version	2
Active	Yes
Owner	Vlocity CME 107 Admin
Created By	Vlocity CME 107 Admin 8/27/2020 11:27AM
Last Modified By	Vlocity CME 107 Admin 8/27/2020 11:27AM

DeleteCreate VersionDeactivate VersionExport

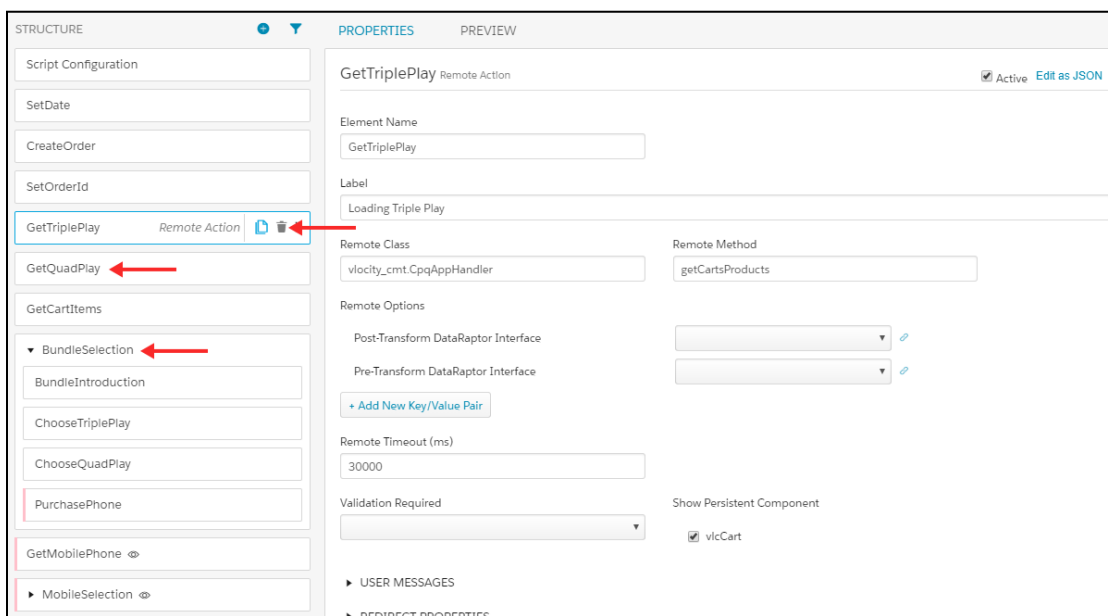
- b. Notice the version number is now 3.

Version	3
---------	---

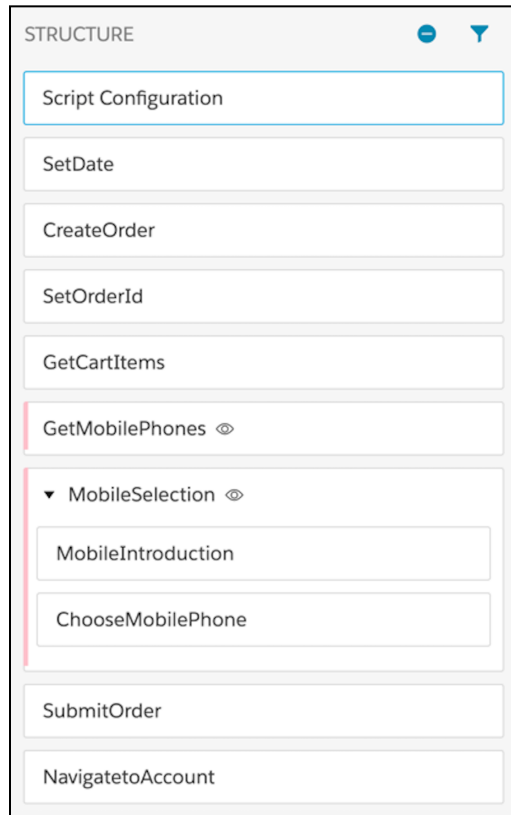
3. From the **STRUCTURE** panel, delete the elements which will not be needed.

- GetTriplePlay
- GetQuadPlay
- BundleSelection
- GetExtras
- GetChannels
- SpecialOffer
- ExtrasSelection

To delete an element, hover over it so the icons appear. Click the **delete** icon  and click **Ok** in the **CONFIRM DELETION** window to delete the element.



4. When you've deleted the elements that you don't need, you should have the following structure.



If you've accidentally deleted too many elements, don't worry! Just delete this version of the OmniScript and start again.

Task 2: Create the order header

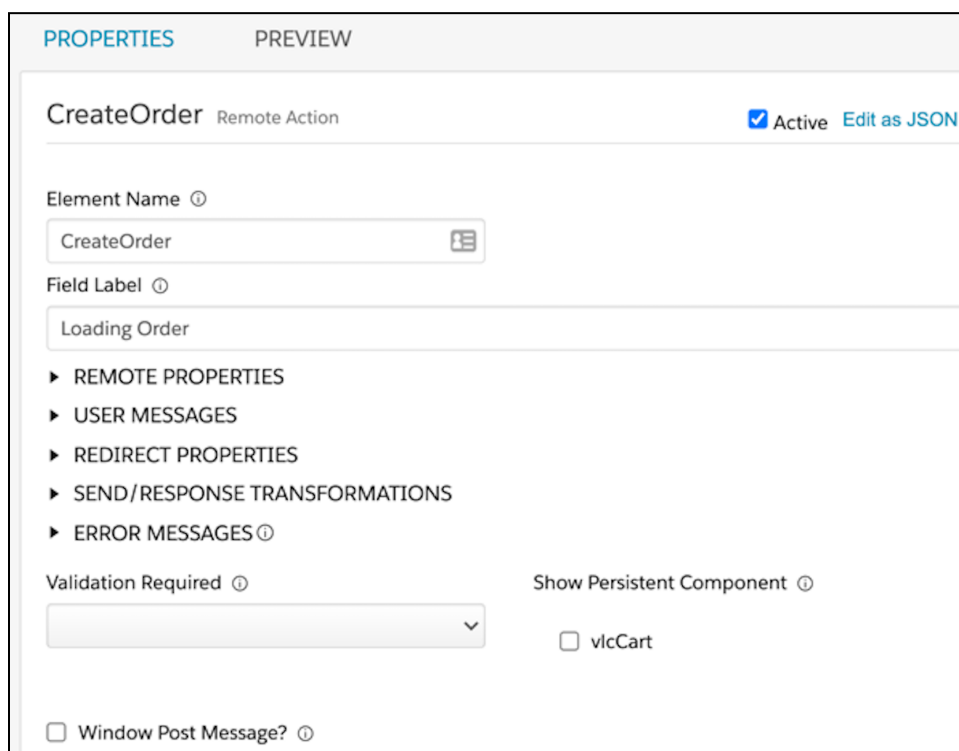
1. Review the **CreateOrder** remote action in the OmniScript.



NOTE:

Remote actions are OmniScript components that call Apex classes. For more information, see OmniScript Remote Action documentation on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

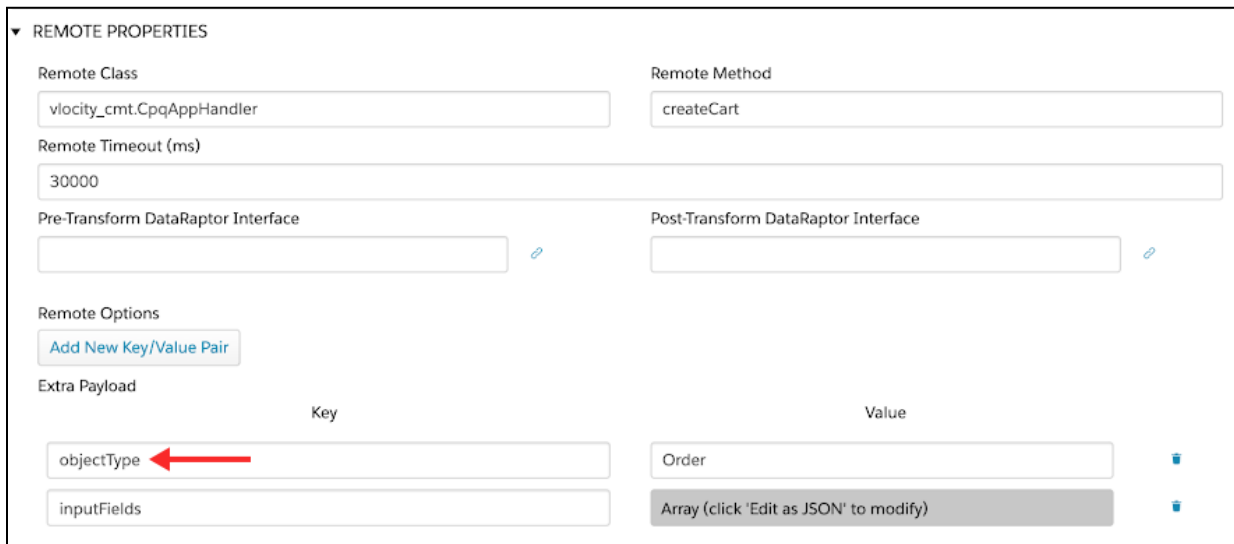
- a. In the **STRUCTURE** pane, click **CreateOrder**. This element invokes CpqAppHandler and uses the remote method **createCart** to create an order.
- b. Notice that under **Show Persistent Component**, **vlcCart** is selected, which means that the cart will display during this step. However, since this is a remote action, it's not necessary to display the cart during this step, so you can uncheck this box.



The screenshot shows the configuration interface for a 'CreateOrder' Remote Action. The interface has two tabs: 'PROPERTIES' (selected) and 'PREVIEW'. The 'CreateOrder' Remote Action is shown as 'Active' with an 'Edit as JSON' link. The configuration fields include:

- Element Name**: CreateOrder
- Field Label**: Loading Order
- REMOTE PROPERTIES**: (expandable section)
- USER MESSAGES**: (expandable section)
- REDIRECT PROPERTIES**: (expandable section)
- SEND/RESPONSE TRANSFORMATIONS**: (expandable section)
- ERROR MESSAGES**: (expandable section)
- Validation Required**: (dropdown menu)
- Show Persistent Component**: ☐ vlcCart
- Window Post Message?**: ☐

- c. Within **REMOTE PROPERTIES**, and under **Extra Payload**, notice the **objectType**. This indicates the object type that createCart should create is an order, instead of an opportunity or quote.



▼ REMOTE PROPERTIES

Remote Class: vlocity_cmt.CpqAppHandler

Remote Method: createCart

Remote Timeout (ms): 30000

Pre-Transform DataRaptor Interface: [Link]

Post-Transform DataRaptor Interface: [Link]

Remote Options: Add New Key/Value Pair

Extra Payload

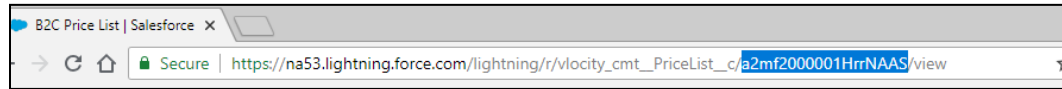
Key	Value
objectType	Order
inputFields	Array (click 'Edit as JSON' to modify)

- d. The **inputFields** are passed in an array, which can't be added in the UI. Therefore, to edit the input fields, you must edit the JSON.

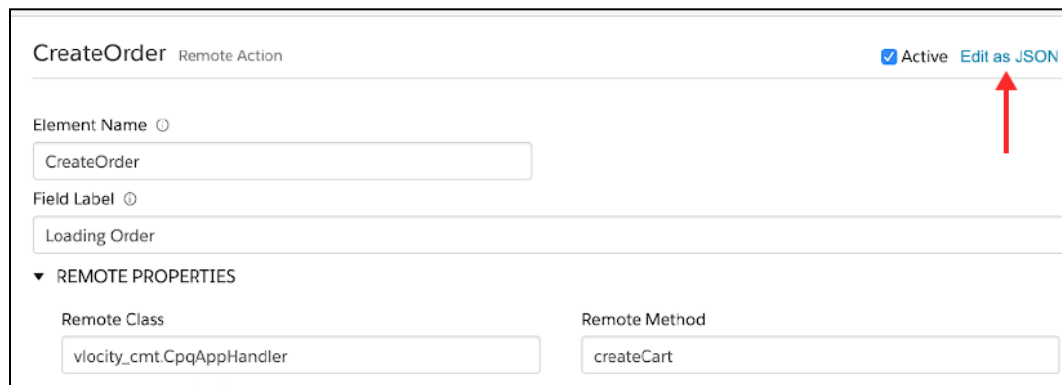
The createCart method requires a price list id, so you must edit the JSON so you can add this training playground's price list id to the list of inputFields.

- Get the price list Id for the **B2C Price List**. You need to pass the price list Id to createCart.
 - In the **Lightning tab navigation** bar, right-click on any tab to open it in a new browser tab.
 - In the **Lightning search dialog**, enter `new mobile plan`. A single order should display.
 - Click the **order number**, and then click the **Details** tab to view the order details.
 - On the **Details** page, find the **Price List** field, and click **B2C Price List**.

- e. Copy the portion of the address (the URL in your browser) directly after PriceList__c/ and before /view, as shown here:



- f. Close the current browser tab and return to the browser tab that has OmniScript Designer and the **CreateOrder** element open.
3. Set the price list in the input fields for createCart.
 - a. Scroll up to the top of the page and click **Edit as JSON**.



- b. In the CreateOrder JSON, expand the display window or scroll down the page.
- c. Under “**extraPayload**”, notice that **%ContextId%** is being passed to createCarts as the account Id. This assumes that this script will be launched in the context of an account and the account Id will be passed in as its context id.
- d. Also notice that this sample script used a pricebook Id to create the order. This must be updated to use a price list Id.

```

"extraPayload": {
  "objectType": "Order",
  "inputFields": [
    {
      "effectivedate": "%TODAY%"
    },
    {
      "status": "Draft"
    },
    {
      "AccountId": "%ContextId%"
    },
    {
      "pricebook2Id": "01s1I000000XWFB"
    }
  ]
},

```

- e. In the JSON, replace the existing pricebook input with the price list object API name and the id of your training playground's price B2C Price List.

Replace	New Value	Notes
pricebook2Id	vlocity_cmt__PriceListId__c	Replace the value between the quotation marks. This replaces the reference to a price book.
"01s1I000000XWFB"	[your org's B2C Price List Id]	Price list record ids begin with a2m.



ALERT:

Be very careful when editing JSON. JSON code is case sensitive, therefore it is imperative that you enter the new values exactly as described.

```
"extraPayload": {
  "objectType": "Order",
  "inputFields": [
    {
      "effectivedate": "%TODAY%"
    },
    {
      "status": "Draft"
    },
    {
      "AccountId": "%ContextId%"
    },
    {
      "vlocity_cmt__PriceListId__c": "a2m1U0000003Nq1QAE"
    }
  ]
},
```

- f. In the upper-right corner, click **Edit in Property Editor** to exit the JSON view.



Now that you've given the createCart method the input it needs, it can successfully create an order and return the order's record id. Since the order id will be needed by other script elements, the next step is to create a set values component to make the order id available to all other script elements. Fortunately, this has already been done in the sample script, so you can simply review it.

Task 3: Set the Order Id value

1. In the **STRUCTURE** panel, click the **SetOrderId** set values component.

Notice under **Element Value Map** that there is an element called **OrderId**. All subsequent CpqAppHandler remote actions will use the OrderId to add products to the correct order.

This Id is returned from the createCart method as a JSON array. To get the Id from the array, you can use a formula to extract the value.



NOTE:

You can create this string using the formula builder. To access the formula builder, click the **fx** button to the left of the value.

2. Also notice that under **Show Persistent Component**, **vlcCart** is *not* selected. Because this element sets a value that does not affect the user interface, the cart does not need to display.

SetOrderId Set Values Active Edit as JSON

Element Name ⓘ

Field Label ⓘ

Element Value Map

Element Name	Type	Value
OrderId ←	JSON Node	=STRING(%records.Id%) fx

+ Add New Value

☐ Window Post Message? ⓘ

☐ Session Storage Message? ⓘ

Show Persistent Component ⓘ

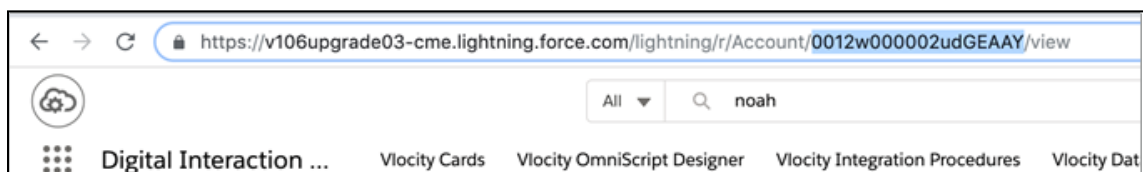
☐ vlcCart ←

Task 4: Set the Context Id value

In addition to setting a value to share an order id, it's often helpful to create a context id set values element, so that you don't have to manually add the context id each time you preview your OmniScript. This special set values element is ignored at run-time, so it's safe to use it as a design-time convenience.

This script will be launched in an account context, so it will use an account id as its context id. So, go get an account id and add it to a Set Values element list.

1. Copy the account id for **Noah White**.
 - a. In the **Lightning tab navigation** bar, right-click on any tab to open it in a new browser tab.
 - b. In the **Lightning search dialog**, enter `noah`.
 - c. In the search results list, click the **White, Noah** account.
 - d. Copy the characters of the URL directly after `Account/` and before `/view`.



- e. Close the current browser tab and return to the browser tab that has OmniScript Designer and the **SetOrderId** element open.
2. Add the ContextId to the Set Values element.
 - a. In the **STRUCTURE** panel, click the **SetDate** element (which is also a Set Values component).
 - b. Under **Element Value Map**, notice the two key/value pairs that ensure that the date for the order will be today's date.
 - c. Click **+ Add New Value** to add a new key/value pair:

Element Name	Value
ContextId	[Paste the account id for Noah White]

PROPERTIES

PREVIEW

SetDate

Set Values

☒ Active
 [Edit as JSON](#)

Element Name ⓘ

SetDate

Field Label ⓘ

Set Date

Element Value Map

Element Name	Type	Value
TODAY	JSON Node	<code>=TODAY()</code>
Date	JSON Node	<code>=DATE(%TODAY%)</code>
ContextId	JSON Node	<code>0012w000002udGEAAY</code>
+ Add New Value		

☐ Window Post Message? ⓘ

Show Persistent Component ⓘ

☐ vlcCart

☐ Session Storage Message? ⓘ

Task 5: Test the CreateOrder remote action

With these elements in place, it's time to test that the CreateOrder remote action is working and setting the OrderId correctly.

1. At the top of the page, click **PREVIEW**. Once again, ignore the error message.
2. Click **{Data}** to view the data JSON.
3. In the data JSON, notice the ContextId has been set as Noah White's account id.

DATA

Clear Data

```
{
  "ContextId": "0012w000002udGEAAY",
  "timeStamp": "2019-12-
13T06:45:05.089Z",
  "userId": "0052w000000VbkKAAS",
  "userName":
"v106upgrade03@cme.edu",
  "userProfile": "System Administrator",
  "userTimeZone": "-480",
  "userCurrencyCode": "USD",
  "layout": "lightning",
  "id": "a292w0000008TNYAA2",
  "orderId": "a292w0000008TNYAA2"
}
```

4. Scroll to the bottom of the data JSON and notice the following:
- The response from the createCarts method (that created the order) in the records node
 - The OrderId has been copied from the Id in the records node

```
"GetCartItems": 473,  
  "InitializeSummaryData": 723  
},  
  "Date": "",  
  "TODAY": "2019-12-  
12T18:30:00.000Z",  
  "totalSize": 1,  
  "messages": [],  
  "records": [  
    {  
      "messages": [],  
      "displaySequence": -1,  
      "Id": "8012w000000TOrHAAW"  
    }  
  ],  
  "OrderId": "8012w000000TOrHAAW"  
}
```

Exercise 8-8: Get Data from the Shared Catalog

Scenario

In talking with the call center manager, Eliza learns she wants to focus on agents being able to quickly help customers choose smartphones and mobile plans. So, Eliza must build the guided selling experience around these products, and in order to do that, she needs to retrieve the product data from the shared catalog.

Goal

- Explain how to use the CPQ interface to get product data
- Understand the `getCartsProducts` method and how it ensures that the products it returns are valid
- Describe how to retrieve specific groups of products using `getCartsProducts` parameters

Tasks

1. Build portability into the script
2. Get product data using `getCartsProducts` remote actions
3. Test the `getCartsProducts` remote action
4. Refine the `getCartsProducts` response using a query
5. Test the query parameter

Time: 20 mins

Getting Data from the Shared Catalog

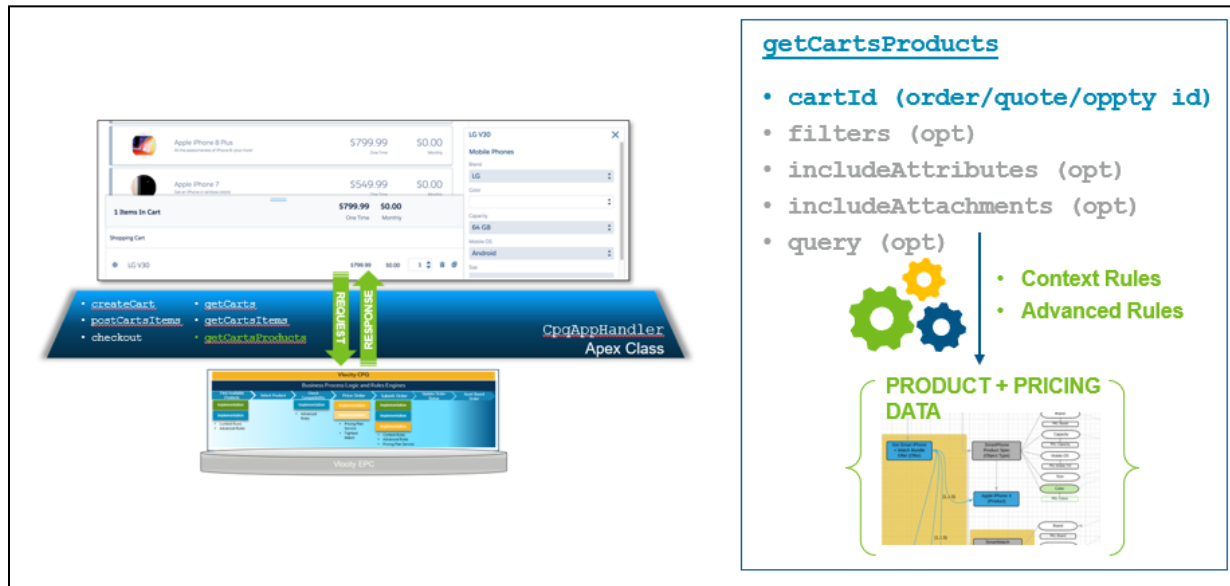


The shared catalog in EPC contains a wealth of CPQ data: products, bundles, attributes, pricing, rules and more. And while the combined technology of Communications Cloud and Salesforce includes many different ways to access data, the key is to ensure that only the right products are presented to the right customers at the right prices—which means that the path to success is using the CPQ application interface once again.

Getting Products using getCartsProducts Method

As you might expect, since `CpqAppHandler` is a CPQ application interface, it knows how to retrieve products—and then some! The method `getCartsProducts` requires a `cartId` (order, quote, or opportunity id) and then returns comprehensive product information, including attributes and pricing.

`getCartsProducts` only retrieves the products that the customer should see based upon the price list that is associated to the order, quote or opportunity that was passed as input and any active Context Rules and Advanced Rules that apply. Products, of course, must also be active and within their selling period dates in order to be retrieved.



Getting Specific Groups of Products

Now that you know how to retrieve product data, the next question is how to retrieve specific groups of products. Since this is a guided selling experience, the goal is to keep it simple and not overwhelm the user with too many choices. Each group of products should display as a selectable list on its own page. If you retrieved all of the products in the price list, you'd have quite a list to page through. In Eliza's scenario, she has been tasked with building a guided selling experience for mobile phones and plans. So, what's the best way to retrieve only mobile phones and plan product data from the shared catalog?

The answer is: it depends. It depends on how your product entities are structured and what grouping mechanisms you use in your catalog. Some deployments may use the Salesforce fields Type, SubType or Family. Others may use Salesforce Catalogs, Categories, Product Class, or Vlocity object types. So, in order to make the right choice, you must first understand how your shared catalog is working and what fields exist to identify the products as a group.

Once you've decided how to identify your product groups, the `getCartProducts` method provides several parameters you can use to filter its response.

filters

An optional parameter that lists a field and the values that must be matched exactly in order to be included in the response. Maps to the IN SOQL comparison operator.

- `filters=Product2.Name:Installation` returns items with product names equal to “Installation”

Multiple search values should be separated using an underscore, like this:

- `filters=Product2.Name:Installation_HomeHub` returns items with product names equal to “Installation” or “HomeHub”

Multiple filters should be separated using a comma, like this:

- `filters=Product2.Name:Installation,Product2.Product_Line__c:DSL`

This is translated to the following SOQL query:

- ```
(Product2.Name IN ('Installation')) AND
(Product2.Product_Line__c IN ('DSL'))
```

## query

An optional parameter that holds a text string. Any products that match (even partially) the text string in their product name or description will be included in the response. Maps to the LIKE SOQL comparison operator.

- `query=installation` returns products that contain the word “installation” in their Name OR Description

## searchBy

An optional parameter that modifies the query parameter to search specific fields others than Name or Description.

- `query=apple&searchBy=Product2.Brand__c` will search for products that have “apple” in the Product2.Brand\_\_c field

## category

An optional parameter that holds a product category id. Any products that are assigned to the category will be included in the response.

- `category=01s41000005dPYQAA2` returns products that have been assigned to this category (id)

These parameters can be combined together to further refine the response. For example, you could have a request that combines filters with a query and a searchBy modifier like this:

- `filters=Product2.Name:Installation&query=DSL&searchBy=Product2.Product_Line__c`

This would find all products with names = “Installation” and that contain the word “DSL” in the Product Line field. This is translated to the following SOQL query:

- `(Product2.Name IN ('installation')) AND  
(Product2.Product_Line__c LIKE '%DSL%')`

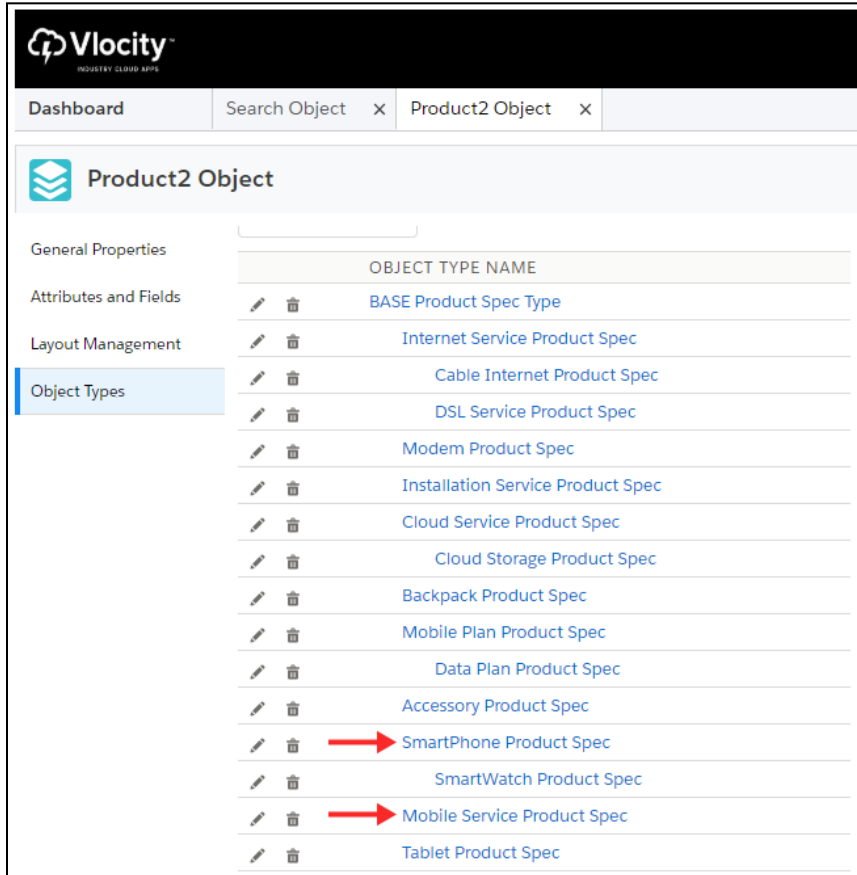


**NOTE:**

For more information on the `getCartProducts` method's parameters, see the documentation on the Cart-Based APIs in the Industries Configure, Price, Quote Web APIs on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

Getting back to Eliza and her task to retrieve mobile phones and plans, she could try the query parameter and search for “mobile phone” or “plan.” However, the results would include not only mobile phones but accessories and plans as well since they mention mobile phones in their descriptions.

However, Eliza is lucky in that products in her shared catalog have been created using Industries' object types, and they happen to have **SmartPhone Product Spec** and **Mobile Plan Product Spec** object types. Perfect! Eliza decides to use the object type field in a filters parameter to retrieve each group of products.



The screenshot shows the Vlocity 'Product2 Object' configuration page. On the left, a sidebar contains navigation links: 'General Properties', 'Attributes and Fields', 'Layout Management', and 'Object Types' (which is selected). The main area displays a table of object types. Two red arrows highlight the 'SmartPhone Product Spec' and 'Mobile Service Product Spec' entries.

| OBJECT TYPE NAME |                                   |
|------------------|-----------------------------------|
|                  | BASE Product Spec Type            |
|                  | Internet Service Product Spec     |
|                  | Cable Internet Product Spec       |
|                  | DSL Service Product Spec          |
|                  | Modem Product Spec                |
|                  | Installation Service Product Spec |
|                  | Cloud Service Product Spec        |
|                  | Cloud Storage Product Spec        |
|                  | Backpack Product Spec             |
|                  | Mobile Plan Product Spec          |
|                  | Data Plan Product Spec            |
|                  | Accessory Product Spec            |
|                  | SmartPhone Product Spec           |
|                  | SmartWatch Product Spec           |
|                  | Mobile Service Product Spec       |
|                  | Tablet Product Spec               |

STRUCTURE

- Script Configuration
- SetDate
- GetCPQObjectIds1
- CreateOrder
- SetOrderId
- GetCartItems
- GetMobilePhones**
- GetMobilePlans

### Retrieve specific products

Remote Class:

Remote Method:

filters

**Product2.vlocity\_cmt\_\_ObjectTypeId\_\_c: a1y1U00000036R5QAI**

Product2 Field API Name      Object Type Record Id

### Send retrieved products to selectable items element

SEND/RESPONSE TRANSFORMATIONS

Response JSON Node:

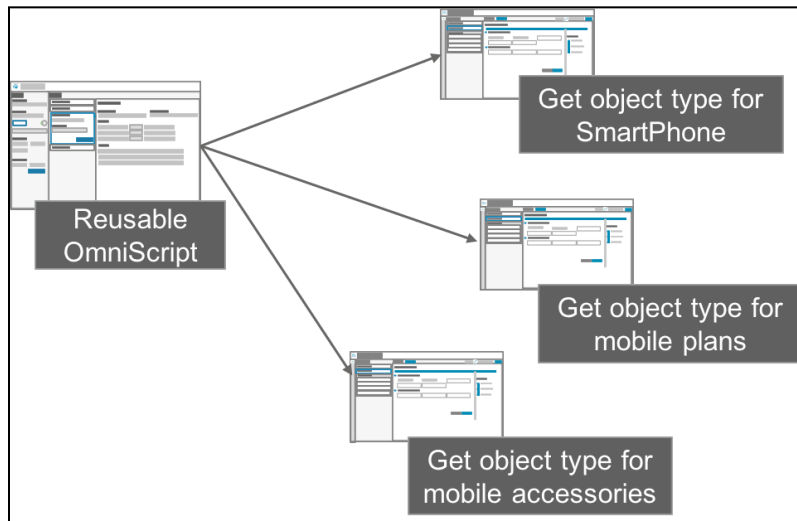
## Ensuring Script Portability

Eliza has just one more issue to solve. The filters parameter expects `[Field API name] : [value]`. So, when filtering by product object type, it expects:

```
filters=Product2.vlocity_cmt__ObjectTypeId__c: a1y1U00000036R5QAI
```

However, as all good OmniScript developers know, hard-coding Salesforce record ids into scripts is a risky business. Any time the script is ported to another environment, such as from sandbox to production, it will break because the Salesforce records ids are not the same between orgs.

*Eliza decides to create DataRaptors to retrieve the record ids for the object types she needs, and then use a reusable OmniScript as a container to make it easy to add these ids as variables to any OmniScript, including this one.*

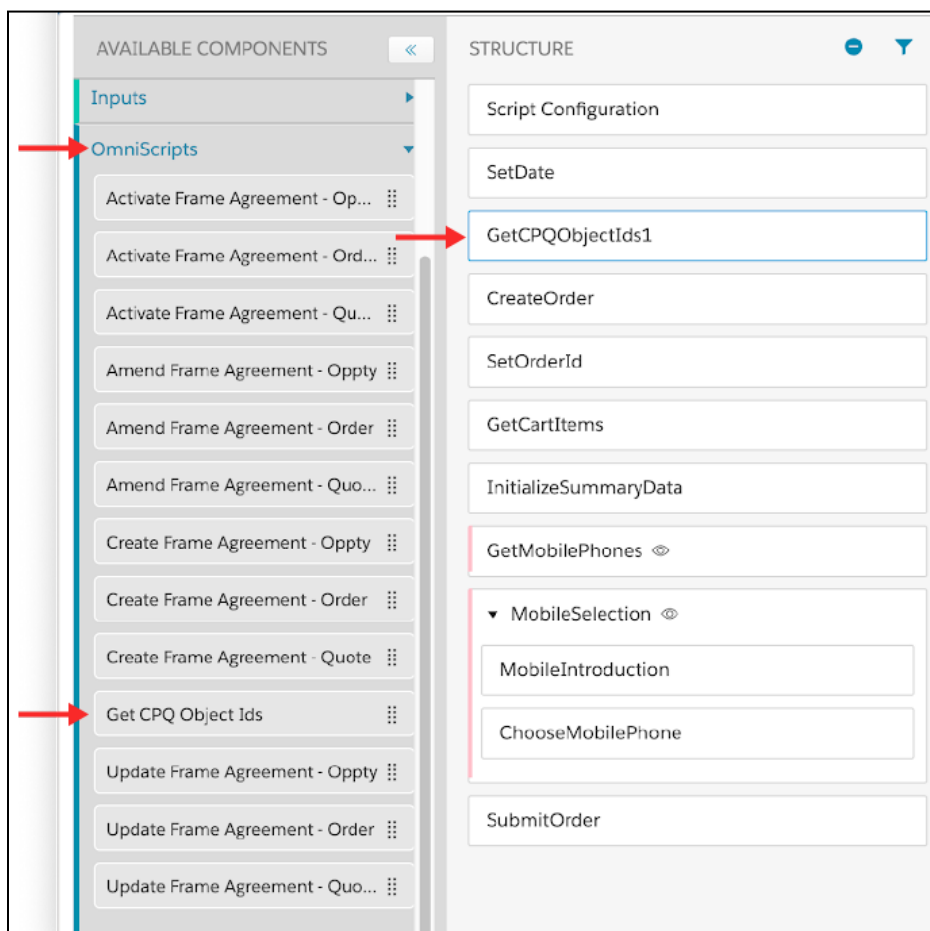


**NOTE:**

A reusable OmniScript called **Get CPQ Object Ids** is already created for you to use. It includes DataRaptors to get price list and object type ids. Feel free to check it out if you want to see how it works!

## Task 1: Build portability into the script

1. In the **AVAILABLE COMPONENTS** panel, expand the **OmniScripts** element group.
2. Drag the reusable OmniScript **Get CPQ Object Ids** directly after **SetDate**.



3. Test the reusable OmniScript.
  - a. At the top of the page, click **PREVIEW**.
  - b. Click **{Data}** to open the data JSON and then click **Clear Data**.
  - c. Click **Debug** to view the responses from the DataRaptor extract action in the reusable OmniScript.

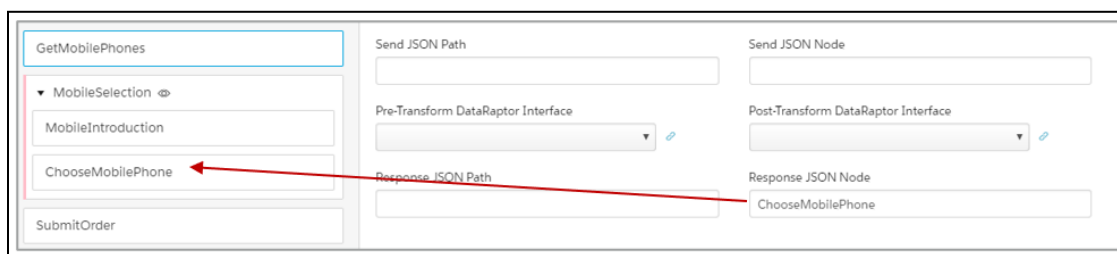
- d. In the **OMNISCRIPDT DEBUG CONSOLE**, expand **GetCPQObjectIdsDR** and its **response**.
- e. Notice the names of the new elements in the data JSON. These are the object types ids that you will use to retrieve products.
- f. Click **X** to dismiss the debug console.



*With these object type ids in hand, Eliza can get started configuring remote actions to retrieve the phone and plan data into her script.*

## Task 2: Get product data using getCartProducts remote actions

1. Configure the **GetMobilePhones** remote action to retrieve smartphone products from the shared catalog.
  - a. In the **STRUCTURE** panel, click **GetMobilePhones**.
  - b. In the **PROPERTIES** pane, notice the Remote Method is **getCartProducts**, which will retrieve product data using the CpqAppHandler application interface.
  - c. Notice also that the **Show Persistent Component vlcCart** property is checked.
  - d. Expand **SEND/RESPONSE TRANSFORMATIONS**. Notice the Response JSON Node is **ChooseMobilePhone** (which is used later as the name of the corresponding selectable items component).



- e. Within **REMOTE PROPERTIES**, under **Extra Payload**, notice the parameters that will be passed to the **getCartProducts** method. The **cartId** parameter is required, and this is already set up to pass the order id from the data JSON.

- f. Next to **filters**, notice the existing script used product family and subtype to retrieve mobile phones. Change this to use the smartphone object type id.

| Property | Value                                                           | Notes                                                                                     |
|----------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| filters  | Product2.vlocity_cmt__ObjectTypeId__c:%SmartPhonesObjectTypeId% | The %% syntax is a merge code to retrieve the SmartPhonesObjectTypeId from the data JSON. |



**ALERT:**

Make sure that there are no spaces in the filters value!

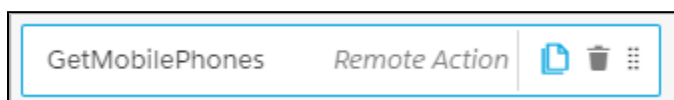
| Extra Payload     |                                                                 |
|-------------------|-----------------------------------------------------------------|
| Key               | Value                                                           |
| cartId            | %OrderId%                                                       |
| filters           | Product2.vlocity_cmt__ObjectTypeId__c:%SmartPhonesObjectTypeId% |
| includeAttachment | true                                                            |

- g. Under **Extra Payload**, notice the **includeAttachment** parameter. This is another optional parameter to the getCartProducts method that retrieves product images.
- h. Scroll down to **CONDITIONAL VIEW**. Notice the conditional that was set up to allow this step to appear only if the user chose to purchase a phone in the previous step. However, in your version of the script, everyone will be expected to browse mobile phones, so delete this conditional.

Click the **delete** icon.



2. Create a remote action to retrieve mobile plan products from the shared catalog. An easy way to do this is to clone the mobile phone remote action.
  - a. In the **STRUCTURE** panel, hover over **GetMobilePhones** and click the clone icon .



- b. In the **PROPERTIES** pane, change the properties of the cloned remote action as follows.

| Property           | Value                                                     |
|--------------------|-----------------------------------------------------------|
| Element Name       | GetMobilePlans                                            |
| Field Label        | Loading Mobile Plans                                      |
| filters            | Product2.vlocity_cmt__ObjectId__c : %MobilePlansObjectId% |
| Response JSON Node | ChooseMobilePlan                                          |

**PROPERTIES**   PREVIEW

**GetMobilePlans** Remote Action Active Edit as JSON

Element Name

Field Label

▼ REMOTE PROPERTIES

Remote Class  Remote Method

Remote Timeout (ms)

Pre-Transform DataRaptor Interface

Post-Transform DataRaptor Interface

Remote Options [Add New Key/Value Pair](#)

Extra Payload

| Key                                            | Value                                                                                     |
|------------------------------------------------|-------------------------------------------------------------------------------------------|
| <input type="text" value="cartId"/>            | <input type="text" value="%OrderId%"/>                                                    |
| <input type="text" value="filters"/>           | <input type="text" value="Product2.vlocity_cmt__ObjectTypeId__c:%MobilePlansObjectType"/> |
| <input type="text" value="includeAttachment"/> | <input type="text" value="true"/>                                                         |

[Add New Key/Value Pair](#)

► USER MESSAGES

► REDIRECT PROPERTIES

▼ SEND/RESPONSE TRANSFORMATIONS

Send JSON Path

Send JSON Node

Pre-Transform DataRaptor Interface

Post-Transform DataRaptor Interface

Response JSON Path

Response JSON Node

► ERROR MESSAGES

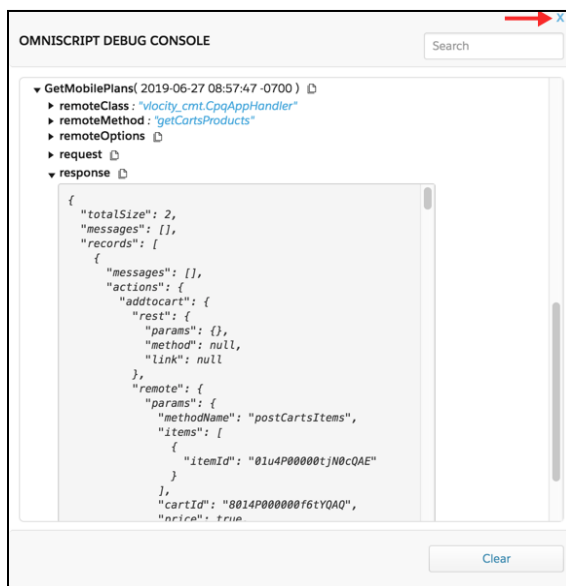
Validation Required

Show Persistent Component ☒ vlcCart

The next step is to test the newly configured remote actions.

## Task 3: Test the getCartProducts remote action

1. At the top of the page, click **PREVIEW**.
2. Ignore the error message, and click **Debug** to view the responses from the remote action calls.
3. In the **OMNISCRIP DEBUG CONSOLE**, expand **GetMobilePhones** and its **response**. Note the **totalSize** of the response, which indicates how many records are included in the JSON response.
4. Expand **GetMobilePlans** and its **response**.
5. Click **Clear**.
6. Click **X** to dismiss the debug console.



In addition to using the filter parameter to look for products with certain object types, you can combine parameters to further refine the response. Imagine that Eliza decides to offer only Apple phones during the guided selling experience. She can use the query parameter to do this.

## Task 4: Refine the getCartProducts response using a query

1. Add a query to retrieve *only* Apple smartphones.
  - a. Go to the OmniScript and open the **GetMobilePhones** remote action.
  - b. In the **PROPERTIES** pane, under **Extra Payload**, add this key/value pair:

| Key   | Value | Notes                                                             |
|-------|-------|-------------------------------------------------------------------|
| query | Apple | This will search for “Apple” in the product name and description. |

Extra Payload

| Key               | Value                                                       |
|-------------------|-------------------------------------------------------------|
| cartId            | %OrderId%                                                   |
| filters           | Product2.vlocity_cmt__ObjectTypeId__c:%SmartPhonesObjectTyp |
| includeAttachment | true                                                        |
| query             | Apple                                                       |

## Task 5: Test the query parameter

1. At the top of the page, click **PREVIEW**.
2. Click **{Data}** to open the data JSON and then click **Clear Data**.
3. Click **Debug** to view the responses from the remote action calls.
4. In the **OMNISCRIP DEBUG CONSOLE**, expand **GetMobilePhones** and its **response**.  
Note the **totalSize** of the response. The number of products included in the response should be fewer than before. Browse through the response to review all of the Apple mobile phones!
5. Click **X** to dismiss the debug console.
6. Click **PROPERTIES** to dismiss the preview.

---

## Exercise 8-9: Create Product Selection Steps

### Scenario

Now that Eliza has all this glorious product data flowing into her OmniScript, she needs to display it so that users can add the products to the persistent cart--and then watch the sales roll in! Eliza must customize the OmniScript to have a step for each of the product groups. Then agents can be guided to each product group to make selections.

### Goal

- Explain how to display and select products in guided selling
- Understand the purpose of `postCartsItems` method
- Describe how `postCartsItems` returns validated product and pricing data
- Explain how you can configure line items in the persistent cart

### Tasks

1. Configure product selection steps
2. Test the guided selling experience

Time: 20 mins

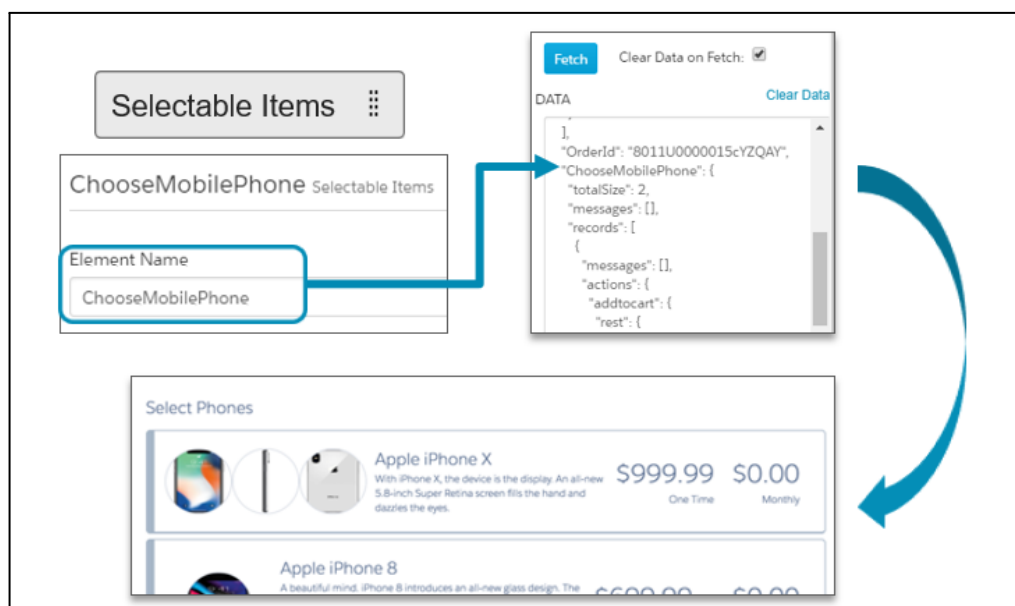
---

Once you've retrieved your products from the shared catalog, you need to display them to the user. In OmniScript, you do this using selectable items components.

## What is a selectable items component?

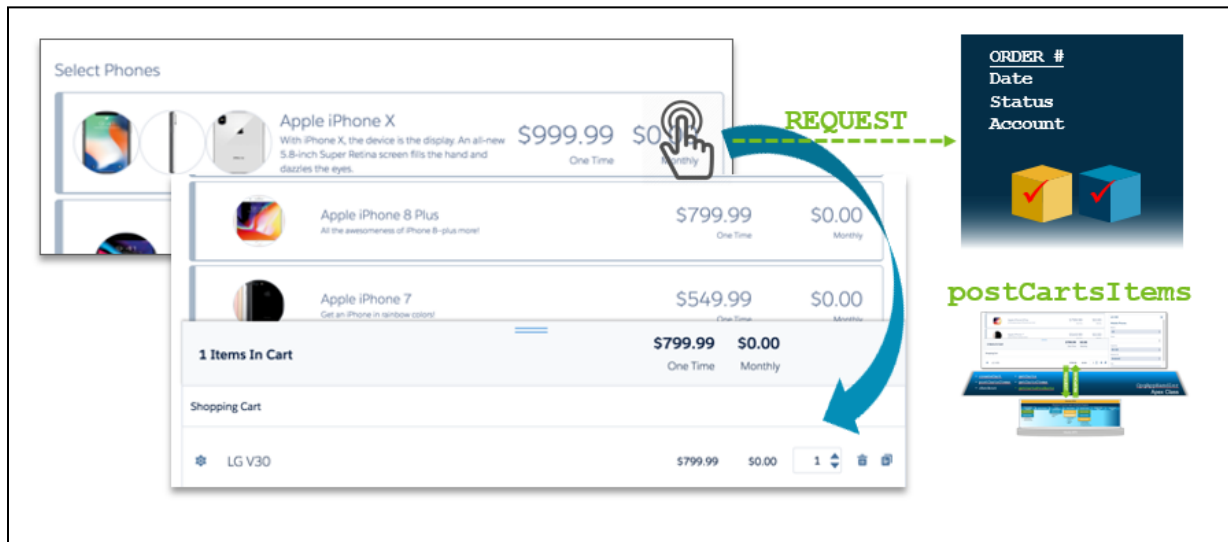
In OmniScript, selectable items components display a list to the user and allow them to select something from it. The component reads the script's data JSON looking for a JSON node that has its same name, and then it uses the data in that node to display a list of items to the user.

At runtime or during Preview, once the OmniScript engine finds the matched JSON node, it will pull the entire section of data out of the data JSON and use it to populate the list. This can be confusing sometimes when your entire product list seems to disappear from the data JSON!



## Adding Products with postCartsItems

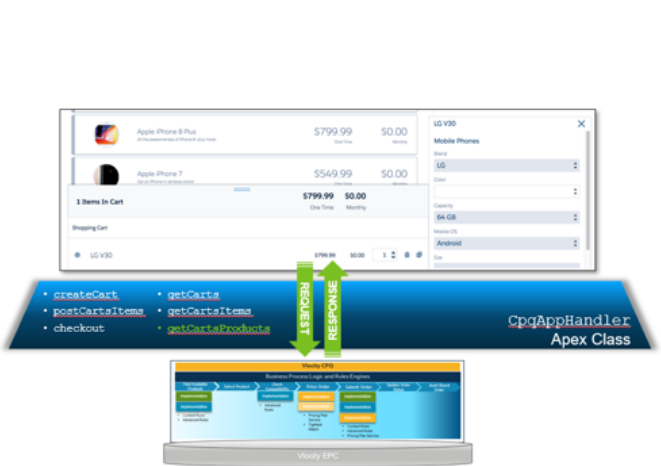
When the user clicks an item in the selectable items list, it adds it to the persistent cart, and then the user can configure it. In addition, behind the scenes, a line item will be added to the order using the postCartsItems method for the CPQ application interface.



`postCartsItems` method requires the `cartId` (order, quote and opportunity id) and the `itemId`, which is the pricebook entry id. If you're wondering where and how to get all of this information, don't worry—it's all there in the response that came back from `getCartsProducts`.

There are additional optional parameters that you can send. For more information on the `postCartsItems` method's optional parameters, see the documentation on the Cart-Based APIs in the Industries Configure, Price, Quote Web APIs on Salesforce Industries/Vlocity Success Community at <https://success.vlocity.com>.

In `postCartsItems` response, you will get validated product and pricing information. This means that the data has been evaluated for cardinality errors and by Advanced Rules for compatibility errors, which will display so the user can correct them, and the pricing service has run to determine the right price, usually using pricing plans and tightest match.



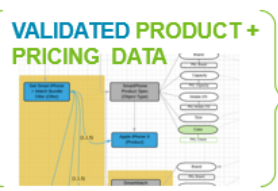
postCartItems

- `cartId` (order/quote/oppty id)
- `itemId` (price list id)
- `fieldsToUpdate` (opt)
- [...]



- Advanced Rules
- Pricing

**VALIDATED PRODUCT + PRICING DATA**



## Task 1: Configure product selection steps

1. Configure the phone selection step.
  - a. In the **STRUCTURE** panel, click the **MobileSelection** step element.
  - b. In the **PROPERTIES** pane, review the **Field Label** and **Instruction** properties. These will be seen by the user, so make any changes that you wish.
  - c. Scroll down to **CONDITIONAL VIEW**. Notice the conditional that was set up to allow this step to appear only if the user chose to purchase a phone in the previous step. However, in your version of the script, everyone will be expected to browse mobile phones, so just like before, delete this conditional.
    - i. Click the **delete** icon.

CONDITIONAL VIEW

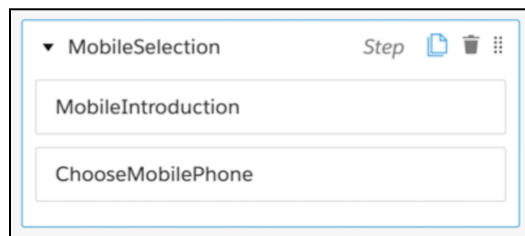
Hide Element If False

AND
ⓘ

PurchasePhone
Is Equal To
Yes
ⓘ

Add Condition ⓘ
Add Group ⓘ

- d. In the **STRUCTURE** panel, click the **MobileIntroduction** text block element.
  - e. In the **PROPERTIES** pane, review the **Field Label** and **Text** properties. These will also be seen by the user, so update as you see fit.
  - f. In the **STRUCTURE** panel, click the **ChooseMobilePhone** selectable items element.
  - g. Notice the **Element Name**. This name must match the node in the data JSON that holds the phone data that was retrieved using `getCartsProducts`.
  - h. Under Remote Properties, notice the **Remote Class** and **Remote Method**. `postCartsItems` is the method that will add the selected product to the order that underlies the cart as well as run the pricing service and advanced rules to alert the user to any configuration issues and to price it.
2. Create a mobile plans selection step. An easy way to do this is to clone the mobile phone selection step.
    - a. In the **STRUCTURE** panel, hover over **MobileSelection** and click the **clone** icon  .



- b. In the **PROPERTIES** pane, change the properties of the cloned step element as follows.

| Property         | Value                                                                    |
|------------------|--------------------------------------------------------------------------|
| Element Name     | PlanSelection                                                            |
| Field Label      | Mobile Plan Selection                                                    |
| Step Instruction | Try to upsell the customer to family plans bundles by telling them about |

---

the lower costs per person and cool extras.

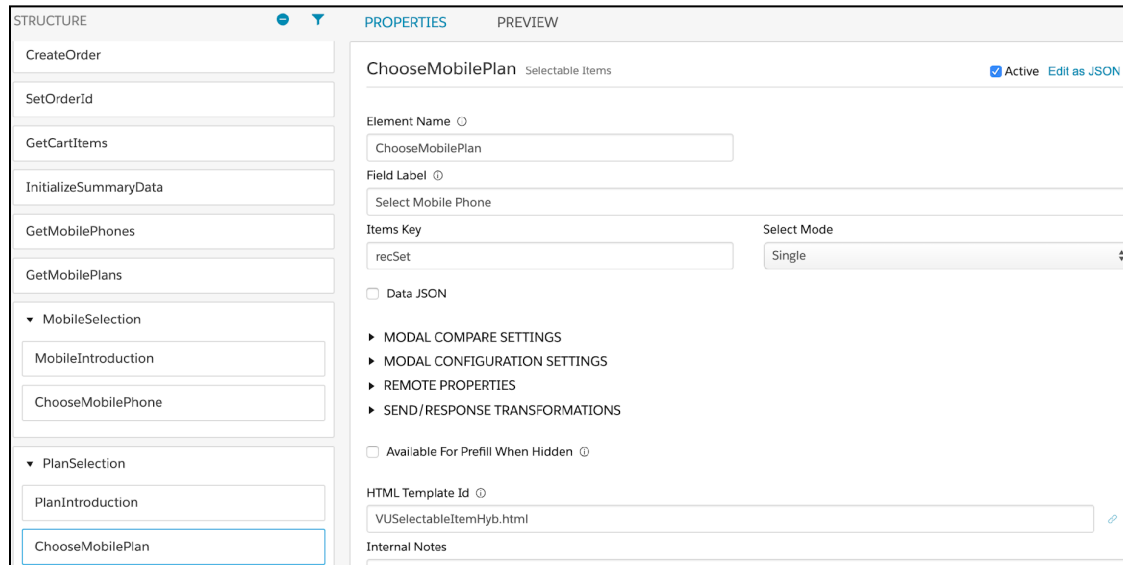
---

3. In the **STRUCTURE** panel, click the cloned text block element, and change its properties as follows.

| Property     | Value                                            |
|--------------|--------------------------------------------------|
| Element Name | PlanIntroduction                                 |
| Field Label  | Choose a plan...                                 |
| Text         | More is, in fact, better when it comes to plans. |

4. In the **STRUCTURE** panel, click the cloned selectable items element, and change its properties as follows

| Property     | Value            | Notes                                                                                                                                      |
|--------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Element Name | ChooseMobilePlan | This element name must match the JSON node that holds the plan product data. Ensure that this has no spaces and is typed exactly as shown. |



The screenshot displays the Salesforce Guided Selling configuration interface. On the left, the 'STRUCTURE' pane shows a tree view with the following components: 'CreateOrder', 'SetOrderId', 'GetCartItems', 'InitializeSummaryData', 'GetMobilePhones', 'GetMobilePlans', 'MobileSelection' (expanded), 'MobileIntroduction', 'ChooseMobilePhone', 'PlanSelection' (expanded), 'PlanIntroduction', and 'ChooseMobilePlan' (selected). The right pane, titled 'PROPERTIES', shows the configuration for the 'ChooseMobilePlan' component, which is a 'Selectable Items' type. It includes the following settings: 'Element Name' (ChooseMobilePlan), 'Field Label' (Select Mobile Phone), 'Items Key' (recSet), 'Select Mode' (Single), 'Data JSON' (unchecked), 'MODAL COMPARE SETTINGS', 'MODAL CONFIGURATION SETTINGS', 'REMOTE PROPERTIES', 'SEND/RESPONSE TRANSFORMATIONS', 'Available For Prefill When Hidden' (unchecked), 'HTML Template Id' (VUSelectableItemHyb.html), and 'Internal Notes'.

## Task 2: Test the guided selling experience

1. Click **PREVIEW** to test the guided selling experience.

**Mobile Selection**

We've got all the latest and greatest.

Select Mobile Phone

|                                                                                    |                                                                                                                                                                                                                                                                                                                                               |                                                    |              |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|--------------|
|   | <b>Apple iPhone X</b><br>With iPhone X, the device is the display. An all-new 5.8-inch Super Retina screen fits the hand and dazzles the eyes.                                                                                                                                                                                                | <b>\$999.99</b><br>One Time                        | 0<br>Monthly |
|   | <b>Apple iPhone 8</b><br>A beautiful mind. iPhone 8 introduces an all-new glass design. The world's most popular camera, now even better. The most powerful and smartest chip ever in a smartphone. Wireless charging that's truly effortless. And augmented reality experiences never before possible. iPhone 8. A new generation of iPhone. | <b>\$699.99</b><br>One Time                        | 0<br>Monthly |
|   | <b>Apple iPhone 8 Plus</b><br>All the awesomeness of iPhone 8—plus more!                                                                                                                                                                                                                                                                      | <b>\$699.99</b><br>One Time                        | 0<br>Monthly |
|   | <b>Apple iPhone 7</b><br>Get an iPhone in rainbow colors!                                                                                                                                                                                                                                                                                     | <b>\$549.99</b><br>One Time                        | 0<br>Monthly |
|   | <b>Apple iPhone Xs</b><br>A Super Retina OLED display. Even faster Face ID. And a breakthrough dual-camera system. The iPhone Xs is everything you love about iPhone. Taken to the extreme.                                                                                                                                                   | <b>Starting at \$49.91/mo or \$999</b><br>One Time | 0<br>Monthly |
|  | <b>Apple iPhone 11</b><br>Just the right amount of everything.                                                                                                                                                                                                                                                                                | <b>\$699.99</b><br>One Time                        | 0<br>Monthly |

Cancel Next

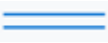
By selecting a mobile plan the customer will be able to take advantage of landline calls being forwarded to their mobile.

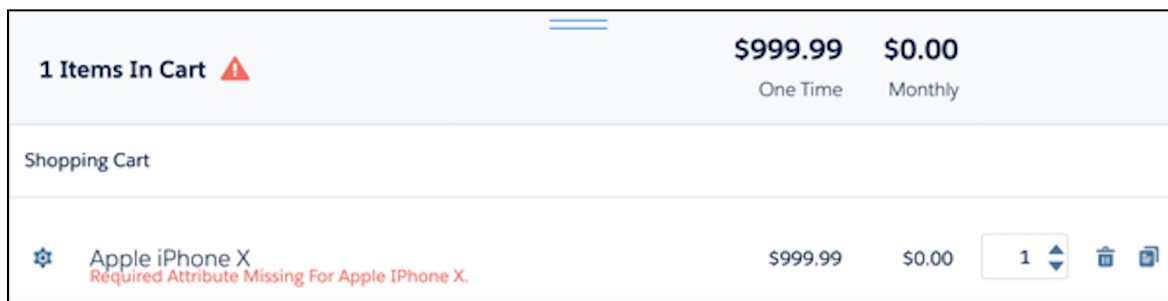
**Steps**

- MOBILE SELECTION
- MOBILE PLAN SELECTION

If you do not see something similar to the screenshot above, something in your configuration is not quite right. Don't worry, it's usually easy to fix. The things to check are:

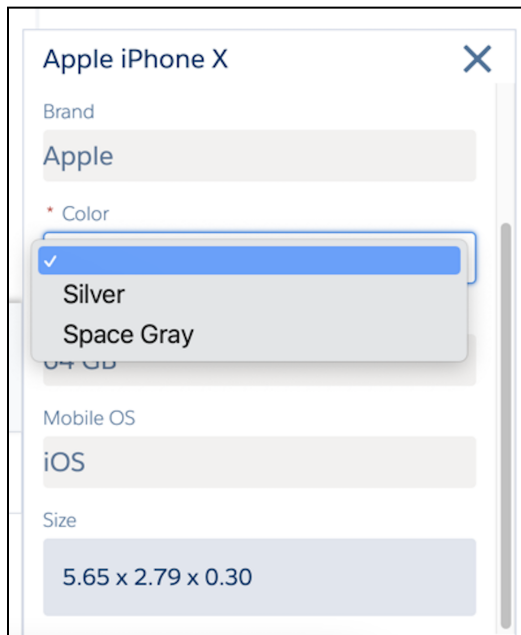
- Click **Debug**, and make sure the phone and plan data is returning from your remote action elements. If not, check your remote actions. If you have data in the response, then...
- Check the **Response JSON Nodes** under SEND/RESPONSE TRANSFORMATIONS in your remote action elements. They should be **ChooseMobilePhone** and **ChooseMobilePlan**, respectively. If not, fix them. If they are correct, then...
- Check the selectable items elements. Their element names match the Response JSON Nodes above. If not, fix them.

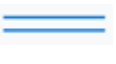
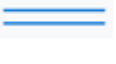
2. Click **Apple iPhone X** to add it to the cart.
3. At the bottom of the page, in the **Total Summary** of the persistent cart, notice the items in cart and the one-time total are no longer zero.
4. Notice there is an  icon in the **Total Summary**. This indicates that the product you selected requires configuration.
5. Click the **double bars**  in the **Total Summary** to expand the persistent cart.
6. Notice that you can increase the quantity, clone the item or delete it. Also notice the message in red. The persistent cart is enforcing required attributes, ensuring that the product is correctly configured before it is ordered.



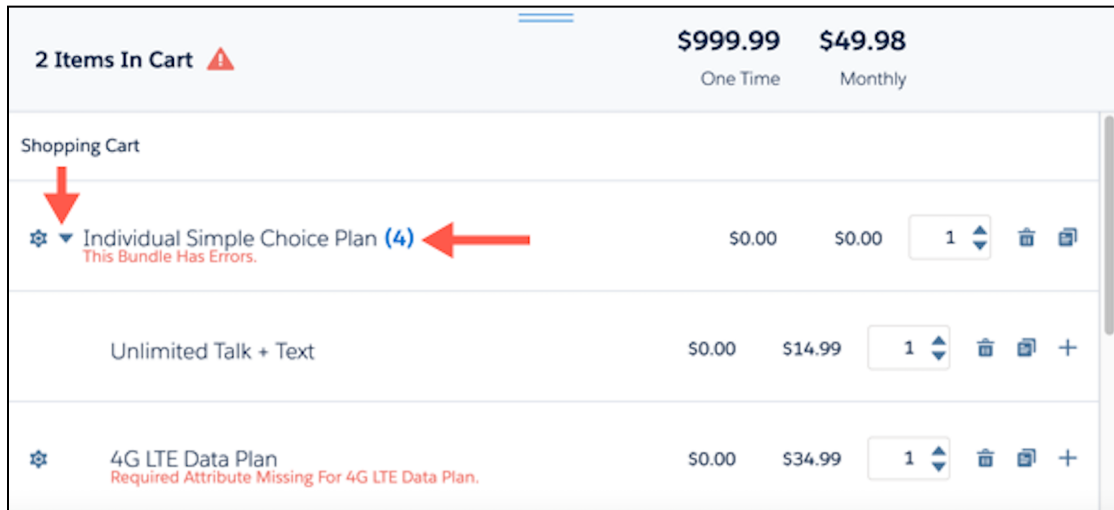
7. Click the **configuration** icon  next to the line item in the cart.

8. In the **configuration** window, click the **Color** field and select a color.

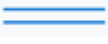


9. Notice the  icon disappears. Click **X** to close the configuration window.
10. Click the **double bars**  to collapse the persistent cart.
11. Click **Next**.
12. Click a plan to add it to the cart. Notice the total monthly price updates.
13. Click the **double bars**  to expand the persistent cart.
14. Notice the number in blue next to the plan line item. This indicates the number of products in the bundle.

15. Click the **expand icon** to view the children in the bundle.



16. Configure the missing attributes. Notice that once the products are configured correctly, the persistent cart removes the red error messages.

17. Click the **double bars**  to collapse the persistent cart.

18. Click **Next**. Behind the scenes, the order was submitted.

19. Click **PROPERTIES** to exit the preview.

Now that Eliza has the product selection steps flowing nicely, it's time to close the deal—in other words, get the submit order step configured and set the script launch point.

---

## Exercise 8-10: Configure the Selling End and Launch Points of the Script

### Scenario

The remaining items are to review the order checkout element in the script, and then set the end and launch points for the script. The call center manager wants guided selling to launch from the account page. So, Eliza must create an OmniStudio Action button that will appear on the Account page to launch the OmniScript and then she can create a Done action to return the user to the account page after the order is submitted.

### Goal

- Explain the function of the checkout method
- Describe what happens during and after checkout
- Explain the components and process of launching guided selling and returning to the account

### Tasks

1. Configure order checkout
2. Configure the end of script navigation
3. Test the guided selling OmniScript

Time: 15 mins

---

---

## Submitting Orders using the checkout Method

Once you've retrieved products, displayed them to the user, added them to the cart, there's one final step you don't want to forget—submitting the order! The CPQ interface, once again, comes into play and provides the checkout method to ensure every order is valid and can be fulfilled.

The checkout method requires the cartId (order, quote or opportunity id). Based on the header object, checkout moves the object to the next state of the ordering process.

- If the header object is Opportunity, submits the opportunity to create a quote.
- If the header object is Quote, submits the quote to create an order.
- If the header object is Order, submits the order to create assets.

The Checkout Items in Cart remote action requires the cartId and ContextId in the Extra Payload section. The ContextId and cartId should both have the same value, that value being the cartId. If you do not set the ContextId value at this point in the OmniScript, it will instead use the previous value it was set to, which is the Account ID. In the OmniScript, the cartId was set to %OrderId% in the JSON. Therefore, ContextId and cartId are both set to the value of %OrderId%.

**SubmitOrder** Remote Action Active Edit as JSON

Element Name ⓘ

Field Label ⓘ

▼ REMOTE PROPERTIES

Remote Class

Remote Method

Remote Timeout (ms)

Pre-Transform DataRaptor Interface 🔗

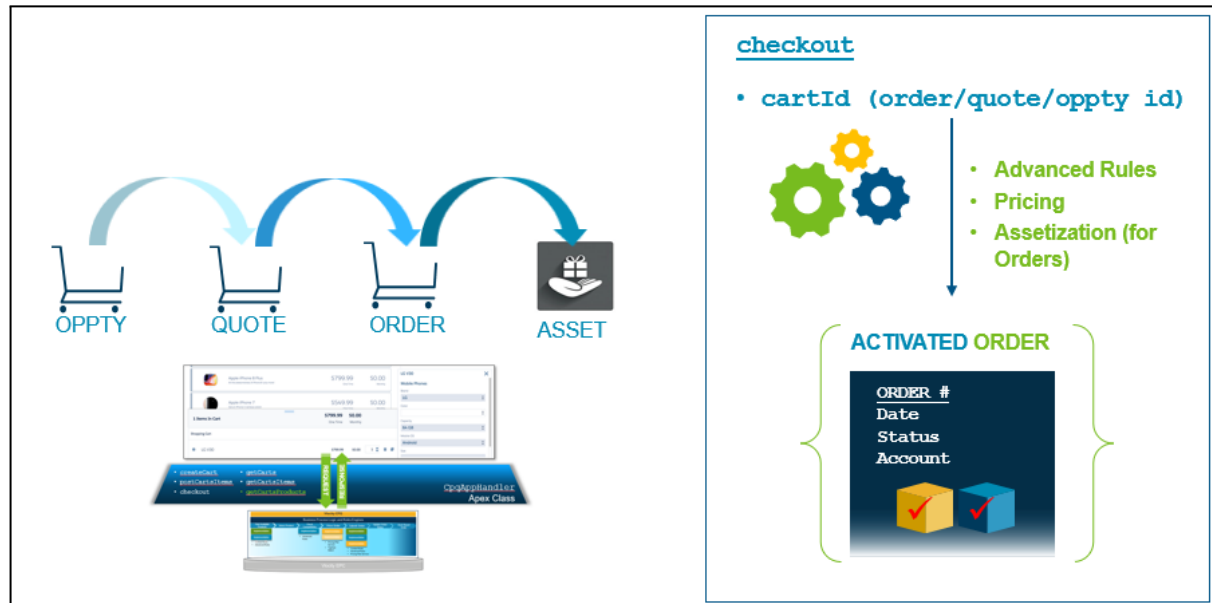
Post-Transform DataRaptor Interface 🔗

Remote Options  
[Add New Key/Value Pair](#)

Extra Payload

| Key                                    | Value                                  |                 |
|----------------------------------------|----------------------------------------|-----------------|
| <input type="text" value="ContextId"/> | <input type="text" value="%OrderId%"/> | <span>🗑️</span> |
| <input type="text" value="cartId"/>    | <input type="text" value="%OrderId%"/> | <span>🗑️</span> |

When you checkout an order, rules and pricing will run one final time, and then the line items will be assetized, creating assets. The Salesforce order status will change from Draft to Activated.



## Task 1: Configure order checkout

1. In the **STRUCTURE** panel, click **SubmitOrder**.
2. In the **PROPERTIES** pane, review the **Remote Method** for submitting an order.

## Task 2: Configure the end of script navigation

Now you need to create a way for the user to return to the account page after going through the guided interaction. We have already placed the **Done Action NavigatetoAccount** in the OmniScript for you.

1. Click on **NavigatetoAccount**.
2. Notice the **Source** property is set by default to **%ContextId%**. Since the script's context id is an account, this will navigate the user to the account id that the script is launched with.
3. Activate the OmniScript.
  - a. In the **STRUCTURE** panel, click **Script Configuration**.
  - b. At the bottom of the **PROPERTIES** pane, click **Activate Version**.

With the script completed, Eliza just needs to create a way to launch it. She'll use an OmniStudio Action, so that it can be launched using a button from the Account page. For the purposes of this exercise, we have already created it in the training playground for you.

## Task 3: Test the guided selling OmniScript

1. Open the account page for **Noah White**.
2. Click on the **Details** tab.
3. In the Power Launcher's **What can I help you with?** box, type `Buy Phone`.
4. Click **Buy Phone+Plan**. This is an OmniStudio Action we preconfigured for the guided selling OmniScript.
5. The guided selling OmniScript displays.
6. Select one or more phones to add to the cart.

Make sure to configure any line items when the  icon appears in the persistent cart, just as you did earlier.

7. Scroll down and click **Next** to go to the next step.
8. Add a plan to the cart and configure its required attributes.
9. Click **Next**. Notice how the script returns you to the account page.
10. When you are returned to the account page for **Noah White**, scroll to the **Service Asset Management** section.

11. Notice the items you purchased with guided selling now show as assets.

| Related Details News                                   |          |                  |                 |                 |                |                   |
|--------------------------------------------------------|----------|------------------|-----------------|-----------------|----------------|-------------------|
| Service Asset Management                               |          |                  |                 |                 |                |                   |
| ASSETS                                                 |          |                  |                 |                 |                |                   |
|                                                        | Quantity | Recurring Charge | Recurring Total | One Time Charge | One Time Total | Promotions        |
| <input type="checkbox"/> Individual Simple Choice Plan | 1        | \$0.00           | \$49.98         | \$0.00          | \$0.00         | <a href="#">i</a> |
| Unlimit...                                             | 1        | \$14.99          | \$14.99         | \$0.00          | \$0.00         | <a href="#">i</a> |
| 4G LTE ...                                             | 1        | \$34.99          | \$34.99         | \$0.00          | \$0.00         | <a href="#">i</a> |
| Mobile ...                                             | 1        | \$0.00           | \$0.00          | \$0.00          | \$0.00         | <a href="#">i</a> |
| Netflix                                                | 1        | \$0.00           | \$0.00          | \$0.00          | \$0.00         | <a href="#">i</a> |

---

## Exercise 8-11: Configure the Persistent Cart

### Scenario

Eliza is in the home stretch! Now that Eliza has configured the guided selling experience, she can now spend some time learning about the persistent cart. With this knowledge in hand, Eliza can propose alternative approaches to her manager in the future.

### Goal

- Describe what the persistent cart is
- Explain how the persistent cart is different from the Cart
- Find the templates used by the persistent cart
- Change the label of the persistent cart
- Explain why the default OmniScript element type must be overridden
- Understand what data JSON nodes must exist for the persistent cart to function

### Tasks

1. Configure the persistent cart
2. Test the persistent cart
3. Prepare the persistent cart's data JSON

**Time:** Optional 15 mins

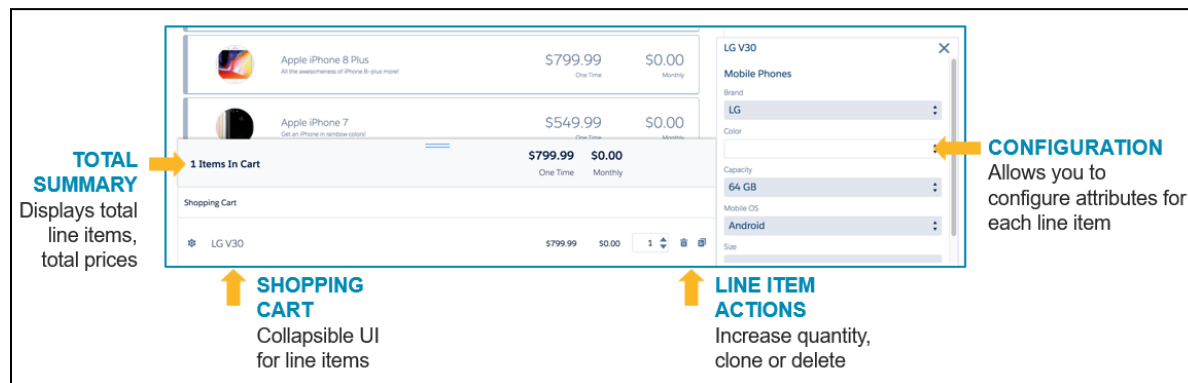
---

## What is the persistent cart?

During order capture, the Cart allows you to browse eligible and available products in the shared catalog, add and configure products in the cart, and ensure your customer gets the right price for each product. The Cart is used by internal employees, like customer service representatives or sales agents.

When you want to create a guided selling process, you need a shopping cart to serve the same purpose as the Cart does during order capture, but it must have a more focused user experience. The persistent cart provides this capability.

The persistent cart is an OmniScript component that allows users to configure and eventually buy products during a guided interaction. The persistent cart provides a more lightweight user interface, while still using the same pricing, rules and products defined in the shared catalog.



## Persistent Cart Features

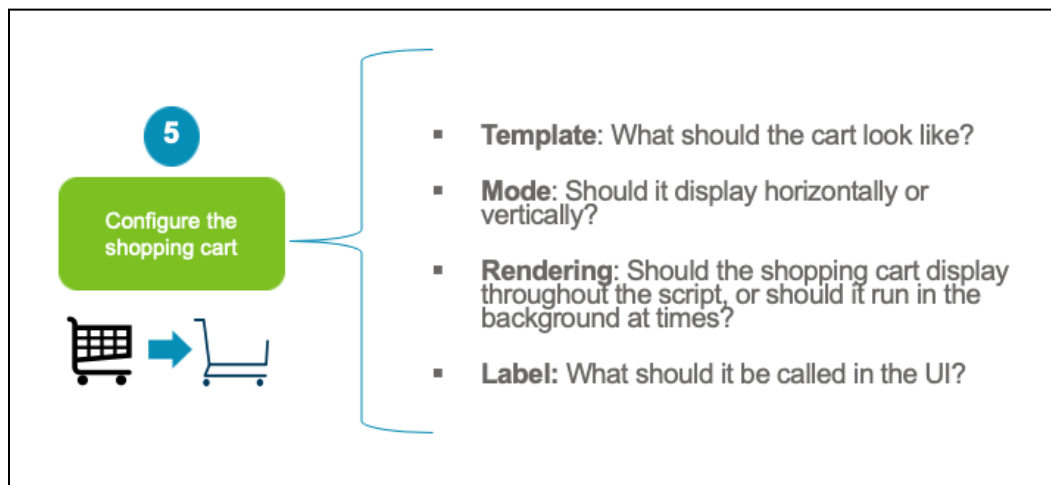
The persistent cart provides the following features:

- **Total Summary**, which displays the number of items in the cart and one-time and recurring price totals
- **Shopping Cart**, a collapsible user interface that displays individual line items with quantity, one-time and recurring prices

- **Line Item Actions**, including ability to clone or delete line item depending on product cardinality
- **Line Item Configuration**, which allows you to configure product attributes

## Configuring the Persistent Cart

To create a guided selling experience, you must configure the persistent cart, specifying each of the following.



## Task 1: Configure the persistent cart

1. Review the persistent cart template.
  - a. Navigate back to the **Customer Mobile Order** Omniscript.
  - b. In the **STRUCTURE** panel, click **Script Configuration**.
  - c. In the **PROPERTIES** panel, under **Element Type To HTML Template Mapping**, notice that the view mode is set to **Horizontal Mode Persistent Component**, and the template is **vlcHCartHyb.html**.

The Horizontal Mode Persistent Component element type, as its name implies, is a persistent component that displays horizontally in the OmniScript.

Persistent components can be used for many things in OmniScript, such as displaying helpful knowledge articles for a troubleshooting script. In order to use the persistent component as a shopping cart, you must override the standard element type's template to use the specialized shopping cart template. You do that by specifying an element type and a template name in the **Element Type To HTML Template Mapping**.

When it comes to the template, **vlcHCartHyb.html** is the legacy default template for the horizontal-mode shopping cart. You should update this to the latest version, which you can find in Vlocity Templates.

Script Configuration

Edit as JSON

OmniScript name ⓘ

Customer Mobile Order

Description

B2C Bundle Order

Type ⓘ

Guided Selling

SubType ⓘ

B2C Bundle Order

Language ⓘ

English

Currency Code ⓘ

☐ Enable LWC ⓘ

☐ Reusable ⓘ

☐ Label outside of field ⓘ

☒ Enable Tracking ⓘ

☐ Fetch Picklist Values At Script Load

☐ Enable Unload Warning ⓘ

Console Tab Title

Console Tab Icon

custom:custom18

Element Type To HTML Template Mapping ⓘ

Element Type

Template Name

Horizontal Mode Persistent Component

vlcHCartHyb.html

- Click the **Lightning App Launcher**  in the upper left-hand corner of the screen.
- In the **Search apps and items...** box, type `Vlocity Templates`.
- Right-click **Vlocity Templates** and select **Open link in new tab**.
- In **Vlocity Templates** in the **Find in page** search dialog at the top of the page, enter `cart`.
- Review the list of cart templates. Notice the `cpq-cart` templates, which are used for the Cart. Find **omniLightning-vlcHCartHyb** in the list and expand it.
- Click the **Version 1** template and review the code.
- Notice at line 34 and 45 that this template calls a second template, **vlcTopCartHierarchyHyb.html**. If you plan to customize the format or appearance of the shopping cart, you may need to modify one or all of the following templates, depending on the types of changes desired.
  - omniLightning-vlcHCartHyb.html
  - omniLightning-vlcTopCartHierarchyHyb.html
  - omniLightning-vlcTopCartChildrensHyb.html

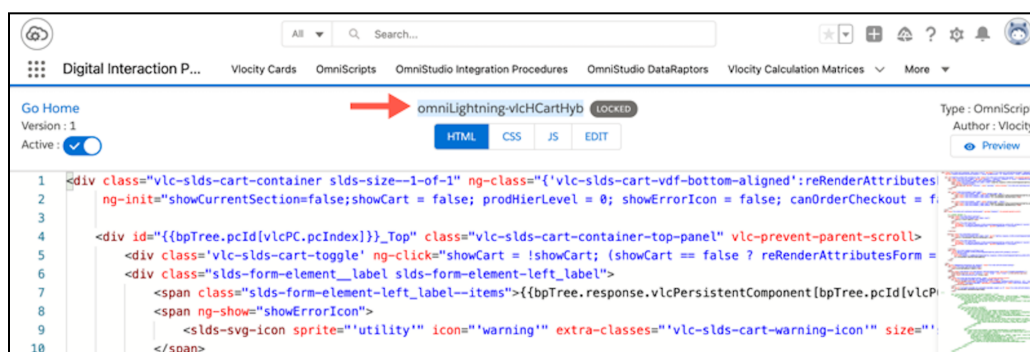


**NOTE:**

omniLightning is the prefix that Salesforce Industries has added to the latest versions of the templates included in the managed package template library.

In this exercise, no need to configure anything! You will use the default templates with no modifications.

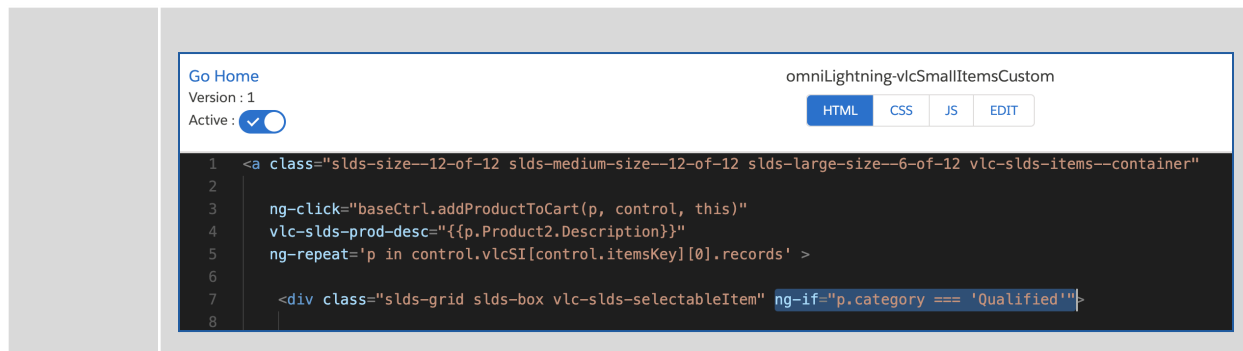
- j. At the top of the **Template Designer**, highlight the name of this template and copy it to your buffer.



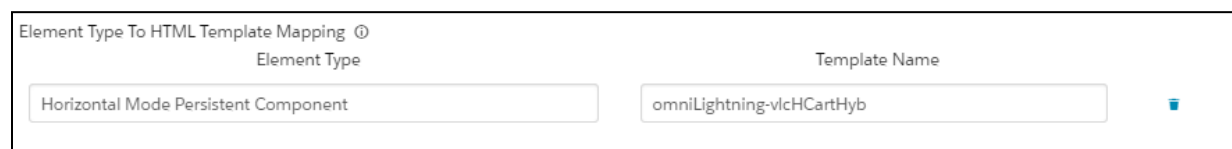
**NOTE:**

If you are using Context Rules you will need to do a few extra things to get it to work in the persistent cart.

1. You'll need to pass the includeIneligible = true parameter to getCartProducts. This parameter will return both qualified and disqualified products from the Context Rules engine, and importantly, it will insert a category node in the response.
2. Add a modification to the selectable items template. By adding the ng-if shown below, the JavaScript in the template will filter the JSON to only display products listed under the Qualified category node. You'll learn more about the selectable items template later in the course.



- k. Click **X** to close the current browser tab and return to the Customer Mobile Order OmniScript in **OmniScript Designer**. You should still have the **Script Configuration** element selected in the **STRUCTURE** panel.
- l. Update the value in the Template Name field:
  - i. Click the **Deactivate Version** button.
  - ii. In the **PROPERTIES** pane under **Element Type To HTML Template Mapping**, replace the **Template Name** by pasting in the contents of your buffer.



**NOTE:**

When you refer to a template in the Template Designer, do not refer to it as an .html file. Instead, you must refer to its template name.

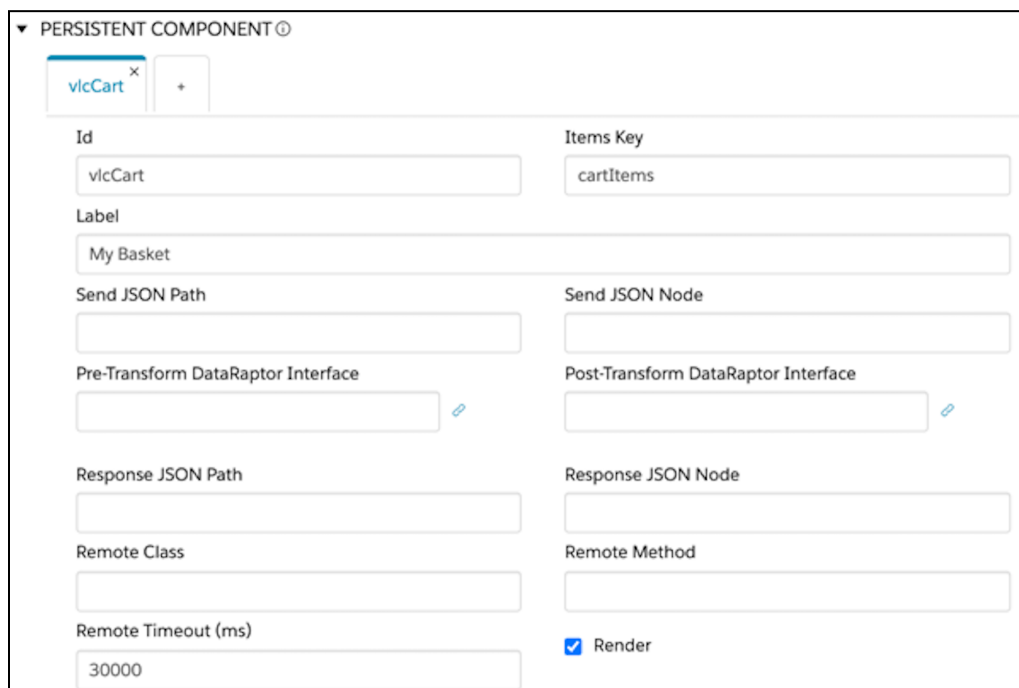
2. Configure the persistent component.
  - a. In the **PROPERTIES** pane, scroll down and expand **PERSISTENT COMPONENT**.
  - b. Note the tab labeled **vlcCart**. This is the persistent cart, also known as **vlcCart**.



**NOTE:**

The persistent cart's Id must always remain `vlcCart`. This is because the Id must match the parent node in the data JSON and also because the default templates expect it.

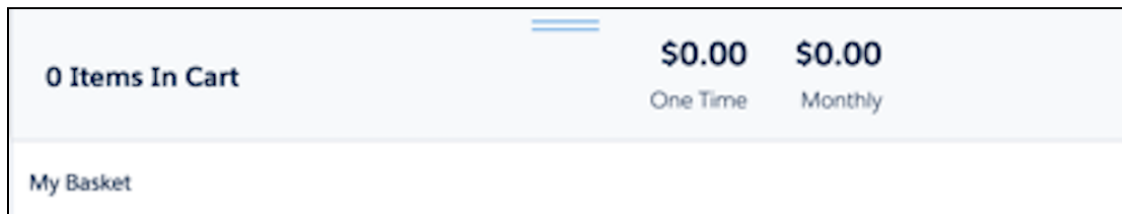
- c. Under **Label**, enter `My Basket`. (Or whatever you'd like to call your shopping cart.)
- d. Delete the values in **Remote Class** and **Remote Method**. Any classes and methods named here are invoked when the script launches, but in this example, they are not needed since there will not be any items in the cart when it launches.
- e. Notice the **Render** property. This flag ensures that the cart will display when the cart launches. If desired, you could uncheck the Render property here but enable it later on, if you wanted the shopping cart to only appear in a later step.
- f. Click the **Activate Version** button.



The screenshot shows the 'PERSISTENT COMPONENT' configuration window for a component named 'vlcCart'. The interface is divided into two columns. The left column contains fields for 'Id' (set to 'vlcCart'), 'Label' (set to 'My Basket'), 'Send JSON Path', 'Pre-Transform DataRaptor Interface', 'Response JSON Path', 'Remote Class', and 'Remote Timeout (ms)' (set to '30000'). The right column contains fields for 'Items Key' (set to 'cartItems'), 'Send JSON Node', 'Post-Transform DataRaptor Interface', 'Response JSON Node', 'Remote Method', and a 'Render' checkbox which is checked.

## Task 2: Test the persistent cart

1. At the top of the pane, click **PREVIEW**.
2. At the bottom of the page, click the two blue lines to expand the cart.
3. Review the persistent cart. Notice the name you gave your shopping cart.



4. Notice that the **Total Summary** displays the number of items in the cart. Since nothing is in the cart at this point, it displays **0 Items in Cart**. However, on the other side, the one time and monthly totals are blank.

You'll look at the reasons why this is next.

5. Click **PROPERTIES** to exit the preview.

## What are vlcCart and vlcCart\_Top and why are they important?

Just like all OmniScripts, a guided selling OmniScript depends on a good, well-formatted data JSON. And when you use the persistent cart, there are two JSON nodes that are important for the cart to operate correctly.

| JSON object | Purpose                                                                                                                                                                                                                                                                                                           |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vlcCart     | Contains the bulk of the data used by the persistent cart and its operations, including the total number of items that have been added to the cart.<br><b>Must exist in the parent node, vlcPersistentComponent, of the data JSON in order for the cart to function and for products to be added to the cart.</b> |
| vlcCart_Top | Contains the one time and monthly pricing totals that appear in the Total Summary.<br><b>If it does not exist in the data JSON, the one time and monthly pricing totals will be blank when the script launches. However, it does not affect other functionality.</b>                                              |



**NOTE:**

JSON, or JavaScript Object Notation, is a minimal, readable format for structuring data. It is used primarily to transmit data between a server and web application.

A JSON object is represented by curly brackets in JSON notation and contains a collection of key/value pairs and/or arrays. For example, the JSON object below is named by its parent node, “vlcCart.”

```

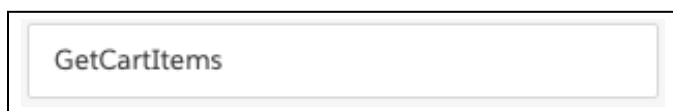
"vlcCart" : {
 "total size" : 0,
 [...]
}
```

To learn more about OmniScript JSON, see the [Create a Simple OmniScript](#) unit in the Build Guided Experiences with OmniStudio trail on Trailhead.

As you will see, this script has a remote action that creates the vlcCart JSON node, which is the essential node that must exist. However, the script’s designers did not take the time to create the second node, vlcCart\_Top, and that’s why the pricing totals are blank. Let’s fix it.

## Task 3: Prepare the persistent cart's data JSON

1. Click the **Deactivate Version** button.
2. In the **STRUCTURE** panel, hover over **GetCartItems** and click the **clone icon** .



3. In the **PROPERTIES** pane, change the properties of the cloned remote action as follows.

| Property                                                                       | Value                 | Notes                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Element Name                                                                   | InitializeSummaryData |                                                                                                                                                                                                                                                                   |
| Remote Method                                                                  | getCarts              | This is a method of the CPQ interface, CpqAppHandler. It returns the pricing, account and other information related to the order, quote or opportunity. For more information, see the developer documentation on Salesforce Industries/Vlocity Success Community. |
| Response JSON Node<br>(To find it, expand<br>SEND/RESPONSE<br>TRANSFORMATIONS) | vlcCart_Top           |                                                                                                                                                                                                                                                                   |

4. Notice under **Show Persistent Component** that **vlcCart** is checked. Even though this step has no user interaction, this must be checked in order for the **Total Summary** to find the information it needs in the data JSON.



**NOTE:**

Checking the Show Persistent Cart property does two things: it displays the persistent cart, and it changes where any response data from the element is written. For example, as you can see below, if the vlcCart is not checked, vlcCart\_Top is written as another node in the root level JSON. However, when it is checked, vlcCart\_Top is nested within the vlcPersistentComponent node, which is where the Total Summary expects it.

| Show Persistent Component                                                                                                                                       | Show Persistent Component                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> vlcCart                                                                                                                                | <input checked="" type="checkbox"/> vlcCart                                                                                                                                                                                                                                                                       |
| <pre>"vlcPersistentComponent": {   "vlcCart": {     "totalSize": 0,     "messages": []   } }, [...], "vlcCart_Top": {   "totalSize": 1,   "messages": [],</pre> | <pre>"vlcPersistentComponent": {   "vlcCart": {     "totalSize": 0,     "messages": []   },   "vlcCart_Top": [     {       "totalSize": 1,       "messages": [],       "records": [         {           "messages": [],           "displaySequence": -1,           "details": {             "totalSize": 1,</pre> |

Now that the persistent cart has been configured, you're probably ready to add some products to the cart. Before you can do that, you must create an order, a quote or an opportunity to hold the line items.

---

## Exercise 8-12: Selling Accessories Challenge

### Scenario

Eliza has shared the new guided selling experience with the call center manager. She was thrilled, and yet hungry for more. She asked Eliza to add a new step in the guided selling experience to sell phone accessories.

### Goal

- Review the requirements below and challenge yourself to create a step to select iPhone accessories
- Challenge exercises solidify what you learned in the exercises, build your skills, and allow you to make mistakes -- which is how you will master this application

### Tasks

1. Get accessory products from the shared catalog
2. Create a step to select accessories
3. Test the new accessories step
4. Limit accessories to only iPhone accessories

Time: 20 mins

---

## Task 1: Get accessory products from the shared catalog

The first step is to retrieve the accessories product data from the shared catalog. We've made this a little easier for you by including the accessories object type id in the Get CPQ Object Type Ids reusable OmniScript. See if you can figure out how to find the correct JSON node name for accessories and use that to create a remote action.

## Task 2: Create a step to select accessories

Add a step to the process flow to display the new accessories data.

## Task 3: Test the new accessories step

Does your new step work? Can you see the accessories and add them to the cart?

## Task 4: Limit accessories to only iPhone accessories

Since the phones in the guided selling experience are all Apple products, can you limit the accessories to iPhone accessories?

---

## Appendix A: Digital Commerce LWC OmniScript Sample App

This OmniScript sample app has a number of components which are detailed in this appendix. This appendix only covers components used in the previous exercises.

 OmniScript  
**DCLWCSampleOS**

|      |         |          |         |        |
|------|---------|----------|---------|--------|
| Type | SubType | Language | Version | Active |
| dc   | lwc     | English  | 1       | false  |



initialiseVal...

SelectOffer

ConfigOffer

ShoppingCart

DigitalCom...

☐ Hide Conditional Elements

Theme  
Lightning

 initialiseValues  
Set Values

 SelectOffer  
Step

Step 1 : Select Offer

dcCatalog  
<vlocity\_cmt:dcCatalog />  
dcChildCatalog  
<vlocity\_cmt:dcChildCatalogs />  
dcOffersList  
<vlocity\_cmt:dcOffersList />

 ConfigOffer  
Step

Step 2 : Config Offer

dcOfferConfig  
<vlocity\_cmt:dcOfferConfig />

 ShoppingCart  
Step

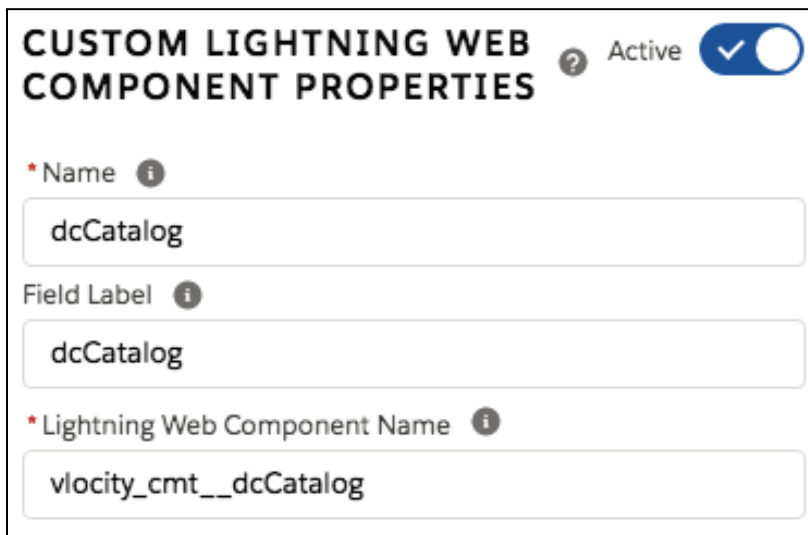
Step 3 : Shopping Cart

dcShoppingCart  
<vlocity\_cmt:dcShoppingCart />

 DigitalCommerceLWCCheckout1

1. **initialiseValues**. Here you are presented with two elements which pass property values to the components in the OmniScript.
  - a. **brainTreePaymentUrl** is the URL for the off-platform payment gateway service that will be used during checkout by the **BraintreepaymentLWC** Lightning web component to handle the user's payment information.
  - b. By establishing a **catalogCode** value at the beginning of our OmniScript, you can pass that value to your subsequent **dcCatalog** Lightning web component. The **dcCatalog** component will then render the UI. Because **dcCatalog** is expecting a **catalogCode**, you cannot leave this blank, otherwise it will not render the UI.
2. **dcCatalog**. This component provides a list of Browse phase catalogs.

This OmniScript component is a **Custom LWC**. The Lightning web component this OmniScript component uses is specified by the **Lightning Web Component Name** field.



**CUSTOM LIGHTNING WEB COMPONENT PROPERTIES** ? Active ☒

\* Name ⓘ  
dcCatalog

Field Label ⓘ  
dcCatalog

\* Lightning Web Component Name ⓘ  
vlocity\_cmt\_\_dcCatalog

- a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES**

This is where the catalog code merge field **%catalogCode%** below **Property Source** is passed to the **catalog-code** property for this **dcCatalog** Lightning

web component. For the user, this represents the selected parent catalog shown on the UI.

**dcCatalog** is one of three Lightning web components that make up the **SelectOffer** step. Each step represents a stage of the buying process and is shown in a step chart. Below is an example of what the step chart can look like.



3. **dcChildCatalog**. This component provides a list of Browse phase child catalogs.

a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component follows a similar pattern as **dcCatalog** to ensure that the child catalog displays the appropriate parent catalog. For the user, this represents the parent catalog shown on the UI just above the child catalogs.

4. **dcOffersList**. This component retrieves all eligible products from the applicable sales catalog.

a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component uses three values to render offers on the page for the user.

**catalog-code** will retrieve offers from the sales catalog associated with this code.

**page-size** determines how many offers show on the page on its initial load. The user can still retrieve all of the offers from the sales catalog by using the UI.

**context** is used to determine which offers are rendered for the user, as determined by context rules.

5. **dcOfferConfig**. This component renders configuration details for the selected product and allows the user to select applicable product attributes prior to adding the configured offer to the cart.

- a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component is passed values from the **SelectOffer** step's **dcOffersList** component for the catalog code, as well as the offer code for the offer/product the user selected. The previous step is identified as **SelectOffer** such as in this value: **%SelectOffer:dcOffersList:catalogCode%**. Similarly, the product **offerCode** is specified.

Finally, the **apiUrl** can be specified. The **apiUrl** refers to a custom domain URL for your potential digital commerce site and is only utilized for anonymous users.

6. **dcShoppingCart**. This component renders the products that have been added to the cart. It also allows the user to modify the cart.

- a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component is passed values from the **SelectOffer** and **ConfigOffer** steps, specifically **dcOffersList** and **dcOfferConfig** components. You probably recognize the catalog code, but not the **cart-context-key** property. This property is generated when offers/products are added to the basket. It is a unique identifier for the basket. It is generated by the **dcOfferConfig** component and passed to this one.

**is-community** is a property for using this app in a Salesforce community site that does not utilize the embedded Checkout OmniScript. If it does not apply, it is set to **false**.

---

Finally, the **apiUrl** can be specified. The **apiUrl** refers to a custom domain URL for your potential digital commerce site and is only utilized for anonymous users.

---

## Appendix B: Digital Commerce LWC Sample Checkout Lightning web components

The last component of the Digital Commerce LWC sample app is a “checkout” OmniScript. This section will cover its Lightning web components. Appendix C will cover its Integration Procedures.

**LWC**

1. **setValues** component.
  - a. By default, **dcReviewOrder** is set up to use the **userName** property.
2. **dcUpdateBillingAddressLWC** component. This component renders the form that allows the user to submit a billing and shipping address. This data is handled by the next component.
3. **BraintreepaymentLWC** component.

- a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component utilizes the **checkoutPaymentUrl** specified in **InitialiseValues** in the DCLWC Sample OS OmniScript. For the sample app, it is pointing to the [Braintree](#) Payment Gateway hosted on Heroku. When it comes to your checkout solution, you will be expected to host or point to the URL of your payment gateway in the parent OmniScript's **initialiseValues** component.

This component renders an interface for the user to enter their credit card details and submit for payment. The OmniScript will perform a remote call to the API to validate the credit card details and confirm payment with the supplied **total** cost provided from **dcShoppingCart**.

4. **ReviewOrder** step.
  - a. Under **CONDITIONAL VIEW**, it is only using this component if **saveCart** does not equal **yes**. If the user opted to save their cart from the Cart page rather than checking out, this OmniScript would not use this step, **updateAddressVIP**, or **SubmitOrderVIP**.

5. **ReviewOrderLWC** component.
  - a. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.** This component utilizes values passed from the **BraintreepaymentLWC** and **updateAddressVIP** components. This component renders billing, shipping, transaction, and user info details for the user to review before submitting their order.

6. **termsConditions** component. This component is a simple Checkbox that is fully editable within OmniScript Designer and [can be configured](#) to prevent the user from advancing past this **ReviewOrder** step.
7. **summary** component. This component presents a confirmation message to the user and utilizes the **orderId** value received from the previous **SubmitOrderVIP** integration procedure.

---

## Appendix C: Digital Commerce LWC Sample Checkout OmniStudio Integration Procedures

The last component of the OmniScript sample app is a “checkout” OmniScript. This section will cover its OmniStudio Integration Procedures.

 OmniScript

**Digital Commerce LWC Checkout**

Pr

Type  
dc

SubType  
lwccheckout

Language  
English

Version  
2

Active  
true



FetchAccou...

setValues

UpdateBillingAddress

updateAddre...

Payment

ReviewOrder

saveCartVIP

SubmitOrd...

orderSummary

navigateTo...

☐ Hide Conditional Elements

Theme  
Lightning

 **FetchAccountDetails**  
Integration Procedure Action

 **setValues**  
Set Values

 **UpdateBillingAddress**  
Step

 **updateAddressVIP**   
Integration Procedure Action

 **Payment**  
Step

 **ReviewOrder**   
Step

 **saveCartVIP**   
Integration Procedure Action

 **SubmitOrderVIP**   
Integration Procedure Action

 **orderSummary**  
Step

 **navigateToOrder**  
Navigate Action

1. **FetchAccountDetails.** This integration allows the user to create a new account or log into their existing account and then passes that account information back into this OmniScript.
  - a. Click the  icon next to **dc\_createAccountLWC**. Some of these components are used to create an account, and others are for existing accounts. They are executed only if they meet their **Execution Conditional Formula**.
  - b. **GetExistingAccount** from the STRUCTURE. Notice that it is using the ContextId as the AccountId in the DataRaptor to retrieve fields from the user's existing account. The **Account** response node is specified in the **SEND / RESPONSE TRANSFORMATIONS** section.
  - c. **SetValues**. This component creates JSON nodes that are used to determine whether certain components in this integration procedure are used. The AccountExists node will be blank if no ContextId is passed into this integration procedure (via the ContextId).
  - d. **getAccountRecordTypeId**. This component gets the account record type ID necessary to create an account. It uses the AccountRecordType passed into this integration procedure via **Extra Payload** in the Checkout OmniScript. Notice the **Execution Conditional Formula** `%SetValues:AccountExists%==''`. This component will only run if the `%SetValues:AccountExists%` node is blank.
  - e. **CreateAccount**. This component is used to create an account using various fields specified in the **Additional Input**. Notice the **Execution Conditional Formula** `%SetValues:AccountExists%==''`. This component will only run if the `%SetValues:AccountExists%` node is blank. We will not be using this component.
  - f. **ResponseActionExistingAccount**. This component determines the integration procedure's response to the Checkout OmniScript. The **Name** and **Phone** fields will be used by **dcReviewOrder**. Notice the **Execution**

g. **ResponseActionNewAccount.** This will similarly return fields like the AccountId back to the Checkout OmniScript for it to complete the order. Notice the **Execution Conditional Formula**

- a. Notice under **CONDITIONAL VIEW**, it is only using this component if **saveCart** equals **yes**. If the user opted to save their cart from the Cart page rather than checking out, this OmniScript **will** use this component.

b. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component utilizes values passed from the main OmniScript for the **catalogCode**, **accountId**, and **cartContextKey**. The **cartContextKey** represents the contents of the user's shopping cart.

- c. Click the  icon next to **dc\_CreateOrder**.

Every component in this integration procedure will execute with the **Execution Conditional Formula** `ISBLANK(%orderId%)` except for **ExistingOrderResponse**. This is because this integration procedure is reused if a user had saved their cart and returned to checkout. In that scenario, it would not need to rerun these order creation components.

- d. **ApexCreateCart** from the STRUCTURE. **createEcomCart** is one of `CpqAppHandler`'s methods. It is responsible for creating an order and it validates cart product pricing with the sales catalog pricelist. **createEcomCart** requires a **catalogCode**, **accountId**, and **cartContextKey**. **createEcomCart** brings everything together for an accurate draft order record.
- e. **GetExistingAccountDetails** from the STRUCTURE. This DataRaptor will bring the user's account details into the Account **Response JSON Node**. It uses an **AccountId** as the input. These details are used by the next component **UpdateOrderBillingAddress** as well as **dcReviewOrder** in the Checkout OmniScript.
- f. **UpdateOrderBillingAddress** from the STRUCTURE. This component uses a DataRaptor to update details (i.e., billing and shipping info) on the newly created order record as identified by the **orderId**.
- g. **NewOrderResponse** from the STRUCTURE. This Response Action returns the data from this procedure to the Checkout OmniScript.
- h. **ExistingOrderResponse** from the STRUCTURE. This action will execute with the **Execution Conditional Formula** `%orderId% !=` which means it will execute if an **orderId** is already present.

4. **SubmitOrderVIP** component.

- a. Notice under **CONDITIONAL VIEW**, it is only using this component if **saveCart** does not equal **yes**. If the user opted to save their cart from the Cart page rather than checking out, this OmniScript would not use this component, **ReviewOrder**, or **updateAddressVIP**.

- b. **CUSTOM LIGHTNING WEB COMPONENT PROPERTIES.**

This component utilizes values passed from the main OmniScript for the **catalogCode**, **accountId**, and **cartContextKey**. The **cartContextKey** represents the contents of the user's shopping cart.

- c. Click the  icon next to **dc\_SubmitOrder**.
- d. **createOrder** from the STRUCTURE. This will ensure an order is created using the exact same integration procedure.

The two following inactive components determine that this integration procedure is configured for CPQ order submission without Order Management.

- e. **SetCreateAssetsValue** from the STRUCTURE. The value that is set here is used as part of the Execution Conditional Formula for **CheckoutWithCPQ**, **DecomposeAndSubmit**, **CheckoutWithCPQAndOM**, and **DecomposeAndSubmitWithCPQAndOM** components.
- f. **canSubmitOrder** from the STRUCTURE. This component uses **SubmitOrderService** with the **cartId** (orderId) to validate the order before submission. This means everything in the order is validated based on rules configured in the Product Console and with [product relationships](#).

The next four components will **not** execute (based on the execution condition formula) unless **canSubmitOrder** returns as true (i.e., order is validated).

- g. **CheckoutWithCPQ** from the STRUCTURE. This Remote Action submits the order using CPQAppHandler and the **checkout** method. It uses the **ContextId** (AccountId), cart/order Id as inputs to submit the order.
- h. **DecomposeAndSubmit** from the STRUCTURE. This component is only used for orgs that are actively using Order Management.
- i. **CheckoutWithCPQAndOM** from the STRUCTURE. This component is only used for orgs that are actively using Order Management and is not applicable to our scenario.
- j. **DecomposeAndSubmitWithCPQAndOM** from the STRUCTURE. This component is only used for orgs that are actively using Order Management.
- k. **SetResponse** from the STRUCTURE. This Response Action returns the data from this procedure to the Checkout OmniScript



**Yay! All done!**